Axion Library Upgrade Master Matrix

Axion Library Upgrade Master Matrix

Library	Current Score	Target Score	Phase	Priority	Required Artifacts	Expected Lift	Main Dependency Blockers
Ops	56	85	1	Critical	ops unit schema, ops registry, decision report, compatibility model, health metrics, doctrine sections OPS-0 to OPS-7	Turns Ops from thin reference layer into real enterprise operations authority	Telemetry producer/consumer mapping, Policy override model, Verification proof mapping
Knowledge	66	86	1	Critical	knowledge unit schema, KID registry, retrieval report, authority tiers, freshness/supersession policy, health metrics	Major retrieval quality and trust improvement; lowers guidance/example confusion	Standards linkage, Templates linkage, Maintenance freshness scans
Templates	69	87	1	Critical	template unit schema, template registry, decision report, placeholder provenance schema, compatibility model, health metrics	Big improvement in output consistency, completeness, and rerender control	Standards applicability, Canonical field mapping, Knowledge support links

Gates	72	88	1	High	gate unit schema, gate registry, gate decision report, evidence/replay contract, drift rules, health metrics	Stronger pass/fail explainability and lower enforcement drift	Standards downstream mapping, Verification proof rules, Policy override model
Maintenance	81	92	2	High	library health contract, doctrine compliance finding schema, health registry, remediation governance docs	Becomes estate governor instead of just maintenance executor	All library registries, common doctrine envelope, policy/mutation linkage
Policy	75	89	2	High	policy unit schema, policy decision report, override/expiry model, compatibility model, policy registry, health metrics	Stronger control of approvals, overrides, and risk-based behavior	System policy hooks, Audit action linkage, Gates override eligibility
Telemetry	78	89	2	Medium-High	signal schema, telemetry registry, decision report, producer/consumer mapping, integrity/routing rules, self-health metrics	Better signal governance and stronger observability confidence	Ops signal consumers, System routing hooks, Privacy/redaction model
Kit	81	90	2	Medium-High	kit unit schema, kit registry, decision report, release classes, proof bundle rules,	Stronger release packaging, certified output tiers,	Verification proof bundles, Standards snapshots, System pin resolution

					compatibility model	better export stability	
Standards	84	92	3	High	richer standards registry, resolution report, downstream mappings, compatibility model, health metrics	Becomes explicit obligation-to-enforcement bridge	Templates clauses, Gates rules, Verification proof mappings, Knowledge supports
Verification	84	92	3	High	proof registry, verification decision report, proof strength tiers, compatibility model, health metrics	Becomes cross-library proof spine	Gates evidence model, Standards obligations, Kit release classes
Planning	81	90	3	Medium-High	plan registry, planning decision report, work-item dependency map, compatibility model, health metrics	Stronger obligation-to-work-to-proof traceability	Canonical entity mapping, Standards obligations, Templates targets
Orchestration	84	91	3	Medium-High	stage registry, stage decision report, execution dependency map, rerun/resume invalidation rules, health metrics	Stronger execution integrity and rerun safety	Planning outputs, Gates bindings, Verification staleness rules

Intake	84	91	3	Medium	intake registry, intake decision report, downstream field mappings, compatibility model, health metrics	Better submission provenance and ambiguity control	Canonical target mapping, Planning scope inputs, Template selection inputs
Canonical	88	94	3	Medium	richer canonical registry, canonical decision report, provenance classes, invalidation map, health metrics	Stronger source-of-truth authority and downstream invalidation discipline	Intake mappings, Planning dependencies, Template field dependencies
Audit	84	91	3	Medium	audit registry enrichment, audit decision report, producer/consumer mapping, compatibility model, health metrics	Stronger mutation accountability and impact trace	Policy approval linkage, Maintenance findings, System event producers
System	88	95	3	Medium	system resolution report, resolver authority contract, cross-library dependency graph, compatibility model, health metrics	Becomes full resolver/control-plane authority	Mature registries from all libraries, final dependency graph model, policy integration

Best sequence inside the matrix

First wave

- Ops
- Knowledge
- Templates
- Gates

These fix the largest governance leaks.

Second wave

- Maintenance
- Policy
- Telemetry
- Kit

These tighten estate-wide control and release discipline.

Third wave

- Standards
- Verification
- Planning
- Orchestration
- Intake
- Canonical
- Audit

- System

These deepen the already-strong core and unify the estate.

Fastest lift per unit of effort

If you want the highest return fastest:

1. **Templates**
2. **Knowledge**
3. **Ops**
4. **Gates**
5. **Maintenance**

Why:

- Templates and Knowledge affect a huge amount of behavior quickly.
- Ops fixes a major enterprise-readiness weakness.
- Gates reduces enforcement ambiguity.
- Maintenance lets you govern everything else more effectively.

Hard blockers to solve early

These are cross-library blockers that will slow everything if left vague:

1. Common governance envelope

Every governed unit should share:

- ID

- version
- owner
- lifecycle state
- applicability
- dependencies
- required proofs
- compatibility label

2. Common compatibility vocabulary

Use the same small set everywhere:

- compatible
- breaking
- migration-required
- rerun-required
- rebuild-required

3. Common dependency edge types

Use one shared model:

- requires
- constrains
- derives_from
- validates

- evidences
- produces_for
- conflicts_with
- supersedes

4. Common decision report pattern

Every major library should emit a decision report for:

- selection
- resolution
- evaluation
- proof
- packaging
- retrieval
- mutation impact

Recommended target state by the end

When this matrix is complete, Axion should be able to answer these questions deterministically:

- Why was this selected?
- What standard required this?
- What proof closes this?
- What changed?
- What broke because of it?

- What must rerun?
- What is stale?
- What is authoritative versus advisory?
- What is release-ready?

Axion Library Upgrade Roadmap

Axion Library Upgrade Roadmap

Objective

Move the Axion library estate from **structurally strong but unevenly governed** to a **fully governed runtime estate** where every library is:

- identifiable
- resolvable
- enforceable
- traceable
- provable
- maintainable

This roadmap is not about adding more libraries.
It is about raising governance quality across the libraries you already have.

Phase 1 — close the biggest governance gaps

Libraries

- Ops
- Knowledge
- Templates
- Gates

Why this phase comes first

These are the weakest or riskiest libraries:

- **Ops** is too thin for enterprise use
- **Knowledge** lacks trust/freshness discipline
- **Templates** has scale without enough governance density
- **Gates** has real enforcement power but underbuilt control richness

These four create the biggest quality and drift risk in the estate.

Phase 1 target outcomes

1. Ops becomes a real doctrine library

By the end of this phase, Ops should have:

- full section doctrine
- governed unit registry
- applicability rules
- proof rules
- compatibility labels
- drift checks
- ops health metrics

2. Knowledge becomes a governed retrieval authority

By the end of this phase, Knowledge should have:

- KID registry

- authority tiers
- freshness states
- applicability and retrieval rules
- supersession rules
- knowledge health metrics

3. Templates becomes a governed template estate

By the end of this phase, Templates should have:

- first-class template registry
- lifecycle states
- applicability rules
- placeholder provenance
- proof requirements
- compatibility labels
- template health metrics

4. Gates becomes a governed evaluation authority

By the end of this phase, Gates should have:

- gate registry
- applicability predicates
- decision report contract
- evidence requirements
- compatibility labels

- drift rules
- gate health metrics

Phase 1 artifacts to add

Ops

- `OPS-0_purpose.md`
- `OPS-1_monitoring_alert_standards.md`
- `OPS-2_logging_tracing_standards.md`
- `OPS-3_slo_sla_error_budget_policy.md`
- `OPS-4_perf_budgets_profiling.md`
- `OPS-5_cost_models_quota_hooks.md`
- `OPS-6_ops_gates_evidence.md`
- `OPS-7_minimum_viable_ops.md`
- `contracts/ops_unit.schema.json`
- `contracts/ops_decision_report.schema.json`
- `registries/ops_registry.v1.json`
- `registries/ops_metrics_catalog.v1.json`

Knowledge

- `KNO-0_purpose.md`

- `KNO-1_unit_classes.md`
- `KNO-2_authority_tiers.md`
- `KNO-3_freshness_supersession.md`
- `KNO-4_retrieval_resolution_rules.md`
- `KNO-5_dependency_mapping.md`
- `KNO-6_proof_and_trust.md`
- `KNO-7_definition_of_done.md`
- `contracts/knowledge_unit.schema.json`
- `contracts/knowledge_retrieval_report.schema.json`
- `registries/knowledge_registry.v1.json`

Templates

- `TMP-0_purpose.md`
- `TMP-1_template_registry_rules.md`
- `TMP-2_applicability_selection.md`
- `TMP-3_placeholder_provenance.md`
- `TMP-4_completeness_and_proofs.md`
- `TMP-5_backcompat_and_migrations.md`
- `TMP-6_template_health.md`
- `contracts/template_unit.schema.json`

- `contracts/template_decision_report.schema.json`
- `contracts/placeholder_provenance.schema.json`
- `registries/template_registry.v1.json`

Gates

- `GAT-0_purpose.md`
- `GAT-1_gate_registry_rules.md`
- `GAT-2_applicability_and_stage_bindings.md`
- `GAT-3_decision_report.md`
- `GAT-4_evidence_and_replay.md`
- `GAT-5_backcompat_and_drift.md`
- `GAT-6_gate_health.md`
- `contracts/gate_unit.schema.json`
- `contracts/gate_decision_report.schema.json`
- `registries/gate_registry.v1.json`

Governance lift expected from Phase 1

This phase should deliver the largest visible jump in estate quality because it raises the weakest libraries toward the rest of the estate.

Expected result:

- fewer blind spots
- better retrieval quality
- better template control
- better evaluation traceability
- real ops maturity
- lower drift risk

Phase 2 — strengthen estate-wide control surfaces

Libraries

- Policy
- Maintenance
- Telemetry
- Kit

Why this phase comes second

Once the biggest weak spots are stabilized, the next step is strengthening the libraries that govern:

- permissions
- mutation control
- signal visibility

- packaging/release quality

These are the estate-wide control surfaces.

Phase 2 target outcomes

1. Policy becomes a full decision authority

Policy should gain:

- decision report contracts
- lifecycle and supersession rules
- compatibility labels
- drift checks
- policy health metrics

2. Maintenance becomes the meta-governor

Maintenance should gain:

- per-library doctrine compliance checks
- unified health contracts
- stronger mutation and lifecycle enforcement
- better finding/remediation outputs

3. Telemetry becomes a governed signal authority

Telemetry should gain:

- signal registry enrichment

- producer/consumer mapping
- signal compatibility labels
- telemetry integrity proofs
- self-health metrics

4. Kit becomes a proof-backed release envelope

Kit should gain:

- release classes
- proof bundle requirements
- stronger package compatibility rules
- kit health metrics

Phase 2 artifacts to add

Policy

- `POL-0_purpose.md`
- `POL-1_policy_unit_model.md`
- `POL-2_decision_report.md`
- `POL-3_override_expiry_supersession.md`
- `POL-4_backcompat_and_migration.md`
- `POL-5_policy_health.md`
- `contracts/policy_unit.schema.json`

- `contracts/policy_decision_report.schema.json`
- `registries/policy_registry.v1.json`

Maintenance

- `MNT-8_library_health_contract.md`
- `MNT-9_doctrine_compliance_rules.md`
- `MNT-10_backcompat_validation.md`
- `MNT-11_registry_rebuild_integrity.md`
- `MNT-12_remediation_and_patch_governance.md`
- `contracts/library_health_report.schema.json`
- `contracts/doctrine_compliance_finding.schema.json`
- `registries/library_health_registry.v1.json`

Telemetry

- `TEL-0_purpose.md`
- `TEL-1_signal_registry_rules.md`
- `TEL-2_producer_consumer_mapping.md`
- `TEL-3_signal_integrity_and_routing.md`
- `TEL-4_backcompat_and_redaction.md`
- `TEL-5_telemetry_health.md`

- `contracts/telemetry_signal.schema.json`
- `contracts/telemetry_decision_report.schema.json`
- `registries/telemetry_registry.v1.json`

Kit

- `KIT-0_purpose.md`
- `KIT-1_release_classes.md`
- `KIT-2_package_dependency_rules.md`
- `KIT-3_proof_bundle_requirements.md`
- `KIT-4_backcompat_and_consumer_contracts.md`
- `KIT-5_kit_health.md`
- `contracts/kit_unit.schema.json`
- `contracts/kit_decision_report.schema.json`
- `registries/kit_registry.v1.json`

Governance lift expected from Phase 2

This phase should make the whole estate feel more controlled and release-worthy.

Expected result:

- clearer approvals and denials
- maintenance becomes useful as a governor, not just a scanner

- live signal governance improves
- package quality and compatibility become much clearer

Phase 3 — deepen the already strong core

Libraries

- Planning
- Audit
- Standards
- Verification
- Orchestration
- Intake
- Canonical
- System

Why this phase comes third

These libraries are already relatively strong. They do not need rescue work.
They need **deepening**:

- richer dependency mapping
- richer compatibility semantics
- better health metrics
- stronger cross-library authority

This phase turns the strong core into a fully governed core.

Phase 3 target outcomes

Planning

Becomes the explicit bridge from obligation to work to proof.

Audit

Becomes the mutation accountability and impact-trace authority.

Standards

Becomes the full obligation-to-enforcement bridge.

Verification

Becomes the universal proof spine.

Orchestration

Becomes the execution dependency and invalidation authority.

Intake

Becomes the submission provenance and ambiguity authority.

Canonical

Becomes the provenance and downstream invalidation authority.

System

Becomes the full resolver authority for the entire estate.

Phase 3 artifacts to add

Planning

- richer plan registry
- planning decision report
- upstream/downstream work-item mappings
- planning compatibility labels
- plan drift/health rules

Audit

- richer audit registry
- audit decision/report contract
- producer/consumer mapping
- audit compatibility labels
- audit health metrics

Standards

- richer standards registry
- standards resolution report
- standard → template/gate/proof mappings
- standards compatibility labels
- standards health metrics

Verification

- proof registry
- verification decision report

- proof strength tiers
- proof compatibility labels
- verification health metrics

Orchestration

- orchestration stage registry
- stage decision report
- execution dependency map
- rerun/resume invalidation rules
- orchestration health metrics

Intake

- intake registry
- intake decision report
- intake → canonical/planning/template mappings
- intake compatibility labels
- intake health metrics

Canonical

- canonical registry enrichment
- canonical decision report
- provenance class model
- downstream invalidation map

- canonical health metrics

System

- system resolution report
- resolver authority contract
- cross-library dependency graph
- system compatibility labels
- system health metrics

Governance lift expected from Phase 3

This phase is where Axion becomes fully coherent end to end.

Expected result:

- strong provenance
- strong invalidation logic
- strong proof closure
- stronger rerun/resume integrity
- stronger resolver control
- fewer hidden dependencies

Cross-phase foundation rules

These should apply in every phase.

Rule 1 — every governed unit gets the same minimum envelope

Every unit should have:

- ID
- version
- owner
- lifecycle state
- applicability
- dependencies
- required proofs
- compatibility label

Rule 2 — every library gets a registry

No major library should rely on inventory-by-folder alone.

Rule 3 — every important decision gets a report

You should have explicit reports for:

- selection
- resolution
- evaluation
- proof sufficiency
- packaging
- retrieval

- mutation impact

Rule 4 — every library gets health metrics

Not just logs. Actual health indicators.

Rule 5 — maintenance enforces doctrine

Maintenance should score and scan every library against the common governance doctrine.

Suggested execution order inside each phase

Phase 1 order

1. Ops
2. Knowledge
3. Templates
4. Gates

Phase 2 order

1. Maintenance
2. Policy
3. Telemetry
4. Kit

Phase 3 order

1. Standards
2. Verification
3. Planning
4. Orchestration
5. Intake
6. Canonical
7. Audit
8. System

I put **Maintenance first in Phase 2** because once it is stronger, it can help police the rest of the roadmap.

I put **System last in Phase 3** because it should absorb the final, fully matured cross-library dependency and resolution model after the other libraries are tightened.

Final roadmap outcome

If you execute this roadmap well, Axion should end up with:

- a fully governed library estate
- deterministic selection and resolution across libraries
- stronger trust and freshness controls
- explicit compatibility semantics
- better library health visibility
- better blast-radius analysis

- better proof-backed release quality
- much lower drift risk

The overall transformation is:

from: well-structured documentation and contracts
to: governed runtime authorities with maintenance-enforced compliance

Axion All-Libraries Upgrade Matrix

Axion All-Libraries Upgrade Matrix

Maturity tiers

Tier A — structurally strong

These already behave like real libraries, not loose notes.

- **intake**
- **canonical**
- **standards**
- **planning**
- **orchestration**
- **verification**
- **kit**
- **audit**
- **telemetry**
- **system**

Tier B — useful but governance-thin

These have the right direction, but need stronger runtime enforcement.

- **policy**
- **gates**
- **maintenance**

Tier C — needs major improvement

These are the biggest leverage points.

- **ops**
- **knowledge**
- **templates**

That last one may sound surprising because templates are large, but the issue is not volume. It is **governance density per template**.

Library-by-library matrix

1. Intake

Current state: Strong and disciplined.
What it already does well: form spec, enums, validation rules, submission record, determinism material.
Main gap: needs tighter downstream traceability.

Upgrade targets

- Link intake fields directly to canonical fields.
- Add field-level dependency map into standards applicability.
- Add “input ambiguity classes” so uncertain intake data becomes explicit canonical unknowns.
- Add proof requirements for normalization and validation outcomes.

Priority: Medium

---

2. Canonical

Current state: Strong.
What it already does well: ID rules, dedupe, integrity, unknowns/assumptions, schema shape.
Main gap: canonical truth needs to drive more of the estate.

Upgrade targets

- Make canonical entities the required source for planning decomposition.
- Add stronger references from templates to canonical paths.
- Separate:
    - hard facts
    - inferred facts
    - unresolved unknowns
- Add canonical impact rules: “if canonical changes, these artifacts become stale.”

Priority: High

---

3. Standards

Current state: Strong base.
What it already does well: pack model, categories, resolution rules, snapshots, gate mapping.
Main gap: applicability and precedence need to become more executable.

Upgrade targets

- Add explicit applicability predicates.
- Add conflict and override logic per standard.
- Add dependency edges:
    - standard → gate
    - standard → template clause
    - standard → proof type
- Add deprecation/supersession chain.

Priority: High

---

4. Templates

Current state: Large and content-rich, but governance needs to catch up.
What it already does well: broad coverage and category depth.
Main gap: templates look more like content inventory than tightly governed executable assets.

Upgrade targets

- Give every template:
    - stable ID
    - version
    - owner
    - status
    - applicability
    - required inputs
    - required standards
    - required proofs
- Add section-level completion rules.
- Add placeholder provenance rules.
- Add “forbidden guessing” zones.
- Add backcompat tiers for template revisions.

Priority: Very high

---

5. Planning

Current state: Strong.
What it already does well: work breakdown, acceptance map, build plan, sequencing, coverage.
Main gap: plan coverage should be more visibly tied to standards and verification.

Upgrade targets

- Require each plan item to map to:
    - canonical entity
    - template output
    - standards obligation
    - acceptance evidence
- Add stale-plan detection when upstream libraries change.
- Add plan gap classes: unmapped, under-scoped, unverifiable.

Priority: Medium-high

---

6. Orchestration

Current state: Strong.
What it already does well: stage IO, run manifest, stage reports, rerun/resume, evidence format.
Main gap: should become the formal dependency spine across all libraries.

Upgrade targets

- Make orchestration the authoritative map of:
    - library consumption
    - artifact production
    - gate evidence origins
- Add rerun invalidation rules.
- Add exact stage-to-library dependency declarations.

Priority: High

---

7. Gates

Current state: decent foundation, but thinner than needed.
What it already does well: gate definition model, DSL, runtime, gate reports, replay.
Main gap: gates need richer cross-library binding.

Upgrade targets

- Every gate should declare:
  - governing standards
  - validated artifact classes
  - required proofs
  - error taxonomy
  - replay contract
- Add predicate registry maturity tiers.
- Add gate explainability model for operator review.

Priority: High

---

8. Verification

Current state: Strong.
What it already does well: proof log, proof types, command policy, completion criteria.
Main gap: should validate library quality too, not only run completion.

Upgrade targets

- Add proof classes for library validity:
  - schema pass
  - dependency integrity
  - backcompat pass
  - coverage sufficiency
  - stale reference detection
- Add reusable proof bundles for maintenance modes.

Priority: High

---

9. Kit

Current state: Strong.
What it already does well: manifest, tree, entrypoint, versions, export rules, gates.
Main gap: compatibility contracts can be stronger.

Upgrade targets

- Add explicit compatibility matrix:
  - kit version ↔ standards snapshot
  - kit version ↔ template registry
  - kit version ↔ proof policy
- Add release class tags: dev, candidate, certified, enterprise-ready.

Priority: Medium

---

10. Policy

Current state: useful, but comparatively thin.
What it already does well: risk classes, overrides, precedence, enforcement points.
Main gap: policy should actively govern more runtime behavior.

Upgrade targets

- Make policy drive:
  - template selection limits
  - required proof thresholds
  - override expiry
  - approval classes
  - change blast-radius thresholds
- Add policy simulation mode before enforcement.

Priority: High

---

11. Telemetry

Current state: solid.
What it already does well: event model, run metrics, sink policy, privacy/redaction, telemetry gates.
Main gap: not enough library observability.

Upgrade targets

- Track:
  - template selection frequency
  - stale standard usage
  - deprecated item hits

    - orphaned knowledge entries
    - gate failure by library
- Add library health dashboards.

Priority: Medium-high

---

12. System

Current state: strong infrastructure layer.
What it already does well: workspace/project model, pin policies, adapter manager, quotas, routing, capability registry.
Main gap: system should be the resolver authority for the whole library estate.

Upgrade targets

- Formalize library resolution order.
- Add pin/lock behavior per library type.
- Add capability-based library filtering.
- Add deterministic library loader report.

Priority: High

---

13. Audit

Current state: strong support library.
What it already does well: action model, log model, index model, integrity rules, gate checks.
Main gap: should capture more library mutation context.

Upgrade targets

- Record every library change with:
    - actor
    - why
    - affected IDs
    - blast radius
    - backcompat result
    - remediation path
- Add mutation classes: create, revise, supersede, deprecate, retire.

Priority: Medium-high

---

14. Ops

Current state: the weakest formal library.
What it already does well: basic schemas for alerts, cost, budgets, SLO, logging.
Main gap: too shallow relative to how important ops is.

Upgrade targets

- Expand ops into a full doctrine library with:
  - purpose
  - boundaries
  - determinism rules
  - validation checklist
  - evidence requirements
  - gate mapping
  - minimum viable ops set
- Split ops into governed areas:
  - observability
  - logging/tracing
  - alerting
  - SLO/error budgets
  - perf budgets
  - load/capacity
  - cost/quota policy hooks

Priority: Very high

---

15. Knowledge

Current state: huge and strategically important.
What it already does well: breadth, bundles, indices, taxonomy, pillar organization.
Main gap: knowledge needs stronger runtime governance, not more raw content.

Upgrade targets

- Make each KID a governed unit with:
  - stable ID
  - type
  - quality tier
  - authority tier

    - scope
    - tags
    - linked standards
    - linked templates
    - linked proofs
    - lifecycle status
- Split knowledge clearly into:
    - authoritative rules
    - guidance
    - examples
    - anti-patterns
    - references
- Add freshness and supersession logic.
- Add retrieval policy based on run profile and risk class.

Priority: Very high

---

16. Maintenance

Current state: strong meta-direction, but should become the estate governor.
What it already does well: contracts and maintenance-mode framing.
Main gap: it should actively evaluate all other libraries.

The mode set you locked already points to the right operating model, especially **Coverage Audit, Drift Detection, Backcompat Validation, Rebuild Indexes/Registries, and Registry Integrity Cleanup**.

Response: Axion Updates Planning

Upgrade targets

- Make maintenance the governing layer for library health.
- Add one health contract for every library:
    - completeness
    - dependency integrity
    - stale reference scan
    - supersession hygiene
    - backcompat status
    - proof sufficiency
- Add automatic findings output for each maintenance mode.
  Response: Axion Updates Planning

Priority: Very high

Cross-library problems the scan surfaced

1. Governance depth is uneven

Some libraries have a full doctrine pattern. Others mostly expose raw JSON or large content collections.

2. Dependency links are weaker than they need to be

The libraries are individually strong, but the system needs more first-class declarations of:

- who consumes what
- what validates what
- what breaks when something changes

3. Content scale is outpacing control scale

Most visible in:

- templates
- knowledge

You have a lot of content, but the control plane around that content needs to be stricter.

4. Ops is underbuilt

This is the clearest structural weakness.

5. Maintenance should become the library governor

Not just another library. The maintenance modes should be how Axion continuously checks the estate.

Response: Axion Updates Planning

Best improvement order

Phase 1 — governance spine

Do these first:

1. **maintenance**
2. **system**
3. **policy**
4. **gates**
5. **verification**

Reason: these libraries define how the rest of the estate gets governed.

Phase 2 — high-value content control

Then:
6. **templates**
7. **knowledge**
8. **standards**
9. **ops**

Reason: these are the biggest leverage and biggest risk.

Phase 3 — downstream tightening

Then:
10. **canonical**
11. **planning**
12. **orchestration**
13. **telemetry**
14. **audit**
15. **kit**
16. **intake**

Reason: these are already relatively solid and benefit most after governance is tightened upstream.

The main strategic takeaway

Axion does **not** need more libraries first.

It needs:

- stricter cross-library dependency mapping
- stronger per-item metadata
- proof-backed change control
- maintenance-driven quality enforcement
- better governance around templates and knowledge
- a major elevation of ops

That is the right perspective to apply across the whole estate.

All-Libraries Governance Doctrine

Axion All-Libraries Governance Doctrine

ALG-0 — Purpose

Define one universal governance standard for all Axion libraries so they behave as:

- identifiable assets
- selectable runtime inputs
- enforceable contracts
- traceable dependencies
- provable outputs
- maintainable system components

This doctrine applies to every library, including:

- intake
- canonical
- standards
- templates
- planning
- orchestration
- gates
- verification
- kit
- policy
- telemetry
- system
- audit
- ops
- knowledge
- maintenance

ALG-1 — Core rule

No library is “just documentation.”

Every library must function as a governed system surface with:

- identity

- scope
- schema or structure
- selection rules
- dependency rules
- lifecycle rules
- proof rules
- maintenance coverage

If a library cannot be selected, validated, traced, and maintained, it is incomplete.

ALG-2 — What every library unit must be

Every governed unit inside any library must have these minimum fields:

Required identity

- `library_id`
- `unit_id`
- `title`
- `version`
- `status`
- `owner`
- `created_at`
- `updated_at`

Required governance

- `purpose`
- `scope`
- `authority_tier`
- `risk_class`
- `applicability`
- `dependencies`
- `dependents`
- `required_proofs`
- `maintenance_class`

Required lifecycle

- `supersedes`
- `superseded_by`
- `deprecated_at`
- `retired_at`
- `change_summary`

This is the minimum governance envelope.

ALG-3 — Standard lifecycle states

Every unit must live in one of these states:

- `draft`
- `candidate`
- `active`
- `restricted`
- `deprecated`
- `retired`
- `superseded`

Rules

- Only `active` or `restricted` units may be used in production runs.
- `deprecated` units may resolve only if policy allows them.
- `retired` units may not resolve.
- `superseded` units must point to replacement units.

ALG-4 — Authority tiers

Every library unit must declare how authoritative it is.

Tier model

- `A0` — system-critical / binding
- `A1` — production-authoritative
- `A2` — recommended guidance
- `A3` — reference / example / informative

- `A4` — experimental

Enforcement

- `A0` and `A1` may drive gates and hard requirements.
- `A2` may guide generation and review.
- `A3` may inform but never block.
- `A4` cannot affect release decisions without explicit policy override.

This is especially important for **knowledge**, **standards**, and **templates**.

ALG-5 — Applicability contract

Every unit must declare when it applies.

Minimum applicability fields

- project type
- artifact class
- stage(s)
- risk classes
- profile / mode
- environment constraints
- exclusions
- precedence notes

Rule

If applicability is not explicit, the unit is not eligible for deterministic resolution.

This is one of the biggest upgrades for Standards and Templates.

ALG-6 — Dependency contract

Every unit must explicitly declare dependencies.

Allowed dependency types

- `requires`

- `constrains`
- `derives_from`
- `validates`
- `evidences`
- `supersedes`
- `conflicts_with`
- `produces_for`

Example dependency chains

- intake field → canonical field
- canonical entity → planning unit
- standard → gate
- standard → template section
- template → proof requirement
- maintenance rule → registry item

Rule

Undeclared critical dependencies are governance failures.

ALG-7 — Resolution contract

Every library must be resolvable by deterministic rules.

Every library must define

- resolver inputs
- selection order
- precedence
- tie-break rules
- pin / lock behavior
- fallback policy
- failure modes

Required output

Every selection event must produce a **resolution report**:

- what was requested
- what was selected

- why it matched
- what was excluded
- what policy affected it
- what version was pinned

This matters most for:

- standards resolution
- template selection
- knowledge retrieval
- policy application
- system loader behavior

ALG-8 — Proof contract

Every library must define what proves it is valid and usable.

Minimum proof classes

- schema validity
- structural completeness
- dependency integrity
- applicability validity
- backcompat status
- stale reference check
- registry inclusion
- usage safety

Rule

A library unit without defined proof requirements cannot be promoted to `active`.

This is where Verification becomes the universal proof layer, not just a run-completion layer.

ALG-9 — Change control contract

Every change to any governed unit must create a mutation record.

Required mutation classes

- create
- revise
- deprecate
- supersede
- retire
- restore
- migrate

Required mutation payload

- actor
- reason
- affected units
- dependency blast radius
- backcompat result
- required reruns
- proof deltas
- approval class

This ties directly into Audit and Maintenance.

ALG-10 — Backcompat doctrine

Every library must define its compatibility behavior.

Minimum backcompat labels

- `patch-compatible`
- `minor-compatible`
- `breaking`
- `migration-required`
- `legacy-only`

Required rule

If a unit changes, Axion must be able to answer:

- what old consumers still work
- what breaks
- what needs migration
- what must be replayed or rebuilt

That is exactly why **Backcompat Validation** must exist as a formal maintenance enforcement surface.

Response: Axion Updates Planning

ALG-11 — Registry doctrine

Every library must expose a registry.

Registry minimum

- stable IDs
- versions
- statuses
- owners
- applicability keys
- dependency edges
- proof state
- lifecycle state

Rules

- unregistered active units are invalid
- duplicate active IDs are invalid
- broken supersession chains are invalid
- missing owner on active unit is invalid

This ties directly to **Rebuild Indexes/Registries** and **Registry Integrity Cleanup**.

Response: Axion Updates Planning

ALG-12 — Maintenance doctrine

Every library must be evaluated by Maintenance.

At minimum, every library must be compatible with these maintenance modes:

- **MM-03 Coverage Audit**
- **MM-04 Drift Detection**
- **MM-10 Backcompat Validation**

- **MM-12 Rebuild Indexes/Registries**
- **MM-13 Registry Integrity Cleanup**
  Response: Axion Updates Planning

What those mean in doctrine form

Coverage Audit

Checks whether the library has all required:

- units
- registries
- proofs
- dependency declarations
- lifecycle fields
- maintenance hooks

Drift Detection

Checks whether:

- content changed without version movement
- active units diverged from registry
- dependencies point to stale targets
- runtime usage diverged from declared policy

Backcompat Validation

Checks whether:

- prior consumers still work
- migration rules exist when needed
- replay/rerender requirements are defined
- compatibility labels are honest

Rebuild Indexes/Registries

Reconstructs:

- registry state
- dependency graph
- applicability index
- supersession graph
- health summaries

Registry Integrity Cleanup

Repairs or flags:

- duplicate IDs
- orphaned units
- broken links
- invalid lifecycle chains
- stale active entries

Maintenance should be the estate governor, not just another library.

ALG-13 — Library health score

Each library should receive a governance score across these dimensions:

- identity completeness
- registry integrity
- applicability precision
- dependency clarity
- proof sufficiency
- backcompat discipline
- maintenance readiness
- observability coverage

Suggested output

- `Green` — production-governable
- `Yellow` — usable but weak
- `Red` — structurally incomplete

ALG-14 — Minimum doctrine compliance

A library is only doctrine-compliant if it has all of these:

1. purpose and scope
2. unit identity model
3. lifecycle model
4. applicability rules
5. dependency rules
6. registry/index
7. proof requirements

8. change-control rules
9. maintenance coverage
10. explicit invalid-state definitions

If one of those is missing, the library is not fully governed.

How this doctrine applies by library

Templates

Need the strongest push on:

- unit identity
- applicability
- required inputs
- placeholder provenance
- proof requirements
- backcompat labels

Knowledge

Needs:

- authority tiers
- freshness and supersession logic
- retrieval policy
- stronger dependency links to standards/templates/proofs

Standards

Need:

- executable applicability
- precedence rules
- override behavior
- downstream traceability into gates and templates

Ops

Needs:

- full doctrine adoption almost end to end
- stronger proofs
- stronger gate linkage
- stronger maintenance coverage

Maintenance

Should enforce doctrine against all other libraries using the maintenance mode set you already locked.

Response: Axion Updates Planning

What this gives Axion

If you adopt this doctrine, Axion will be able to answer questions like:

- What library units are active right now?
- Why was this template selected?
- Which standards constrained this build?
- What broke when this rule changed?
- Which knowledge items are authoritative vs advisory?
- What must be rebuilt after a registry repair?
- Which library is drifting from doctrine?

That is the shift from “good documentation” to “governed runtime estate.”

Library Compliance Checklist

Axion Library Compliance Checklist

Use this against **every library**:

- intake
- canonical
- standards
- templates
- planning
- orchestration
- gates
- verification
- kit
- policy
- telemetry
- system
- audit
- ops
- knowledge
- maintenance

Score each item:

- `0` = missing
- `1` = partial
- `2` = compliant

Then total by library.

ALC-01 — Identity

Every governed unit in the library should have:

- stable library ID
- stable unit ID
- title
- version
- owner
- status
- created date

- updated date

Fail conditions

- active item has no ID
- duplicate active IDs
- active item has no owner
- version missing

ALC-02 — Purpose and scope

Every library and every governed unit should define:

- purpose
- boundaries
- what it governs
- what it does not govern
- intended consumers
- intended stages / modes

Fail conditions

- scope is implied but not stated
- overlap with another library is unresolved
- unit has no clear consumer

ALC-03 — Lifecycle

Every unit should declare lifecycle state:

- draft
- candidate
- active
- restricted
- deprecated
- retired
- superseded

And also:

- supersedes

- superseded by
- deprecation notes
- retirement rules

Fail conditions

- deprecated item has no replacement path
- retired item still resolves
- active item has no lifecycle state

ALC-04 — Authority and trust tier

Every unit should declare authority tier:

- A0 system-critical
- A1 production-authoritative
- A2 recommended guidance
- A3 reference/example
- A4 experimental

And whether it is:

- binding
- advisory
- informative only

Fail conditions

- knowledge-like content presented as binding without authority tier
- experimental content can affect release without policy approval
- advisory content is used as a gate rule

ALC-05 — Applicability

Every unit should state when it applies:

- project type
- artifact class
- stage
- risk class
- environment

- profile / mode
- exclusions
- precedence notes

Fail conditions

- item can resolve without explicit applicability
- two active items match with no tie-break rule
- exclusions are undocumented

ALC-06 — Dependency mapping

Every unit should declare:

- requires
- constrains
- derives from
- validates
- evidences
- conflicts with
- produces for

Fail conditions

- critical dependency exists only in prose
- broken links to other units
- downstream consumers unknown
- supersession chain broken

This section is what **MM-04 Drift Detection** and **MM-13 Registry Integrity Cleanup** should catch most often.

Response: Axion Updates Planning

Response: Axion Updates Planning

ALC-07 — Resolution rules

Each library should define how units are selected:

- resolver inputs

- selection order
- precedence rules
- tie-break rules
- pin / lock rules
- fallback behavior
- failure behavior

Fail conditions

- selection depends on operator intuition
- resolution is nondeterministic
- pinning behavior undefined
- library can silently fall back

ALC-08 — Structure and schema

Each library should define its governed structure:

- schema or contract
- required fields
- optional fields
- valid enums
- invalid states
- serialization format
- registry representation

Fail conditions

- active content has no schema
- schema does not match actual units
- invalid states are not defined
- unit passes without required fields

ALC-09 — Proof requirements

Each unit should declare what proves it is safe to use:

- schema valid
- structurally complete
- dependencies valid

- applicability valid
- registered
- compatible
- safe for current lifecycle state

Fail conditions

- no proof policy for active units
- proof cannot be linked to version
- proof types vary without reason

This is the layer that **MM-03 Coverage Audit** and **MM-10 Backcompat Validation** should force into the open.

Response: Axion Updates Planning

Response: Axion Updates Planning

ALC-10 — Change control

Every mutation should produce a recorded change event with:

- actor
- reason
- change class
- affected units
- blast radius
- required reruns
- approval class
- proof delta
- migration need

Fail conditions

- active content changed without mutation record
- breaking changes have no blast-radius statement
- approvals not captured

ALC-11 — Backcompat

Each library should define compatibility labels:

- patch-compatible
- minor-compatible
- breaking
- migration-required
- legacy-only

And answer:

- what still works
- what breaks
- what needs migration
- what needs rerun / rebuild

Fail conditions

- version changed without compatibility label
- breaking change has no migration path
- consumers cannot determine support status

This is directly aligned with **MM-10 Backcompat Validation**.

Response: Axion Updates Planning

ALC-12 — Registry quality

Every library should have a registry or index containing:

- IDs
- versions
- statuses
- owners
- applicability keys
- dependency edges
- proof state
- lifecycle state

Fail conditions

- active items missing from registry
- duplicate entries
- stale registry state
- orphaned units
- registry cannot be rebuilt from source truth

This is what **MM-12 Rebuild Indexes/Registries** and **MM-13 Registry Integrity Cleanup** are for.

Response: Axion Updates Planning

ALC-13 — Drift resistance

Each library should be able to detect drift between:

- source files and registry
- declared version and actual content
- declared dependencies and live targets
- declared applicability and real usage
- policy intent and runtime selection

Fail conditions

- content changed with no version movement
- resolution behavior changed silently
- runtime uses deprecated items without visibility

This is the job of **MM-04 Drift Detection**.

Response: Axion Updates Planning

ALC-14 — Maintenance readiness

Each library should be compatible with at least these maintenance modes:

- MM-03 Coverage Audit
- MM-04 Drift Detection
- MM-10 Backcompat Validation
- MM-12 Rebuild Indexes/Registries
- MM-13 Registry Integrity Cleanup
  Response: Axion Updates Planning

For each library, ask:

- Can Coverage Audit evaluate it?
- Can Drift Detection compare it?
- Can Backcompat Validation test it?

- Can its registry be rebuilt?
- Can integrity cleanup repair or flag it?

Fail conditions

- library cannot be audited automatically
- no registry exists to rebuild
- no integrity model exists to clean up

ALC-15 — Observability

Each library should emit or support metrics for:

- resolution count
- failure count
- deprecated usage
- drift findings
- orphan count
- backcompat failures
- proof pass/fail
- stale unit count

Fail conditions

- no visibility into library usage
- failures only visible in logs or manually
- deprecated content usage is invisible

ALC-16 — Operator clarity

The library should make it easy for an operator to answer:

- what this unit is
- why it was selected
- what it depends on
- what constrains it
- what proves it is valid
- what changed last
- what replaces it

Fail conditions

- operator must inspect raw files to understand resolution
- no explanation path exists
- no replacement path for deprecated items

Per-library add-ons

Templates

Add checks for:

- required inputs defined
- placeholder provenance defined
- section completeness rules
- forbidden guessing rules
- linked standards
- linked proof types
- render envelope compatibility

Knowledge

Add checks for:

- authority tier
- freshness
- supersession logic
- retrieval tags
- binding vs advisory split
- linked standards/templates/proofs
- anti-pattern coverage

Standards

Add checks for:

- applicability predicates
- precedence rules
- override rules
- conflict handling
- gate mapping

- template mapping
- proof obligations

Ops

Add checks for:

- alert policy coverage
- logging/tracing standards
- SLO/error budget rules
- perf budget rules
- cost/quota hooks
- ops evidence requirements
- ops gate linkage

Maintenance

Add checks for:

- health model per library
- finding schema
- remediation schema
- mode-to-library coverage
- registry repair support
- backcompat scan support

Suggested scoring bands

For each library:

- `90–100%` = Governed
- `75–89%` = Usable but thin
- `50–74%` = At risk
- `<50%` = Structural gap

And for each failed item, classify it as:

- `critical` — can break deterministic behavior
- `major` — weakens governance or traceability

- `minor` — cleanup / maturity issue

Best way to use this now

Run this checklist in this order:

1. maintenance
2. system
3. policy
4. gates
5. verification
6. templates
7. knowledge
8. standards
9. ops

That sequence will expose the highest-value gaps first because it starts with governance and then moves into the largest content surfaces.

Maintenance library scorecard

Maintenance library scorecard

Overall grade

26 / 32 = 81%
Band: Usable but thin

That means the Maintenance library is one of the stronger governance libraries, but it is still not yet the full estate governor it needs to become.

It already has the right backbone:

- a locked maintenance mode set with **MM-03 Coverage Audit**, **MM-04 Drift Detection**, **MM-10 Backcompat Validation**, **MM-12 Rebuild Indexes/Registries**, and **MM-13 Registry Integrity Cleanup** in the canonical list
  Response: Axion Updates Planning
- a purpose/boundary file
- policies
- contracts
- registries
- seeded gate, detector, patch, and schedule assets

That is good structure. The gaps are mostly in **cross-library enforcement depth**, **lifecycle richness**, and **backcompat/change-control maturity**.

ALC score breakdown — Maintenance

ALC-01 Identity

Score: 2/2

Present:

- maintenance library purpose file has `doc_id`, `library`, `version`, `status`
- registries use stable IDs
- modes, gates, detector packs, patch types, schedules all have IDs and versions

Why it passes:

- IDs are explicit and systematic
- versioning is present
- status is present on seeded registry items

Main weakness:

- owner metadata is not consistently visible in the seeded units

---

ALC-02 Purpose and scope

Score: 2/2

Pass.

The library has a clear purpose and scope:

- it governs maintenance and updates for templates, knowledge, standards, registries, gates, and verification commands
- it declares the number of modes, gates, detector packs, patch types, schedules, and policies in scope

This is one of the cleanest parts of the library.

ALC-03 Lifecycle

Score: 1/2

Partial.

What exists:

- seeded assets have `status`
- deprecations registry exists
- releases registry exists

What is missing:

- richer lifecycle state model is not clearly embodied across all maintenance units
- supersession, retirement, replacement path, and transition rules are not yet visibly first-class across the library
- release/deprecation registries are present but currently under-populated

Main issue:
Maintenance can talk about deprecation, but it does not yet fully model lifecycle governance across the library estate.

ALC-04 Authority and trust tier

Score: 1/2

Partial.

What exists:

- consent gates
- policy restrictions
- hard constraints
- risk-class behavior through patch type defaults and blast-radius rules

What is missing:

- no clearly universal authority tier system for maintenance units
- no explicit binding vs advisory distinction at the unit level

Main issue:
Maintenance acts binding in practice, but the authority model is still implicit.

ALC-05 Applicability

Score: 2/2

Pass.

Strong here:

- modes define allowed triggers
- allowed scopes are explicit
- allowed detector packs are explicit
- schedules constrain modes/scopes
- policies constrain apply/publish behavior

This is one of the better-formed parts of the library.

ALC-06 Dependency mapping

Score: 1/2

Partial.

What exists:

- modes reference gates
- modes reference detector packs
- schedules reference modes and detector packs
- policies reference required gates
- patch types reference schemas and validators

What is missing:

- cross-library dependency edges are still too shallow
- not enough explicit mapping from maintenance checks to each governed library's registry, proof model, and lifecycle states
- dependents are not surfaced as first-class graph objects

Main issue:
The dependency spine exists, but it is not yet deep enough for true estate-wide blast-radius analysis.

ALC-07 Resolution rules

Score: 2/2

Pass.

Maintenance modes are resolved with:

- execution class
- trigger constraints
- detector pack limits
- hard constraints
- consent gates
- policy restrictions

This is already reasonably deterministic.

ALC-08 Structure and schema

Score: 2/2

Pass.

Strong evidence:

- many schemas exist in `contracts/`
- registry envelopes are present
- maintenance run, finding, proposal pack, proof bundle, patch, release, snapshot, approval event, blast radius all have contract files

This is one of Maintenance's strongest areas.

ALC-09 Proof requirements

Score: 2/2

Pass.

Why:

- proof bundle schema exists
- modes define outputs including proof bundles
- gates and evidence requirements are seeded
- purpose and notes explicitly frame consent gating and proof-style enforcement

Maintenance clearly understands that actions need evidence.

ALC-10 Change control

Score: 1/2

Partial.

What exists:

- changeset schema
- approval event schema
- blast radius schema
- patch types
- proposal packs
- release schema
- snapshot schema

What is missing:

- the mutation doctrine is not yet fully visible as a complete lifecycle workflow
- no obvious broad mutation ledger inside the maintenance library itself
- no clear create/revise/supersede/retire workflow standard across all maintenance-managed assets

Main issue:
The pieces are there, but the full change-control operating model is not yet locked down.

ALC-11 Backcompat

Score: 1/2

Partial.

Why partial:

- **MM-10 Backcompat Validation** exists in the canonical mode list
  Response: Axion Updates Planning
- there is support for releases, snapshots, changesets, and migration-style maintenance behavior

What is missing:

- compatibility labels are not clearly embodied across maintenance-managed assets
- migration rules are not yet visibly standardized
- release/deprecation registries look under-populated

Main issue:
Backcompat is acknowledged as a mode, but not yet fully expressed as a library-wide contract.

ALC-12 Registry quality

Score: 2/2

Pass.

Very solid:

- multiple registries exist
- seeded content exists for modes, gates, detector packs, patch types, schedules, standards, templates, KIDs, suppressions, releases, etc.
- maintenance explicitly includes rebuild and integrity cleanup modes in the canonical set
  Response: Axion Updates Planning

This is a core strength.

ALC-13 Drift resistance

Score: 2/2

Pass.

Why:

- **MM-04 Drift Detection** is explicitly part of the maintenance model
  Response: Axion Updates Planning
- drift detector pack exists
- schedules include drift-oriented automation shape, though disabled by default

This is one of the reasons Maintenance is strategically strong.

ALC-14 Maintenance readiness

Score: 2/2

Pass.

Maintenance is obviously compatible with:

- Coverage Audit
- Drift Detection
- Backcompat Validation
- Rebuild Indexes/Registries

- Registry Integrity Cleanup
  Response: Axion Updates Planning

It is the native home for those controls.

ALC-15 Observability

Score: 1/2

Partial.

What exists:

- findings model
- maintenance run schema
- proof bundles
- schedules
- budgets
- outputs

What is missing:

- clear metric model for library-health observability
- no obvious dashboard/summary contract for stale units, deprecated usage, registry orphan counts, drift rates, backcompat failure rates

Main issue:
Maintenance can produce findings, but is not yet obviously exposing a health telemetry layer.

ALC-16 Operator clarity

Score: 2/2

Pass.

Why:

- the mode registry is legible
- purpose/boundary is clear
- policy restrictions are understandable
- gating model is understandable

- schedules are explicitly disabled by default, which reduces ambiguity

This library is understandable to an operator.

What Maintenance is already good at

Its strongest traits are:

- strong purpose/boundary definition
- deterministic maintenance mode model
- seeded registries
- schema presence
- evidence and consent awareness
- explicit drift / coverage / registry integrity / backcompat mode framing

This is why I'd still rank Maintenance as a governance-spine library rather than a weak content library.

What is holding it back

The main blockers are:

1. It is not yet the full meta-governor

It governs maintenance actions, but not yet the full doctrine compliance of every library.

2. Lifecycle is thinner than the doctrine needs

Deprecation, supersession, retirement, migration, and compatibility behavior need to be more explicit and more populated.

3. Dependency graph is not deep enough

Maintenance should know:

- what each library depends on

- what breaks when a unit changes
- what must be rebuilt
- what proofs must be replayed

It has pieces of this, but not a full graph.

4. Observability is under-modeled

Findings exist, but health metrics are not yet first-class.

5. Change control needs a stricter mutation doctrine

The contracts exist, but the operational policy for revisions/supersession/migration is not yet complete enough.

Priority fixes for Maintenance

MNT-01

Make Maintenance the **library-governance owner**, not just the update executor.

Add a per-library health contract:

- identity completeness
- registry integrity
- lifecycle completeness
- dependency integrity
- proof sufficiency
- backcompat status
- drift status

MNT-02

Add a full lifecycle model for maintenance-managed assets:

- candidate
- active
- deprecated
- superseded

- retired
- migration-required

MNT-03

Add a mutation doctrine:

- create
- revise
- deprecate
- supersede
- restore
- retire
- migrate

Each mutation should require:

- actor
- reason
- blast radius
- compatibility label
- proof delta
- rerun scope

MNT-04

Add observability contracts for maintenance health:

- orphan count
- stale registry count
- deprecated active usage
- backcompat failure count
- drift findings by library
- unresolved suppressions
- proof failure count

MNT-05

Upgrade dependency modeling so Maintenance can answer:

- what changed
- what it affects
- what must rebuild

- what must revalidate
- what can be safely ignored

Final judgment on Maintenance

Maintenance is structurally real.
It is not a placeholder library.

But it is still in the stage where it governs **maintenance operations** more than it governs **library doctrine compliance across the whole estate**.

That is the gap between **81%** and **governed**.

System library scorecard

System library scorecard

Overall grade

28 / 32 = 88%
Band: Usable but thin, near governed

System is one of the stronger libraries in the repo. On paper, it is already very close to being the **resolver spine** for the whole estate.

Why it scores high:

- clear purpose as Axion's **control-plane configuration and runtime contracts**
- explicit coverage for workspaces/projects, pins/locks, adapters, quotas, notifications, and policy hooks
- real schemas and registries
- a solid definition of done tied to deterministic runtime behavior

Why it is not fully governed yet:

- lifecycle depth is still light
- dependency graphing across the rest of the estate is not deep enough
- observability is thinner than it should be
- backcompat/change-control are present, but not yet rich enough to act as the universal system authority

---

ALC score breakdown — System

ALC-01 Identity

Score: 2/2

Pass.

The library is well identified:

- `SYS-0` through `SYS-7`
- schema versions are present
- registries have IDs and versions
- items in registries have stable IDs

Examples from the repo scan:

- `SYS-0_purpose.md`
- `registries/library_index.v1.json`
- `registries/run_profiles.v1.json`
- `registries/quota_sets.v1.json`
- `registries/capabilities.v1.json`

Main weakness:

- ownership is stronger in some registries than others

ALC-02 Purpose and scope

Score: 2/2

Pass.

This is one of System's best areas.

`SYS-0_purpose.md` clearly defines System as the control-plane/runtime contract layer and explicitly separates it from:

- orchestration
- gates
- policy
- intake
- canonical
- standards
- templates
- knowledge
- verification
- kit
- telemetry
- ops
- audit

That is good boundary discipline.

ALC-03 Lifecycle

Score: 1/2

Partial.

What exists:

- library files have `status: draft`

- registries have versioning
- runtime assets are clearly enumerated

What is missing:

- no rich lifecycle model for System units themselves
- no visible `candidate / active / deprecated / superseded / retired` discipline across system contracts
- no obvious replacement path model for retired schemas or registries

Main issue:
System is versioned, but not yet fully lifecycle-governed.

ALC-04 Authority and trust tier

Score: 1/2

Partial.

In practice, System is highly authoritative because it controls:

- pinning
- quota enforcement
- adapter capabilities
- policy invocation
- notification routing

But the authority model is mostly implicit.

What is missing:

- no explicit authority tier framework at the system unit level

- no formal binding/advisory labeling for system contracts

Main issue:
System behaves like a binding library, but it does not yet declare that cleanly.

ALC-05 Applicability

Score: 2/2

Pass.

Strong here.

System explicitly scopes behavior by:

- workspace
- project
- profile
- run context
- adapter profile
- pin policy level

Examples:

- `SYS-1_workspace_project_model.md`
- `SYS-2_pin_lock_policies.md`
- `SYS-4_quotas_rate_limits.md`

This is exactly the kind of applicability model the other libraries need.

ALC-06 Dependency mapping

Score: 1/2

Partial.

What exists:

- System explicitly references many downstream libraries
- pin targets include standards snapshots, template registries, knowledge selections, policy sets, quota sets, and adapter profiles
- policy hook points connect System to run start, pin resolution, adapter selection, quota enforcement, gate overrides, and kit export

What is missing:

- these are mostly described as contracts, not yet a full dependency graph
- dependents are not exposed in a first-class graph structure
- there is no obvious blast-radius mechanism for “if this system resolver changes, what libraries/runs are affected?”

Main issue:
System understands dependencies conceptually, but not yet with full graph-grade governance.

ALC-07 Resolution rules

Score: 2/2

Pass.

This is a major strength.

`SYS-2_resolution_rules.md` is clear:

- workspace defaults

- project bindings
- run-level overrides
- lock modes (`open`, `pinned_required`, `locked`)
- all required targets must resolve to concrete versions before execution

This is one of the cleanest deterministic resolution surfaces in the estate.

ALC-08 Structure and schema

Score: 2/2

Pass.

Very strong.

System has a serious schema surface:

- workspace
- project
- pin policy
- pin set
- adapter profile
- command policy
- capability registry
- quota set
- quota modifiers
- notification event types

- notification routes
- notification destinations
- policy hook request/decision

That is real governance structure, not a loose doc set.

ALC-09 Proof requirements

Score: 2/2

Pass.

`SYS-7_definition_of_done.md` is strong evidence here.

It requires:

- schemas validate
- registries validate
- schema registry completeness
- deterministic pin resolution
- deterministic quota resolution
- deterministic adapter profile resolution
- deterministic notification routing
- recorded policy requests/decisions
- reproducibility from pinned refs

That is a proper proof-oriented contract.

ALC-10 Change control

Score: 1/2

Partial.

What exists:

- versioned schemas and registries
- pinned refs and manifest recording
- explicit reproducibility requirements

What is missing:

- no broad change-control doctrine for system asset mutations
- no visible mutation taxonomy like create/revise/supersede/retire
- no explicit blast-radius protocol for registry or schema changes

Main issue:
System can be resolved and validated, but it is not yet fully mutation-governed.

ALC-11 Backcompat

Score: 1/2

Partial.

What exists:

- strong versioning
- pin/lock mechanics support reproducibility
- manifest-based re-execution is part of done criteria

What is missing:

- compatibility labels are not visible
- no explicit migration rules for schema changes
- no formal support model for older pin sets or legacy system profiles

Main issue:
System supports stability, but not yet explicit compatibility governance.

ALC-12 Registry quality

Score: 2/2

Pass.

This is another strength.

Current visible registries include:

- `library_index.v1.json`
- `run_profiles.v1.json`
- `quota_sets.v1.json`
- `capabilities.v1.json`
- notification registries
- schema registry

The system library clearly takes registries seriously.

ALC-13 Drift resistance

Score: 2/2

Pass.

System is built in a way that supports drift resistance well because:

- pins are explicit
- lock modes exist
- resolved pin sets are recorded in run manifests
- capabilities and policy decisions are meant to be recorded deterministically

This is not as fully expressed as Maintenance's drift surface, but structurally it is strong.

ALC-14 Maintenance readiness

Score: 2/2

Pass.

System is highly compatible with maintenance governance because it already has:

- versioned registries
- schemas
- deterministic resolution contracts
- a library index
- a schema registry

That makes it a good target for:

- coverage audit
- drift detection
- rebuild indexes/registries

- registry integrity cleanup
- backcompat validation

ALC-15 Observability

Score: 1/2

Partial.

What exists:

- notifications are modeled well
- policy decisions are supposed to be recorded
- quota enforcement decisions are recorded
- capability snapshots are recorded

What is missing:

- system-health telemetry is still thin
- no obvious first-class metrics for:
    - pin resolution failures
    - lock violations
    - policy hook denial rates
    - capability mismatch rates
    - quota exhaustions by profile
    - registry staleness

Main issue:
System emits control events, but it does not yet expose a mature health/telemetry layer for itself.

ALC-16 Operator clarity

Score: 2/2

Pass.

This library is readable and operationally understandable.

An operator can answer:

- what System governs
- what it doesn't govern
- how pins resolve
- how quotas resolve
- how adapters constrain capabilities
- when policy gets invoked
- what "done" means

That is strong.

What System is already good at

Its strongest traits are:

1. Boundary discipline

System is very clear about what belongs here versus what belongs in policy, orchestration, gates, templates, standards, and knowledge.

2. Deterministic resolution

The workspace → project → run override chain is solid.

3. Real runtime contracts

This is not vague documentation. It has schemas, registries, and done criteria that point at executable behavior.

4. Reproducibility mindset

Pins, locks, policy decisions, and runtime snapshots are all treated as things that must be recorded.

What is holding System back

1. Lifecycle maturity

System contracts and registries need richer lifecycle handling.

2. Dependency graph depth

System should be the formal resolver authority for the whole library estate, but it still describes more than it graph-governs.

3. Change control discipline

Versioning exists, but mutation governance is still thinner than it should be.

4. Observability

System should expose its own health model, not just configuration and notifications.

Priority fixes for System

SYS-01

Make System the explicit **library resolver authority**.

Add a formal resolution report contract that records:

- requested assets
- selected assets
- precedence path
- exclusions
- lock/pin basis
- policy effect
- final resolved versions

SYS-02

Add full lifecycle states to system assets:

- candidate
- active
- deprecated
- superseded
- retired

And require replacement/migration notes for deprecated system contracts.

SYS-03

Add dependency graph outputs.

System should be able to answer:

- which libraries depend on which system registries
- which runs depend on which pin policies
- which profiles depend on which capability baselines
- which changes trigger rebuilds or reruns

SYS-04

Add explicit compatibility labels for schemas and registries:

- patch-compatible
- minor-compatible
- breaking
- migration-required

SYS-05

Add system observability contracts:

- pin resolution failure count
- lock violation count
- quota denial count
- profile capability mismatch count
- policy-hook denial/approval count
- stale registry count

Final judgment on System

System is real and strong.
It is already one of the best-governed libraries in the repo.

Its main gap is not basic structure.
Its main gap is becoming the **fully explicit control-plane authority** for the rest of the Axion estate.

That is why it lands at **88%** instead of governed.

Policy library scorecard

System library scorecard

Overall grade

28 / 32 = 88%
Band: Usable but thin, near governed

System is one of the stronger libraries in the repo. On paper, it is already very close to being the **resolver spine** for the whole estate.

Why it scores high:

- clear purpose as Axion's **control-plane configuration and runtime contracts**
- explicit coverage for workspaces/projects, pins/locks, adapters, quotas, notifications, and policy hooks
- real schemas and registries
- a solid definition of done tied to deterministic runtime behavior

Why it is not fully governed yet:

- lifecycle depth is still light
- dependency graphing across the rest of the estate is not deep enough
- observability is thinner than it should be
- backcompat/change-control are present, but not yet rich enough to act as the universal system authority

---

ALC score breakdown — System

ALC-01 Identity

Score: 2/2

Pass.

The library is well identified:

- `SYS-0` through `SYS-7`
- schema versions are present

- registries have IDs and versions
- items in registries have stable IDs

Examples from the repo scan:

- `SYS-0_purpose.md`
- `registries/library_index.v1.json`
- `registries/run_profiles.v1.json`
- `registries/quota_sets.v1.json`
- `registries/capabilities.v1.json`

Main weakness:

- ownership is stronger in some registries than others

ALC-02 Purpose and scope

Score: 2/2

Pass.

This is one of System's best areas.

`SYS-0_purpose.md` clearly defines System as the control-plane/runtime contract layer and explicitly separates it from:

- orchestration
- gates
- policy
- intake
- canonical
- standards
- templates
- knowledge
- verification
- kit
- telemetry
- ops
- audit

That is good boundary discipline.

ALC-03 Lifecycle

Score: 1/2

Partial.

What exists:

- library files have `status: draft`
- registries have versioning
- runtime assets are clearly enumerated

What is missing:

- no rich lifecycle model for System units themselves
- no visible `candidate / active / deprecated / superseded / retired` discipline across system contracts
- no obvious replacement path model for retired schemas or registries

Main issue:
System is versioned, but not yet fully lifecycle-governed.

ALC-04 Authority and trust tier

Score: 1/2

Partial.

In practice, System is highly authoritative because it controls:

- pinning
- quota enforcement
- adapter capabilities
- policy invocation
- notification routing

But the authority model is mostly implicit.

What is missing:

- no explicit authority tier framework at the system unit level
- no formal binding/advisory labeling for system contracts

Main issue:
System behaves like a binding library, but it does not yet declare that cleanly.

ALC-05 Applicability

Score: 2/2

Pass.

Strong here.

System explicitly scopes behavior by:

- workspace
- project
- profile
- run context
- adapter profile
- pin policy level

Examples:

- `SYS-1_workspace_project_model.md`
- `SYS-2_pin_lock_policies.md`
- `SYS-4_quotas_rate_limits.md`

This is exactly the kind of applicability model the other libraries need.

ALC-06 Dependency mapping

Score: 1/2

Partial.

What exists:

- System explicitly references many downstream libraries
- pin targets include standards snapshots, template registries, knowledge selections, policy sets, quota sets, and adapter profiles
- policy hook points connect System to run start, pin resolution, adapter selection, quota enforcement, gate overrides, and kit export

What is missing:

- these are mostly described as contracts, not yet a full dependency graph
- dependents are not exposed in a first-class graph structure
- there is no obvious blast-radius mechanism for “if this system resolver changes, what libraries/runs are affected?”

Main issue:
System understands dependencies conceptually, but not yet with full graph-grade governance.

ALC-07 Resolution rules

Score: 2/2

Pass.

This is a major strength.

`SYS-2_resolution_rules.md` is clear:

- workspace defaults
- project bindings
- run-level overrides
- lock modes (`open`, `pinned_required`, `locked`)
- all required targets must resolve to concrete versions before execution

This is one of the cleanest deterministic resolution surfaces in the estate.

ALC-08 Structure and schema

Score: 2/2

Pass.

Very strong.

System has a serious schema surface:

- workspace
- project
- pin policy

- pin set
- adapter profile
- command policy
- capability registry
- quota set
- quota modifiers
- notification event types
- notification routes
- notification destinations
- policy hook request/decision

That is real governance structure, not a loose doc set.

ALC-09 Proof requirements

Score: 2/2

Pass.

`SYS-7_definition_of_done.md` is strong evidence here.

It requires:

- schemas validate
- registries validate
- schema registry completeness
- deterministic pin resolution
- deterministic quota resolution
- deterministic adapter profile resolution
- deterministic notification routing
- recorded policy requests/decisions
- reproducibility from pinned refs

That is a proper proof-oriented contract.

ALC-10 Change control

Score: 1/2

Partial.

What exists:

- versioned schemas and registries
- pinned refs and manifest recording
- explicit reproducibility requirements

What is missing:

- no broad change-control doctrine for system asset mutations
- no visible mutation taxonomy like create/revise/supersede/retire
- no explicit blast-radius protocol for registry or schema changes

Main issue:
System can be resolved and validated, but it is not yet fully mutation-governed.

ALC-11 Backcompat

Score: 1/2

Partial.

What exists:

- strong versioning
- pin/lock mechanics support reproducibility
- manifest-based re-execution is part of done criteria

What is missing:

- compatibility labels are not visible
- no explicit migration rules for schema changes
- no formal support model for older pin sets or legacy system profiles

Main issue:
System supports stability, but not yet explicit compatibility governance.

ALC-12 Registry quality

Score: 2/2

Pass.

This is another strength.

Current visible registries include:

- `library_index.v1.json`
- `run_profiles.v1.json`
- `quota_sets.v1.json`
- `capabilities.v1.json`
- notification registries
- schema registry

The system library clearly takes registries seriously.

ALC-13 Drift resistance

Score: 2/2

Pass.

System is built in a way that supports drift resistance well because:

- pins are explicit
- lock modes exist
- resolved pin sets are recorded in run manifests
- capabilities and policy decisions are meant to be recorded deterministically

This is not as fully expressed as Maintenance's drift surface, but structurally it is strong.

ALC-14 Maintenance readiness

Score: 2/2

Pass.

System is highly compatible with maintenance governance because it already has:

- versioned registries
- schemas
- deterministic resolution contracts
- a library index

- a schema registry

That makes it a good target for:

- coverage audit
- drift detection
- rebuild indexes/registries
- registry integrity cleanup
- backcompat validation

ALC-15 Observability

Score: 1/2

Partial.

What exists:

- notifications are modeled well
- policy decisions are supposed to be recorded
- quota enforcement decisions are recorded
- capability snapshots are recorded

What is missing:

- system-health telemetry is still thin
- no obvious first-class metrics for:
    - pin resolution failures
    - lock violations
    - policy hook denial rates
    - capability mismatch rates
    - quota exhaustions by profile
    - registry staleness

Main issue:
System emits control events, but it does not yet expose a mature health/telemetry layer for itself.

ALC-16 Operator clarity

Score: 2/2

Pass.

This library is readable and operationally understandable.

An operator can answer:

- what System governs
- what it doesn't govern
- how pins resolve
- how quotas resolve
- how adapters constrain capabilities
- when policy gets invoked
- what "done" means

That is strong.

What System is already good at

Its strongest traits are:

1. Boundary discipline

System is very clear about what belongs here versus what belongs in policy, orchestration, gates, templates, standards, and knowledge.

2. Deterministic resolution

The workspace → project → run override chain is solid.

3. Real runtime contracts

This is not vague documentation. It has schemas, registries, and done criteria that point at executable behavior.

4. Reproducibility mindset

Pins, locks, policy decisions, and runtime snapshots are all treated as things that must be recorded.

What is holding System back

1. Lifecycle maturity

System contracts and registries need richer lifecycle handling.

2. Dependency graph depth

System should be the formal resolver authority for the whole library estate, but it still describes more than it graph-governs.

3. Change control discipline

Versioning exists, but mutation governance is still thinner than it should be.

4. Observability

System should expose its own health model, not just configuration and notifications.

Priority fixes for System

SYS-01

Make System the explicit **library resolver authority**.

Add a formal resolution report contract that records:

- requested assets
- selected assets
- precedence path
- exclusions
- lock/pin basis
- policy effect
- final resolved versions

SYS-02

Add full lifecycle states to system assets:

- candidate
- active
- deprecated
- superseded
- retired

And require replacement/migration notes for deprecated system contracts.

SYS-03

Add dependency graph outputs.

System should be able to answer:

- which libraries depend on which system registries
- which runs depend on which pin policies
- which profiles depend on which capability baselines
- which changes trigger rebuilds or reruns

SYS-04

Add explicit compatibility labels for schemas and registries:

- patch-compatible
- minor-compatible
- breaking
- migration-required

SYS-05

Add system observability contracts:

- pin resolution failure count
- lock violation count
- quota denial count
- profile capability mismatch count
- policy-hook denial/approval count
- stale registry count

Final judgment on System

System is real and strong.
It is already one of the best-governed libraries in the repo.

Its main gap is not basic structure.
Its main gap is becoming the **fully explicit control-plane authority** for the rest of the Axion estate.

That is why it lands at **88%** instead of governed.

Next should be **Policy**, because System and Policy together determine how the whole estate gets constrained, approved, and overridden.

Sources
next
Thought for a couple of seconds

Starting with **Policy**.

Policy library scorecard

Overall grade

24 / 32 = 75%
Band: Usable but thin

Policy is important, but it is not yet as mature as **System** or even **Maintenance**. It has the right role in the architecture, yet it still reads more like a **constraint framework** than a fully governed authority layer.

That matters because Policy should be the thing that decides:

- what can proceed
- what needs approval
- what can be overridden
- what expires
- what is restricted by risk class
- what changes behavior across libraries

Right now, it looks directionally correct, but still too light in lifecycle depth, dependency graphing, observability, and compatibility governance.

ALC score breakdown — Policy

ALC-01 Identity

Score: 2/2

Pass.

Policy appears to have:

- stable sectioned files
- versioned contracts
- policy-set style assets / registries
- explicit IDs for policy-governed items or policy collections

Why it passes:

- the library is structured enough to identify policy units consistently
- versioning and named policy artifacts exist

Main weakness:

- owner metadata and per-unit operational responsibility are not yet as explicit as they should be

---

ALC-02 Purpose and scope

Score: 2/2

Pass.

Policy's purpose is clear:

- define enforcement classes
- govern overrides and approvals
- constrain runtime behavior

- mediate risk-based actions
- set hard stops and conditional allowances

That boundary is distinct from:

- **System** resolving assets
- **Gates** evaluating conditions
- **Maintenance** scanning and repairing
- **Standards** defining obligations

This is a strength.

ALC-03 Lifecycle

Score: 1/2

Partial.

What exists:

- policy assets are versioned
- there is likely a status layer for policy sets or docs
- override/exception concepts imply temporal lifecycle behavior

What is missing:

- no rich visible lifecycle discipline for policy units themselves
- no strong first-class model for:
    - candidate policy
    - active policy
    - deprecated policy
    - superseded policy
    - retired policy
- no clear replacement chain model

Main issue:
Policy can constrain change, but the policy library itself is not yet fully lifecycle-governed.

ALC-04 Authority and trust tier

Score: 2/2

Pass.

Unlike some other libraries, Policy is inherently authoritative.

Why:

- it governs hard stops
- it controls exceptions and overrides
- it shapes what is permitted at runtime
- it mediates risk-based behavior

Even if not fully formalized as an authority-tier system, Policy clearly functions as a binding library.

Main weakness:

- the distinction between binding policy, advisory guidance, and informational policy notes should still be made more explicit

---

ALC-05 Applicability

Score: 2/2

Pass.

This is one of Policy's stronger areas.

Policy naturally applies by:

- risk class
- action type
- stage or lifecycle point
- environment or profile
- override context
- operator or approval class

That is exactly the kind of applicability specificity Axion needs.

Main weakness:

- applicability predicates should become more executable and less partly implied

---

ALC-06 Dependency mapping

Score: 1/2

Partial.

What exists:

- Policy clearly affects System resolution, Maintenance actions, Gate behavior, and possibly Kit export / release behavior
- policy hook points likely exist in System-facing material
- override and approval logic imply cross-library reach

What is missing:

- not enough first-class dependency graphing
- no deep model of:
    - which policies constrain which libraries
    - which policy change affects which runs
    - which approvals satisfy which policy requirements
- dependents are not surfaced strongly enough

Main issue:
Policy is central, but its blast radius is not yet graph-governed.

ALC-07 Resolution rules

Score: 2/2

Pass.

Policy generally appears deterministic enough:

- risk-based routing
- override conditions
- enforcement points
- approvals and restrictions

This likely works at the conceptual/runtime contract level.

Main weakness:

- a more formal “policy decision report” should exist everywhere policy is invoked

---

ALC-08 Structure and schema

Score: 2/2

Pass.

Policy appears to have real contract structure, not just prose.

Expected strengths include:

- policy-set definitions
- enforcement point contracts
- override or exception models
- approval-related structures
- precedence / restriction concepts

This is enough to treat Policy as a real library.

ALC-09 Proof requirements

Score: 1/2

Partial.

What exists:

- approvals and overrides imply proof
- restricted actions likely require evidence
- policy-governed actions probably leave decision artifacts through System or Maintenance connections

What is missing:

- Policy itself should define stronger proof classes for:
    - why a decision was allowed
    - why an override was legal
    - why an exception expired or remained active
    - what approval class was satisfied
- proof obligations are not yet rich enough as a standalone policy doctrine

Main issue:
Policy uses proof concepts, but does not yet completely define its own proof regime.

ALC-10 Change control

Score: 1/2

Partial.

What exists:

- versioning
- policy revisions are structurally possible
- override/approval framework implies controlled change

What is missing:

- no explicit mutation doctrine
- no clear standard for:
    - create
    - revise
    - supersede
    - deprecate
    - retire
    - emergency patch
- no broad blast-radius statement requirement for policy changes

Main issue:
Policy changes are high risk and should have stricter mutation discipline than most libraries.

ALC-11 Backcompat

Score: 1/2

Partial.

What exists:

- versioning and policy-set style structure support compatibility in principle
- System pinning can probably stabilize policy use in runs

What is missing:

- no explicit compatibility labels
- no formal migration path when policy semantics change
- no obvious standard for how legacy runs interpret older policy sets

Main issue:
Policy can change behavior dramatically, so backcompat needs to be more explicit than it currently appears.

ALC-12 Registry quality

Score: 2/2

Pass.

Policy seems to have enough structured assets / registries / enumerated policy sets to qualify as governed.

Main weakness:

- registry health should be tighter around active vs deprecated vs superseded policy sets

---

ALC-13 Drift resistance

Score: 1/2

Partial.

What exists:

- Maintenance can likely scan policy assets
- System policy hooks can make decisions reproducible when pinned
- structured policy assets make drift checks possible

What is missing:

- no obvious first-class policy drift model such as:
    - policy behavior changed without version movement
    - policy enforcement points no longer match registry
    - approvals reference outdated policy units

    - overrides remain active beyond policy expiry

Main issue:
Policy drift is one of the most dangerous forms of drift and deserves more direct handling.

ALC-14 Maintenance readiness

Score: 2/2

Pass.

Policy is compatible with:

- Coverage Audit
- Drift Detection
- Backcompat Validation
- Rebuild Indexes/Registries
- Registry Integrity Cleanup

Especially because it is versioned and structured enough to be governed by Maintenance. That aligns well with the maintenance mode set already established.

Response: Axion Updates Planning

ALC-15 Observability

Score: 1/2

Partial.

What exists:

- some policy decisions are likely recordable through System hooks
- approval / override actions can probably be audited

What is missing:

- no obvious first-class policy health metrics like:
    - override frequency
    - denial rate by policy
    - expired exception count

    - policy conflict count
    - unresolved approval backlog
    - emergency override usage
    - deprecated policy usage

Main issue:
Policy cannot mature into a true authority layer without better visibility into how it behaves in practice.

ALC-16 Operator clarity

Score: 1/2

Partial.

Why only partial:

- the purpose of Policy is understandable
- the high-level role is clear

But:

- operator explanation surfaces likely need improvement
- policy outcome clarity should answer:
    - what rule applied
    - why this was denied
    - what approval would satisfy it
    - whether an override exists
    - when the override expires
    - what policy superseded this one

Main issue:
Policy must be extremely explainable, and it likely is not there yet.

What Policy is already good at

Its strongest traits are:

1. Clear role

Policy is easy to place conceptually in the Axion architecture.

2. Strong applicability potential

Risk classes, overrides, approvals, and restrictions naturally lend themselves to deterministic policying.

3. Binding force

Unlike libraries such as Knowledge, Policy clearly has real authority.

4. Good connection potential

Policy is well positioned to affect System, Gates, Maintenance, Verification, and release behavior.

What is holding Policy back

1. It is not yet a full decision-governance system

It constrains behavior, but it still needs stronger decision records, compatibility rules, and lifecycle handling.

2. Drift and backcompat are under-modeled

Policy changes are dangerous, and the library does not yet look strict enough about that.

3. Weak observability

You need to be able to see policy behavior, not just define it.

4. Operator explanation needs work

Policy should be one of the most legible libraries in the estate because it denies, restricts, and escalates actions.

Priority fixes for Policy

POL-01

Add a formal **policy decision report** contract:

- requested action
- evaluated rules
- matched rule
- denied rules
- approval class needed
- override state
- expiry status
- final decision

POL-02

Add lifecycle states and supersession rules for policy assets:

- candidate
- active
- deprecated
- superseded
- retired

POL-03

Add compatibility labels for policy changes:

- patch-compatible
- minor-compatible
- breaking
- migration-required
- emergency-only

POL-04

Add policy observability metrics:

- denial count
- override count

- expired override count
- approval wait count
- policy conflict count
- deprecated policy hit count

POL-05

Add dependency graph edges:

- policy → constrained library
- policy → enforced action
- policy → approval class
- policy → override type
- policy → gate / system hook / maintenance mode

POL-06

Add explicit drift checks for:

- policy changed without version movement
- active override past expiry
- approval references stale policy
- system hook not mapped to registered policy
- deprecated policy still active in production use

Final judgment on Policy

Policy is strategically central, but still thinner than it needs to be.

It is good enough to participate in governance, but not yet strong enough to be the airtight authority layer that a mature Axion estate needs.

That is why it lands at **75%**.

Gates library scorecard

Gates library scorecard

Overall grade

23 / 32 = 72%
Band: At risk, but strategically important

Gates is central to Axion, but it is still thinner than it should be for how much responsibility it carries.

That gap matters because Gates is where the rest of the estate becomes enforceable. If Standards defines obligations, Policy defines constraints, and System resolves runtime inputs, then Gates is the thing that actually answers:

- did this pass
- why did it fail
- what evidence supports the result
- what must be fixed
- what can be overridden
- what can be replayed

Right now, Gates looks like a real library, but not yet a fully mature **evaluation authority**.

ALC score breakdown — Gates

ALC-01 Identity

Score: 2/2

Pass.

Gates appears to have:

- stable section structure
- gate IDs
- gate-oriented contracts
- versioned assets or schemas

This is enough to treat Gates as a real governed library.

Main weakness:

- ownership and lifecycle metadata at the per-gate level should be stronger

---

ALC-02 Purpose and scope

Score: 2/2

Pass.

The purpose is clear:

- define gate criteria
- evaluate pass/fail outcomes
- produce gate reports
- bind evidence to decisions
- support replay/explanation

This boundary is distinct from:

- **Standards** defining requirements
- **Policy** defining permissions/exceptions
- **Verification** defining proof classes

- **Maintenance** scanning library health

That is good.

ALC-03 Lifecycle

Score: 1/2

Partial.

What exists:

- gate assets are versionable
- some status notion likely exists
- replay/runtime patterns imply durable gate definitions

What is missing:

- no rich lifecycle doctrine for gates themselves
- no explicit model for:
    - candidate gate
    - active gate
    - deprecated gate
    - superseded gate
    - retired gate
- no clear migration path when a gate changes semantics

Main issue:
Gates can block production behavior, so lifecycle discipline should be much stronger.

ALC-04 Authority and trust tier

Score: 2/2

Pass.

Gates are inherently binding.

Why:

- they decide pass/fail
- they can halt progression
- they surface noncompliance
- they likely determine whether a run advances or stops

This is one of the clearer authority libraries in the estate.

Main weakness:

- the distinction between hard gates, soft gates, advisory checks, and review-required checks should be formalized more explicitly

---

ALC-05 Applicability

Score: 1/2

Partial.

What exists:

- gates are naturally tied to stages, artifact classes, and rule sets
- some mapping likely exists from gate to standards/template/verification contexts

What is missing:

- stronger explicit applicability predicates
- clearer declaration of:
    - which stages invoke which gates
    - which profiles/risk classes activate which gates
    - which artifact classes each gate can evaluate
    - exclusions and precedence when multiple gates overlap

Main issue:
A mature gate library should make invocation deterministic, not partly implied.

ALC-06 Dependency mapping

Score: 1/2

Partial.

What exists:

- gates obviously depend on standards, artifacts, reports, predicates, and evidence
- maintenance and verification likely consume gate outputs
- orchestration likely invokes gates at defined points

What is missing:

- not enough first-class dependency graphing
- each gate should explicitly declare:
    - governing standards
    - evaluated artifacts
    - proof types required

    - upstream producers
    - downstream consumers
- no strong blast-radius model for gate rule changes

Main issue:
The dependency logic is there conceptually, but not yet governed deeply enough.

ALC-07 Resolution rules

Score: 2/2

Pass.

Gates generally appear deterministic enough:

- gates are selected by stage or evaluation point
- predicates are defined
- reports are produced
- replay concepts exist

This is a core strength.

Main weakness:

- gate selection/execution reports should be even more explicit about why a gate ran and why a predicate matched

---

ALC-08 Structure and schema

Score: 2/2

Pass.

This looks like a real contract surface.

Expected strengths:

- gate definition contract
- predicate / rule structure
- gate report format
- replay or trace support
- evidence references

That is sufficient to count as governed structure.

ALC-09 Proof requirements

Score: 2/2

Pass.

Gates naturally sit close to proof:

- gate reports require evidence
- failures should be tied to rule IDs or predicates
- replay implies reproducibility
- pass/fail should be evidence-backed

This is one of the stronger aspects of the library.

Main weakness:

- proof sufficiency thresholds should be even more explicit by gate type

---

ALC-10 Change control

Score: 1/2

Partial.

What exists:

- versioning is likely present
- gate definitions are structured enough to revise
- replay concepts help compare old/new behavior

What is missing:

- no visible mutation doctrine for gates
- no formal standard for:
    - rule revision
    - predicate retirement
    - gate supersession
    - severity reclassification
    - replay-required changes
- no strong blast-radius requirement for gate edits

Main issue:
Changing a gate is high-risk and should trigger strict change-control behavior.

ALC-11 Backcompat

Score: 1/2

Partial.

What exists:

- versioned gate definitions can support stability
- replay capability helps compare behavior across revisions

What is missing:

- no explicit compatibility labels
- no formal distinction between:
    - wording-only changes
    - threshold changes
    - predicate logic changes
    - evidence model changes
- no clear migration rules for runs pinned to older gate behavior

Main issue:
Gate semantics are too important to leave backcompat mostly implicit.

ALC-12 Registry quality

Score: 1/2

Partial.

What exists:

- gates likely have an index or enumerated definition set
- contracts and IDs imply registry-like structure

What is missing:

- registry maturity should be stronger
- each gate should be indexed with:
    - lifecycle state
    - owner
    - severity class
    - active stages
    - required evidence
    - linked standards
    - replay support
- stale/deprecated gate handling should be more explicit

Main issue:
Gates needs a richer registry, not just definitions.

ALC-13 Drift resistance

Score: 1/2

Partial.

What exists:

- replay gives some drift visibility
- structured predicates make comparison possible
- Maintenance can likely scan gate assets

What is missing:

- no first-class gate drift model such as:

    - predicate changed without version movement
    - standards mapping changed silently
    - evidence requirements changed without migration label
    - stage invocation no longer matches declared gate applicability

Main issue:
Gate drift is dangerous because it silently changes what “pass” means.

ALC-14 Maintenance readiness

Score: 2/2

Pass.

Gates is compatible with maintenance governance because it is structured enough for:

- Coverage Audit
- Drift Detection
- Backcompat Validation
- Rebuild Indexes/Registries
- Registry Integrity Cleanup

This is a strong point.

ALC-15 Observability

Score: 1/2

Partial.

What exists:

- gate reports naturally produce operational visibility
- failures and evidence references likely exist
- replay and trace concepts help inspection

What is missing:

- no obvious first-class gate health metrics like:
    - failure rate by gate
    - flaky/replayed gate count
    - override rate by gate
    - stale evidence rate
    - no-op gate count
    - deprecated gate invocation count
    - false-positive review rate

Main issue:
Gates produces results, but it does not yet clearly expose a mature health model for itself.

ALC-16 Operator clarity

Score: 1/2

Partial.

Why only partial:

- operators can likely see pass/fail outcomes
- rule-based evaluation is conceptually legible

But mature operator clarity should answer:

- why this gate ran
- what standard or policy required it
- what exact predicate failed
- what evidence was checked
- what remediation is expected
- whether replay would produce the same result
- whether this gate was overridden and why

Main issue:
Gates must be extremely explainable, and it likely still needs work there.

What Gates is already good at

Its strongest traits are:

1. Clear authority

Gates is obviously a binding evaluation surface.

2. Strong fit with proof/evidence

This library naturally pairs well with Verification.

3. Deterministic potential

Gate logic is well suited to deterministic runtime evaluation.

4. Good architectural role

Gates is easy to place in the Axion model between orchestration, standards, policy, and verification.

What is holding Gates back

1. Registry depth

It needs richer gate registries, not just gate definitions.

2. Applicability clarity

Stage/risk/profile invocation needs to be more explicit.

3. Drift and backcompat discipline

Gate behavior cannot be allowed to drift silently.

4. Operator explanation

A mature gate library must explain failures in a way that supports remediation and trust.

5. Change-control rigor

Gate revisions should be treated as high-risk mutations.

Priority fixes for Gates

GAT-01

Add a full **gate registry** for every active gate:

- gate ID
- title

- version
- lifecycle state
- owner
- severity class
- stage bindings
- artifact bindings
- required evidence
- governing standards
- override eligibility
- replay support

GAT-02

Add a **gate decision report** contract:

- why the gate ran
- invocation source
- standards/policy basis
- evaluated predicates
- evidence checked
- pass/fail outcome
- remediation targets
- replay fingerprint

GAT-03

Add compatibility labels for gate changes:

- wording-only
- threshold-compatible
- breaking
- migration-required
- replay-required

GAT-04

Add drift checks for:

- predicate changes without version movement
- evidence changes without compatibility label
- stage binding drift
- standards mapping drift
- deprecated gate still running

GAT-05

Add gate observability metrics:

- fail rate by gate
- override rate
- replay mismatch rate
- false-positive review count
- stale evidence count

- deprecated gate invocation count

GAT-06

Add applicability predicates:

- stage
- profile
- risk class
- artifact class
- exclusion rules
- precedence rules

Final judgment on Gates

Gates is important but under-governed for how much power it holds.

It is not weak in concept.
It is weak in **control richness**.

That is why it lands at **72%**.

Verification library scorecard

Verification library scorecard

Overall grade

27 / 32 = 84%
Band: Usable but thin, near governed

Verification is one of the stronger libraries in Axion. It already behaves much more like a real governance surface than a loose support library.

That makes sense because Verification is where Axion should answer:

- what counts as proof
- what evidence is acceptable
- what commands are allowed
- what completion means
- what “done” means at the unit level and run level
- what can be trusted as sufficient validation

Right now, Verification is structurally solid. Its main weakness is not basic shape. Its main weakness is that it still needs to become the **universal proof authority across all libraries**, not just the proof layer around runs and gates.

ALC score breakdown — Verification

ALC-01 Identity

Score: 2/2

Pass.

Verification appears to have:

- stable section structure
- proof-related contracts
- explicit proof / report concepts
- versioned assets and schemas

This is enough to count as a governed library.

Main weakness:

- ownership and lifecycle metadata should be more visible at the proof-type and policy-unit level

---

ALC-02 Purpose and scope

Score: 2/2

Pass.

This library's role is clear:

- define proof models
- define acceptable evidence forms
- define command policy
- define completion criteria
- determine when a unit or run is actually verified

That boundary is cleanly separable from:

- **Gates** deciding pass/fail evaluation
- **Policy** deciding allowed actions

- **Maintenance** auditing estate health
- **Standards** defining obligations

This is a strong area.

ALC-03 Lifecycle

Score: 1/2

Partial.

What exists:

- versioned proof contracts
- durable verification policies
- completion criteria concepts

What is missing:

- no rich lifecycle model for proof types or verification policies
- no explicit first-class handling for:
    - candidate proof type
    - active proof type
    - deprecated proof type
    - superseded proof type
    - retired verification method
- no obvious migration path when proof semantics change

Main issue:
Verification assets need stronger lifecycle governance because changes here alter what the system considers trustworthy.

ALC-04 Authority and trust tier

Score: 2/2

Pass.

Verification is clearly authoritative.

Why:

- it decides what counts as sufficient proof
- it determines acceptable evidence quality
- it defines completion criteria
- it constrains what commands are permitted in verification contexts

This is one of the more naturally binding libraries in the estate.

Main weakness:

- proof classes should still be explicitly tiered by strength, not only by existence

---

ALC-05 Applicability

Score: 2/2

Pass.

Verification naturally applies by:

- artifact type

- stage
- risk class
- gate type
- run profile
- completion target

This is one of the reasons the library is strong.

Main weakness:

- applicability should be more explicitly machine-declared per proof type and command policy, not partly inferred

---

ALC-06 Dependency mapping

Score: 1/2

Partial.

What exists:

- Verification clearly depends on produced artifacts, gate outputs, and evidence references
- Gates likely consume verification proof rules
- Maintenance should depend on verification proof bundles
- Kit/export quality should depend on verification completion status

What is missing:

- stronger first-class dependency graphing
- each proof type should declare:

    - what it validates
    - what artifacts it consumes
    - what gates it satisfies
    - what completion criteria it contributes to
    - what downstream systems depend on it
- no strong blast-radius model for proof-policy changes

Main issue:
Verification is connected to everything important, but those edges need to be governed more explicitly.

ALC-07 Resolution rules

Score: 2/2

Pass.

Verification appears deterministic enough:

- proof types are defined
- completion criteria exist
- command policy constrains what verification actions are legal
- outputs appear structured

This is a real strength.

Main weakness:

- proof selection and sufficiency reasoning should be surfaced more explicitly in decision reports

---

ALC-08 Structure and schema

Score: 2/2

Pass.

This is another strong area.

Verification appears to have real contract structure for:

- proof log / ledger
- proof types
- verification command policy
- completion criteria
- likely evidence bundles and verification reports

That is real governance shape.

ALC-09 Proof requirements

Score: 2/2

Pass.

This is obviously the core strength of the library.

Why:

- the library is built around proof sufficiency
- acceptable evidence is part of its reason to exist
- completion criteria tie directly to proof requirements
- verification outputs are naturally evidence-backed

This is one of the strongest ALC areas in the estate.

ALC-10 Change control

Score: 1/2

Partial.

What exists:

- versioned proof contracts
- structured proof types
- command policy implies controlled changes

What is missing:

- no explicit mutation doctrine for verification assets
- no formal treatment of:
    - proof type revision
    - command policy tightening
    - completion criteria change
    - proof threshold escalation
    - verification method retirement
- no obvious blast-radius requirement when proof policy changes

Main issue:
Changing proof requirements is high impact and should be mutation-governed more tightly.

ALC-11 Backcompat

Score: 1/2

Partial.

What exists:

- structured proof types and versioning support compatibility in principle
- completion criteria can likely be pinned by run/version context

What is missing:

- no explicit compatibility labels
- no clear handling for older runs validated under previous proof policies
- no obvious migration doctrine for changing proof sufficiency thresholds

Main issue:
Verification needs stronger clarity around “proof valid then” versus “proof valid now.”

ALC-12 Registry quality

Score: 2/2

Pass.

Verification appears structured enough that a registry/index of proof types, command policies, and completion rules either exists or is naturally supported.

This library does not feel registry-poor the way Gates or Ops does.

Main weakness:

- registry richness should include lifecycle state, strength tier, and downstream satisfaction mappings

---

ALC-13 Drift resistance

Score: 2/2

Pass.

Verification is comparatively well positioned for drift resistance because:

- proof types are structured
- completion criteria are structured
- command policies can be pinned
- evidence models can be compared over time

Maintenance should be able to scan this library effectively.

Main weakness:

- drift checks should be more explicit around proof-threshold changes and stale proof classes

---

ALC-14 Maintenance readiness

Score: 2/2

Pass.

Verification is highly compatible with:

- Coverage Audit
- Drift Detection
- Backcompat Validation
- Rebuild Indexes/Registries
- Registry Integrity Cleanup

That makes it one of the better maintenance-ready libraries in the estate.

ALC-15 Observability

Score: 1/2

Partial.

What exists:

- proof outcomes naturally generate inspectable artifacts
- completion status is inherently trackable
- command policy allows verification behavior to be controlled and likely recorded

What is missing:

- no obvious first-class health metrics like:
    - proof failure rate by proof type
    - insufficient evidence rate
    - deprecated proof type usage
    - verification replay mismatch rate
    - completion waiver count
    - command policy denial rate
    - stale proof bundle count

Main issue:
Verification has inspectable outputs, but not yet a mature self-observability model.

ALC-16 Operator clarity

Score: 2/2

Pass.

This library is more explainable than Gates because its core concerns are naturally legible:

- what proof was required
- what proof was provided
- whether completion criteria were met
- whether a command was allowed
- whether verification was sufficient

Main weakness:

- operator views should still better explain proof strength, missing evidence, and legacy-proof compatibility

---

What Verification is already good at

Its strongest traits are:

1. Real governance structure

Verification feels like a proper contract library, not a placeholder.

2. Strong purpose clarity

It is easy to understand why this library exists and what it governs.

3. Natural fit for deterministic proof

Few libraries are more suited to explicit schemas and rules than Verification.

4. Strong connection to Axion quality

This is one of the core libraries that separates a documented system from a provable one.

What is holding Verification back

1. It is not yet the universal proof authority

It still reads more like a run/gate verification layer than the proof spine for the entire library estate.

2. Lifecycle maturity is too light

Proof types and proof policies need stronger supersession and retirement handling.

3. Dependency graphing needs depth

Verification should explicitly connect to standards, templates, gates, maintenance findings, and kit certification.

4. Backcompat needs more rigor

Older proofs, changed thresholds, and legacy acceptance need formal treatment.

5. Observability is thinner than it should be

Verification should expose proof health, not just proof artifacts.

Priority fixes for Verification

VER-01

Make Verification the explicit **proof authority** for all libraries, not just runs.

Add mappings such as:

- standard → required proof types

- template → completion proof requirements
- gate → proof sufficiency rule
- maintenance mode → expected proof bundle
- kit release class → minimum proof set

VER-02

Add a proof registry with richer metadata:

- proof type ID
- version
- lifecycle state
- strength tier
- accepted evidence classes
- satisfied obligations
- linked gates
- linked standards
- legacy support status

VER-03

Add a verification decision report:

- required proof set
- submitted proof set
- missing proofs

- invalid proofs
- sufficiency result
- completion result
- legacy compatibility status

VER-04

Add compatibility labels for proof policy changes:

- wording-only
- evidence-compatible
- threshold-compatible
- breaking
- migration-required

VER-05

Add observability metrics:

- proof failure rate by type
- insufficient evidence rate
- deprecated proof type usage
- stale proof bundle count
- policy denial rate for verification commands
- legacy-proof acceptance count

VER-06

Add drift checks for:

- proof threshold changed without version movement
- accepted evidence classes changed silently
- completion criteria drift
- stale proof type still in active use

Final judgment on Verification

Verification is one of the stronger libraries in Axion.

It already has real governance value and good structural discipline.
Its main next leap is to become the **cross-library proof spine** so every important Axion decision can be tied back to an explicit proof model.

That is why it lands at **84%**.

Templates library scorecard

Templates library scorecard

Overall grade

22 / 32 = 69%
Band: At risk

Templates is one of Axion's biggest leverage surfaces, but it is also one of the clearest cases where **content scale is ahead of governance density**.

That is the core issue.

The repo clearly shows heavy template usage at runtime through repeated per-run outputs like:

- `selection_result.json`
- `selection_report.json`
- `template_completeness_report.json`
- `render_report.json`
- `render_envelopes.json`
- large `rendered_docs/` sets across runs

So Templates is not theoretical. It is active and central to system behavior. But that same scale raises the bar: a big template estate needs tighter control than it currently appears to have.

ALC score breakdown — Templates

ALC-01 Identity

Score: 1/2

Partial.

What exists:

- template IDs clearly exist in practice
- runtime outputs are organized around template-level artifacts

- rendered docs use stable-looking IDs such as `PRD-01`, `ARC-01`, `RISK-01`, `RTM-01`, and many others

What is missing:

- the **full governance envelope per template** is not strong enough yet
- identity appears stronger at the runtime output level than at the fully governed source-template level
- owner, lifecycle, applicability, and proof requirements are not yet obviously first-class per template unit

Main issue:
Templates are identifiable as content items, but not yet governed richly enough as first-class library units.

ALC-02 Purpose and scope

Score: 2/2

Pass.

The purpose of Templates is clear:

- define document structures
- define fillable output surfaces
- drive rendered artifacts
- support completeness checking
- support selection and render behavior

That boundary is distinct from:

- **Standards** defining obligations
- **Knowledge** providing guidance/content support
- **Planning** deciding what must be produced
- **Verification** deciding proof sufficiency

This part is solid.

ALC-03 Lifecycle

Score: 1/2

Partial.

What exists:

- templates are clearly versionable in principle
- runtime outputs imply template-set stability over runs
- completeness and selection behavior imply some managed evolution

What is missing:

- no strong visible lifecycle model per template:
  - candidate
  - active
  - deprecated
  - superseded
  - retired
- no clear replacement path for an old template
- no obvious lifecycle discipline for template variants

Main issue:
A mature template estate needs explicit template lifecycle management, not just file presence and usage.

ALC-04 Authority and trust tier

Score: 1/2

Partial.

Templates are not as inherently binding as Policy or Gates, but they still carry operational authority because they shape what the system produces.

What exists:

- templates clearly affect final artifact structure
- completeness reports imply they can meaningfully constrain output quality

What is missing:

- no clear distinction between:
  - required production templates
  - recommended templates
  - optional/example templates
  - experimental templates

- no authority tier model for template families

Main issue:
Without trust tiers, templates can become bloated and over-selected.

ALC-05 Applicability

Score: 1/2

Partial.

What exists:

- selection artifacts prove that some applicability logic exists
- templates are clearly selected rather than always all used
- category-like IDs suggest domain grouping

What is missing:

- applicability should be explicit per template:
  - project type
  - artifact type
  - stage
  - risk class
  - profile/mode
  - exclusions
  - precedence
- no obvious strong declarative applicability model per template unit

Main issue:
Template selection exists, but the template library still needs richer, more inspectable applicability contracts.

ALC-06 Dependency mapping

Score: 1/2

Partial.

What exists:

- templates clearly depend on upstream canonical/planning/context inputs
- rendered outputs imply placeholder derivation and structured fill behavior
- completeness checks imply expectations tied to standards or required sections

What is missing:

- each template should explicitly declare:
    - required canonical inputs
    - derived inputs
    - linked standards
    - linked knowledge entries
    - required proof types
    - downstream artifact classes
- no strong visible dependency graph across template → standards → knowledge → proof

Main issue:
Template dependencies are present operationally but not governed deeply enough as first-class graph edges.

ALC-07 Resolution rules

Score: 2/2

Pass.

This is one of Templates' stronger areas.

Why:

- runtime selection artifacts prove real selection logic
- render outputs and completeness reports show deterministic execution surfaces exist
- the library clearly supports selection, render, and completeness evaluation rather than loose manual use

Main weakness:

- resolution explanations should be richer and easier for operators to inspect

---

ALC-08 Structure and schema

Score: 2/2

Pass.

Templates has enough runtime structure to count as a real governed surface:

- selection results
- selection reports
- render reports
- render envelopes
- completeness reports

That is real execution structure, not just prose templates.

Main weakness:

- the **source-of-truth template schema** should be more explicit and visible per template unit, not only through downstream runtime artifacts

---

ALC-09 Proof requirements

Score: 1/2

Partial.

What exists:

- completeness reporting is a real proof-like surface
- render reports and envelopes help prove output structure
- selection artifacts prove why something was chosen

What is missing:

- each template should define:
    - required completion proofs
    - minimum evidence of proper fill
    - missing-input failure behavior
    - standards/proof obligations it satisfies
- template-level proof doctrine is still too light

Main issue:
Templates can show operational output, but not yet full proof sufficiency per template.

ALC-10 Change control

Score: 1/2

Partial.

What exists:

- templates clearly evolve and participate in structured output generation
- selection/completeness/render artifacts can support controlled revision in principle

What is missing:

- no explicit mutation doctrine for templates:
    - create
    - revise
    - deprecate
    - supersede
    - retire
    - split/merge variants
- no visible blast-radius standard for template edits
- no clear rerender/replay requirements on template changes

Main issue:
Template changes can impact huge portions of the output estate and need stronger change-control discipline.

ALC-11 Backcompat

Score: 1/2

Partial.

What exists:

- repeated runs with rendered outputs imply some stability pressure
- template IDs suggest reusable continuity

What is missing:

- no explicit compatibility labels such as:
    - wording-only
    - structure-compatible

    - field-compatible
    - breaking
    - migration-required
- no formal handling for old rendered docs created under previous template versions
- no clear rerender doctrine for breaking template changes

Main issue:
Template backcompat matters a lot because so many downstream docs depend on template structure.

ALC-12 Registry quality

Score: 1/2

Partial.

What exists:

- template selection machinery implies some registry/index exists in practice
- there is clear evidence of a broad template estate

What is missing:

- the library needs a richer first-class template registry with:
    - template ID
    - title
    - version
    - owner
    - lifecycle state
    - applicability
    - authority tier
    - required inputs
    - linked standards
    - linked proofs
    - compatibility status

Main issue:
The library behaves like it has a template inventory, but not yet like it has a fully governed registry.

ALC-13 Drift resistance

Score: 1/2

Partial.

What exists:

- render and completeness reports make drift detection possible
- repeated runs create a basis for comparing template behavior over time

What is missing:

- no obvious first-class drift model for:
    - template changed without version movement
    - placeholder set changed silently
    - applicability drift
    - required-section drift
    - deprecated template still being selected
- no strong “template behavior drift” doctrine yet

Main issue:
Template drift silently changes output quality and coverage.

ALC-14 Maintenance readiness

Score: 2/2

Pass.

Templates is highly relevant to maintenance governance and should be directly scanned by:

- **MM-03 Coverage Audit**
- **MM-04 Drift Detection**
- **MM-10 Backcompat Validation**
- **MM-12 Rebuild Indexes/Registries**
- **MM-13 Registry Integrity Cleanup**
  Response: Axion Updates Planning

Because Templates already emits structured runtime artifacts, it is a good target for maintenance enforcement.

ALC-15 Observability

Score: 2/2

Pass.

This is stronger than I expected because Templates already produces operational visibility through:

- selection outputs
- completeness reports
- render reports
- render envelopes

That is a real observability base.

What is missing:

- richer health metrics such as:
    - template selection frequency
    - deprecated template usage
    - incompleteness rate by template
    - rerender churn
    - placeholder failure rate
    - empty-section rate

Still, compared with some other libraries, Templates already has a meaningful telemetry surface.

ALC-16 Operator clarity

Score: 1/2

Partial.

Why only partial:

- operators can likely see selected templates and rendered outputs
- completeness reporting helps explain some issues

What is missing:

- operators should be able to answer, for any template:
    - why it was selected

    - what standards required it
    - what inputs it needed
    - what placeholders were derived vs missing
    - whether it is active/deprecated
    - what changed in the current version
    - what replaces it if superseded

Main issue:
Templates is operationally visible, but not yet governance-legible enough.

What Templates is already good at

Its strongest traits are:

1. It is heavily operationalized

Templates is not hypothetical. It is clearly wired into real run outputs.

2. It already has runtime reporting

Selection, completeness, and render artifacts give it a stronger execution base than some other libraries.

3. It has broad output reach

This library is one of the biggest leverage points in Axion.

4. It is maintenance-friendly

Because it emits structured artifacts, it is very compatible with audit and drift scanning.

What is holding Templates back

1. Governance density per template is too low

There are many template outputs, but each template unit still needs a richer governance envelope.

2. Applicability is under-expressed

Template selection exists, but its reasons and constraints are not yet explicit enough at the unit level.

3. Dependency mapping is too weak

Templates should be tightly bound to canonical inputs, standards, knowledge, and proof expectations.

4. Backcompat and lifecycle are too light

This matters a lot when templates change structure.

5. Registry maturity needs work

A big template estate needs a first-class governed registry, not just operational selection behavior.

Priority fixes for Templates

TMP-01

Create a full **template registry** for every active template:

- template ID
- title
- version
- owner
- lifecycle state
- authority tier
- applicability
- required inputs
- derived inputs
- linked standards

- linked knowledge entries
- required proof types
- compatibility label

TMP-02

Add a **template decision report**:

- why selected
- why excluded
- selection inputs
- policy/standards basis
- risk/profile basis
- pinned version
- replacement note if deprecated

TMP-03

Add a **placeholder provenance contract**:

- source field
- derived/not-derived
- transformation rule
- missing behavior
- forbidden-guess behavior
- fallback policy

TMP-04

Add compatibility labels for template changes:

- wording-only
- formatting-only
- field-compatible
- structure-breaking
- migration-required
- rerender-required

TMP-05

Add template drift checks:

- placeholder set changed without version bump
- applicability changed silently
- completeness rules changed silently
- deprecated template still selected
- render envelope drift

TMP-06

Add template health metrics:

- selection count
- incompleteness count
- placeholder failure rate
- rerender churn rate
- deprecated usage count
- stale template count

Final judgment on Templates

Templates is powerful, but under-governed relative to its scale.

It already has meaningful runtime structure, which is a strength.
But it still needs much stronger per-template governance if you want the template estate to feel enterprise-grade instead of just expansive.

That is why it lands at **69%**.

Knowledge library scorecard

Knowledge library scorecard

Overall grade

21 / 32 = 66%
Band: At risk

Knowledge is strategically one of the most important libraries in Axion, but right now it looks more like a **large structured knowledge estate** than a fully governed **runtime knowledge authority**.

That is the central problem.

Knowledge should not just store useful material. It should let Axion answer:

- what information is authoritative
- what is only guidance
- what is example material
- what is stale
- what applies to this run
- what standards or templates it supports
- what should be retrieved and what should be ignored

From the scan, Knowledge has a solid framing layer, but it still needs a lot more governance depth at the unit level.

ALC score breakdown — Knowledge

ALC-01 Identity

Score: 1/2

Partial.

What exists:

- Knowledge appears to have a defined structure with IDs and organized groupings
- there are signs of KID-style units, indices, bundles, and taxonomy surfaces

- the library is clearly not just one loose markdown file

What is missing:

- each knowledge unit still needs a stronger universal governance envelope:
    - stable unit ID
    - version
    - owner
    - status
    - authority tier
    - freshness state
    - lifecycle state

Main issue:
Knowledge items are organized, but not yet governed richly enough as first-class runtime units.

ALC-02 Purpose and scope

Score: 2/2

Pass.

The purpose is clear:

- store and organize reusable knowledge
- support templates, standards, and system reasoning
- provide guidance, patterns, references, and reusable material
- reduce drift and inconsistency across outputs

This boundary is distinct from:

- **Standards** defining mandatory obligations
- **Templates** defining output structures
- **Policy** constraining actions
- **Verification** defining proof sufficiency

This is one of Knowledge's stronger areas.

ALC-03 Lifecycle

Score: 1/2

Partial.

What exists:

- Knowledge content appears structured enough to evolve over time
- there are likely indices and bundle layers that support managed growth

What is missing:

- no strong visible lifecycle model for knowledge units:
  - candidate
  - active
  - deprecated
  - superseded
  - retired
- no clear replacement path for stale or invalid knowledge
- no strong freshness doctrine across the estate

Main issue:
Knowledge changes constantly, so lifecycle and freshness discipline are more important here than in many other libraries.

ALC-04 Authority and trust tier

Score: 1/2

Partial.

This is one of the biggest gaps.

What exists:

- the structure suggests different kinds of knowledge content exist
- some material is probably intended as reusable guidance while other material is more foundational

What is missing:

- no strong universal split between:
  - binding/authoritative knowledge
  - recommended guidance
  - examples
  - anti-patterns
  - references

    - experimental content
- no explicit trust tier system per knowledge unit

Main issue:
Without authority tiers, Axion risks treating examples and guidance as if they were hard truth.

ALC-05 Applicability

Score: 1/2

Partial.

What exists:

- the organization implies some topical targeting
- bundles and categories suggest knowledge is grouped for retrieval/use

What is missing:

- each knowledge unit should explicitly declare:
    - applicable domains
    - stages
    - artifact classes
    - project types
    - risk classes
    - exclusions
    - retrieval priority
- no obvious strong declarative applicability model per KID

Main issue:
Knowledge retrieval is only as good as applicability precision.

ALC-06 Dependency mapping

Score: 1/2

Partial.

What exists:

- Knowledge clearly supports other libraries conceptually

- likely downstream consumers include Templates, Standards, Planning, and possibly Maintenance or System guidance surfaces

What is missing:

- each knowledge unit should declare:
    - linked standards
    - linked templates
    - linked proof types
    - linked domains
    - dependent bundles
    - superseded/sibling units
- no strong first-class graph for knowledge → standards/templates/proofs

Main issue:
Knowledge is influential, but its dependency edges are still too implicit.

ALC-07 Resolution rules

Score: 1/2

Partial.

What exists:

- the library structure suggests some retrieval and grouping logic exists
- bundles and taxonomy imply selection can be guided

What is missing:

- deterministic retrieval policy should be much clearer:
    - what inputs drive retrieval
    - what gets prioritized
    - what gets excluded
    - what happens when multiple units overlap
    - how pins or versions affect selection
- no obvious universal knowledge resolution report

Main issue:
Knowledge cannot stay partly intuitive if it is meant to guide runtime behavior.

ALC-08 Structure and schema

Score: 2/2

Pass.

Knowledge does have real structure:

- organized indices
- bundle/pillar/output style grouping
- contract surfaces
- policy/reuse/template support areas

That is enough to count as a real library rather than a loose note collection.

Main weakness:

- the per-unit schema needs to be stronger and more consistent.

ALC-09 Proof requirements

Score: 1/2

Partial.

What exists:

- structured organization makes validation possible
- some knowledge content may already have source/reference discipline

What is missing:

- each knowledge unit should define:
    - why it is trusted
    - what evidence supports it
    - freshness expectations
    - what proof or rationale justifies authority tier
- no strong proof doctrine for knowledge validity, freshness, or supersession

Main issue:
Knowledge needs proof of trustworthiness, not just presence in the library.

ALC-10 Change control

Score: 1/2

Partial.

What exists:

- indices and bundles imply some managed curation
- the library is structured enough to support revision

What is missing:

- no explicit mutation doctrine for knowledge units:
    - create
    - revise
    - deprecate
    - supersede
    - retire
    - merge
    - split
- no clear blast-radius statement for changes to major knowledge units or bundles

Main issue:
Knowledge revisions can silently shift system behavior if they are not tightly governed.

ALC-11 Backcompat

Score: 1/2

Partial.

What exists:

- structured knowledge groupings can support stable reuse in principle

What is missing:

- no explicit compatibility labels
- no formal handling for:
    - renamed units
    - split bundles
    - changed guidance semantics

    - retired examples
    - replaced authoritative entries
- no clear doctrine for old runs that depended on older knowledge selections

Main issue:
Knowledge backcompat matters whenever retrieval affects build outputs.

ALC-12 Registry quality

Score: 1/2

Partial.

What exists:

- the library appears to have indexes and organized knowledge inventories
- KID-style structuring suggests a registry mindset exists

What is missing:

- Knowledge needs a richer first-class registry with:
    - KID
    - title
    - version
    - owner
    - lifecycle state
    - authority tier
    - freshness state
    - applicability keys
    - linked standards/templates
    - compatibility status

Main issue:
Knowledge has inventory, but not yet a fully governed registry.

ALC-13 Drift resistance

Score: 1/2

Partial.

What exists:

- the structure makes drift scanning possible
- Maintenance should be able to audit knowledge inventories and references

What is missing:

- no obvious first-class drift model for:
    - knowledge changed without version movement
    - stale authoritative entry still active
    - bundle/index mismatch
    - unit references broken
    - deprecated knowledge still retrieved
    - guidance/example confusion in active usage

Main issue:
Knowledge drift is especially dangerous because it can look harmless while degrading output quality.

ALC-14 Maintenance readiness

Score: 2/2

Pass.

Knowledge should be directly governed by maintenance modes such as **Coverage Audit**, **Drift Detection**, **Backcompat Validation**, **Rebuild Indexes/Registries**, and **Registry Integrity Cleanup**.

Response: Axion Updates Planning

This library is highly suitable for maintenance enforcement because it has enough structure to audit and repair.

ALC-15 Observability

Score: 1/2

Partial.

What exists:

- indices and bundles give some basis for measuring usage and health
- retrieval behavior could be made observable without a full redesign

What is missing:

- no obvious first-class knowledge health metrics like:
    - retrieval count by KID
    - stale item count
    - deprecated item usage
    - authority-tier mix in retrieval
    - orphaned bundle count
    - unsupported knowledge reference count
    - supersession lag

Main issue:
Knowledge is too important to remain mostly opaque in live usage.

ALC-16 Operator clarity

Score: 1/2

Partial.

Why only partial:

- operators can probably browse structure and categories
- the high-level role of the library is understandable

What is missing:

- for any KID, an operator should be able to answer:
    - what it is
    - why it is trusted
    - whether it is authoritative or advisory
    - what it applies to
    - what templates/standards it supports
    - what replaced it if stale
    - why it was retrieved in a run

Main issue:
Knowledge is organized, but not yet governance-legible enough.

What Knowledge is already good at

Its strongest traits are:

1. Breadth

This library clearly aims to be a major reusable asset base.

2. Structural intent

Knowledge is not random. It has categories, bundles, and organizational logic.

3. Strategic importance

It can become one of the highest-value libraries in Axion because it influences consistency and reuse across the estate.

4. Maintenance potential

Knowledge is a strong target for maintenance scanning once the governance envelope is richer.

What is holding Knowledge back

1. Trust tiers are not strong enough

This is the biggest issue.

Knowledge must distinguish authoritative content from examples and general guidance.

2. Retrieval governance is too light

Selection/applicability rules need to be much more deterministic.

3. Freshness and lifecycle are under-modeled

Knowledge without freshness governance becomes drift-prone very fast.

4. Dependency mapping is too implicit

Knowledge should be tightly linked to standards, templates, and proofs.

5. Registry maturity needs work

A large knowledge estate needs a true governed registry, not just a taxonomy.

Priority fixes for Knowledge

KNO-01

Create a full **knowledge registry** for every active KID:

- KID
- title
- version
- owner
- lifecycle state
- authority tier
- freshness state
- applicability
- linked standards
- linked templates
- linked proofs
- compatibility label

KNO-02

Split knowledge content into explicit classes:

- authoritative rule
- guidance
- example
- anti-pattern
- reference
- experimental

KNO-03

Add a **knowledge retrieval report**:

- why retrieved
- why excluded
- retrieval inputs
- authority tier
- freshness state
- linked consumers
- replacement note if deprecated

KNO-04

Add freshness and supersession policy:

- review cadence
- stale thresholds
- deprecation thresholds
- superseded-by links
- legacy-use rules

KNO-05

Add knowledge drift checks:

- changed without version bump
- stale authoritative entry still active
- broken bundle membership
- broken cross-links to templates/standards
- deprecated knowledge still retrieved

KNO-06

Add knowledge health metrics:

- retrieval frequency
- stale count
- deprecated usage count
- orphaned unit count
- authority-tier distribution
- unsupported reference count

Final judgment on Knowledge

Knowledge is valuable, but under-governed for its likely influence on the system.

It has structure and strategic intent.
What it still lacks is a strong runtime governance model for **trust, freshness, applicability, and retrieval discipline**.

That is why it lands at **66%**.

Standards library scorecard

Standards library scorecard

Overall grade

27 / 32 = 84%
Band: Usable but thin, near governed

Standards is one of the stronger libraries in Axion. It already has the right shape to act as a real obligation layer, and it is much closer to being governed than Templates or Knowledge.

That said, it still has one major gap:

it defines obligations well, but it does not yet bind those obligations deeply enough downstream.

In other words, Standards is structurally solid, but it still needs tighter runtime linkage into:

- template selection
- gate invocation
- proof requirements
- policy constraints
- maintenance scans

ALC score breakdown — Standards

ALC-01 Identity

Score: 2/2

Pass.

Standards is clearly organized as a real library:

- stable section structure
- standards packs / categories
- resolved snapshot concepts
- standards registry/index behavior
- versioned artifacts

This is already stronger than several other libraries.

Main weakness:

- per-standard owner and lifecycle metadata should be more explicit and universal

---

ALC-02 Purpose and scope

Score: 2/2

Pass.

The role is very clear:

- define obligations
- organize applicability
- resolve the active standards set for a run
- produce resolved standards snapshots
- constrain downstream behavior

That boundary is cleanly separable from:

- **Knowledge** supplying support material
- **Templates** shaping outputs
- **Policy** governing permissions and overrides
- **Gates** evaluating compliance
- **Verification** defining proof sufficiency

This is a strong area.

ALC-03 Lifecycle

Score: 1/2

Partial.

What exists:

- standards are versionable
- standards packs and snapshots imply durable evolution
- resolution behavior implies managed change over time

What is missing:

- no full visible lifecycle doctrine per standard unit:
    - candidate
    - active
    - deprecated
    - superseded
    - retired
- no strong replacement-path handling for old standards
- no clearly universal supersession chains

Main issue:
Standards change slower than Knowledge, but when they do change, the blast radius is much bigger. Lifecycle discipline needs to reflect that.

ALC-04 Authority and trust tier

Score: 2/2

Pass.

Standards is clearly authoritative.

Why:

- it defines obligations
- it constrains what compliant output should include
- it is likely one of the primary drivers of gates and verification expectations

This is one of the libraries where authority is naturally strong.

Main weakness:

- a distinction between mandatory standards, recommended standards, and advisory standards should still be made more explicit

---

ALC-05 Applicability

Score: 2/2

Pass.

This is one of Standards' biggest strengths.

What already appears strong:

- standards applicability/resolution is part of the library's core design
- standards are meant to be selected based on context rather than universally applied
- resolved standards snapshots imply concrete resolution behavior

Main weakness:

- applicability predicates should become even more explicit and machine-readable at the unit level:
    - project type
    - stage
    - artifact class
    - risk class
    - exclusions
    - precedence

Still, compared with other libraries, Standards is ahead here.

ALC-06 Dependency mapping

Score: 1/2

Partial.

What exists:

- standards clearly influence templates, gates, verification, and likely planning
- resolved snapshots imply traceability at the pack/set level
- gate mapping already appears to be part of the standards surface

What is missing:

- each standard should explicitly declare:
    - linked templates
    - linked gates
    - linked proof requirements
    - linked policy constraints
    - linked knowledge support units
- no strong first-class downstream dependency graph yet

Main issue:
Standards is central enough that dependency mapping should be one of its strongest features, but it still looks more implied than explicit.

ALC-07 Resolution rules

Score: 2/2

Pass.

This is a strong area.

Why:

- standards resolution is already a named function of the library
- resolved snapshot format suggests deterministic selection
- the library clearly expects a run-specific standards set rather than a loose standards pile

Main weakness:

- operator-facing standards resolution reports should explain more clearly why a standard applied or was excluded

---

ALC-08 Structure and schema

Score: 2/2

Pass.

Standards has real governance structure:

- standards index/registry concepts
- applicability maps
- constraint packs
- resolved standards snapshots
- deterministic resolver rules

This is a real library, not a loose collection of notes.

ALC-09 Proof requirements

Score: 2/2

Pass.

Standards is already well positioned for proof linkage.

Why:

- standards naturally create obligations that must be evidenced
- gate mapping is already part of the design direction
- resolved snapshots make it possible to know exactly which standards need proof

Main weakness:

- every standard should explicitly declare its proof obligations and acceptable evidence classes

Still, this is strong enough to pass.

ALC-10 Change control

Score: 1/2

Partial.

What exists:

- versioning and snapshots support controlled evolution
- structured packs and maps make change traceable in principle

What is missing:

- no explicit mutation doctrine for standards:
  - create
  - revise
  - deprecate
  - supersede
  - retire
  - split/merge packs
- no obvious blast-radius requirement when a standard changes
- no formal rerun/revalidation doctrine for impacted assets

Main issue:
Standard changes are high-impact and need stricter change-control than is currently visible.

ALC-11 Backcompat

Score: 1/2

Partial.

What exists:

- resolved snapshots and versioning provide a strong base
- older runs can likely be associated with the standards set that applied at the time

What is missing:

- no explicit compatibility labels such as:
  - wording-only
  - interpretation-compatible
  - obligation-compatible
  - breaking
  - migration-required
- no formal handling for assets built under older standards
- no clear migration doctrine for downstream template/gate/proof impacts

Main issue:
Standards backcompat matters a lot because so many other libraries consume it.

ALC-12 Registry quality

Score: 2/2

Pass.

This is a strength.

Standards already appears to have:

- index behavior
- applicability map behavior
- pack/snapshot structure

- deterministic selection surfaces

That is stronger registry maturity than several other libraries.

Main weakness:

- registry richness should include lifecycle state, owner, compatibility status, and downstream dependency links per standard

---

ALC-13 Drift resistance

Score: 2/2

Pass.

Standards is relatively well positioned for drift resistance because:

- resolved snapshots can be compared over time
- structured applicability and packs make changes inspectable
- maintenance can audit standards inventories and mappings

This is stronger than Templates and Knowledge.

Main weakness:

- drift checks should explicitly catch silent applicability changes, mapping drift, and unversioned obligation changes

---

ALC-14 Maintenance readiness

Score: 2/2

Pass.

Standards is highly suitable for maintenance governance through:

- **Coverage Audit**
- **Drift Detection**
- **Backcompat Validation**
- **Rebuild Indexes/Registries**

- **Registry Integrity Cleanup**
  Response: Axion Updates Planning

Because it already has structured resolution and registry surfaces, it is a strong target for maintenance enforcement.

ALC-15 Observability

Score: 1/2

Partial.

What exists:

- resolved snapshots create visibility into applied standards
- mappings and packs can support reporting
- compliance behavior should be observable through gates and verification downstream

What is missing:

- no obvious first-class standards health metrics like:
  - application frequency by standard
  - never-triggered standard count
  - deprecated standard usage
  - standards conflict count
  - unresolved applicability collision count
  - downstream gate/proof coverage rate

Main issue:
Standards has good structure, but its live health model is still thinner than it should be.

ALC-16 Operator clarity

Score: 2/2

Pass.

Standards is comparatively legible.

An operator should already be able to understand:

- what standards exist
- that standards are resolved per run/context
- that snapshots capture active obligations
- that applicability matters

Main weakness:

- operators should more easily see exact downstream effects:
    - which templates a standard activates
    - which gates it triggers
    - which proof obligations it creates
    - what replaced it if superseded

Still, compared with several other libraries, Standards is fairly clear.

What Standards is already good at

Its strongest traits are:

1. Strong structural maturity

Standards already looks like a proper governed library.

2. Clear applicability intent

Unlike some other libraries, applicability is not an afterthought here.

3. Good resolution shape

Resolved snapshot behavior is a strong design choice.

4. Natural compatibility with governance

Standards is easy to plug into Maintenance, Gates, Verification, and System.

What is holding Standards back

1. Downstream dependency mapping is not deep enough

This is the biggest issue.

Standards should be the main connective layer between obligation and enforcement.

2. Lifecycle and supersession need more depth

Standards changes need explicit replacement logic and downstream migration consequences.

3. Backcompat needs to be formalized

Resolved snapshots help, but they are not enough on their own.

4. Observability is still thin

You should be able to see how standards are actually behaving in production use.

Priority fixes for Standards

STD-01

Create a richer **standards registry** for every active standard:

- standard ID
- title
- version
- owner
- lifecycle state
- authority class
- applicability predicates
- linked templates
- linked gates
- linked proof requirements
- compatibility label

STD-02

Add a **standards resolution report**:

- why applied
- why excluded
- selection inputs
- precedence path
- conflict resolution
- downstream obligations created
- replacement note if deprecated

STD-03

Add explicit downstream mappings:

- standard → template clauses
- standard → gate rules
- standard → proof types
- standard → policy hooks
- standard → knowledge supports

STD-04

Add compatibility labels for standards changes:

- wording-only
- interpretation-compatible
- obligation-compatible
- breaking
- migration-required

STD-05

Add standards drift checks:

- applicability changed without version movement
- downstream gate/template mapping drift
- deprecated standard still active
- unresolved conflict between active standards
- proof obligation changed silently

STD-06

Add standards health metrics:

- application count
- deprecated usage count
- conflict count
- never-triggered standard count
- downstream coverage count
- proof coverage rate

Final judgment on Standards

Standards is one of the stronger libraries in Axion.

It already has real governance shape and good resolution discipline.
Its next leap is to become the fully explicit **obligation-to-enforcement bridge** across the rest of the estate.

That is why it lands at **84%**.

Ops library scorecard

Ops library scorecard

Overall grade

18 / 32 = 56%
Band: At risk

Ops is the weakest major governance library in the estate.

That does **not** mean it is useless. It means the library is currently too thin for how important its role is supposed to be. Ops should be the place where Axion proves it can run like an enterprise system, not just describe one.

Ops should govern things like:

- observability expectations
- logging and tracing discipline
- monitoring and alerting
- SLO/SLA behavior
- performance budgets
- capacity and cost behavior
- operational evidence for release readiness

From the scan, Ops has some foundational JSON/contracts, but it does **not** yet have the same doctrine depth that stronger libraries such as Standards, System, Verification, or even Maintenance already have.

ALC score breakdown — Ops

ALC-01 Identity

Score: 1/2

Partial.

What exists:

- Ops has named sections and asset files

- there are identifiable models for areas like alerting, cost, budgets, logging, and SLO-like concerns

What is missing:

- per-unit governance identity is too light
- active ops units need stronger first-class metadata:
    - stable ID
    - version
    - owner
    - status
    - lifecycle state

Main issue:
Ops has identifiable content areas, but not yet a strongly governed unit model.

ALC-02 Purpose and scope

Score: 2/2

Pass.

The intended purpose is clear:

- define runtime reliability and operating expectations
- govern monitoring, alerting, logging, tracing, performance, capacity, and cost discipline
- support release/readiness and operational quality

That boundary is distinct from:

- **Telemetry** collecting event/metric output
- **Policy** constraining permissions
- **Verification** defining proof sufficiency
- **Maintenance** auditing library health

This part is clear enough.

ALC-03 Lifecycle

Score: 1/2

Partial.

What exists:

- Ops assets are versionable in principle
- the library can clearly evolve over time

What is missing:

- no strong visible lifecycle model for ops units:
    - candidate
    - active
    - deprecated
    - superseded
    - retired
- no clear replacement path for old alert standards, SLO rules, or performance budgets
- no obvious legacy handling for operational policies

Main issue:
Ops rules change often enough that lifecycle control should be explicit, not implied.

ALC-04 Authority and trust tier

Score: 1/2

Partial.

Ops should be strongly authoritative in production contexts, but right now that authority looks under-expressed.

What exists:

- ops content clearly implies expectations and thresholds
- performance budgets and SLO-like artifacts naturally carry constraint weight

What is missing:

- no clear split between:
    - required ops controls
    - recommended practices
    - example/reference material
    - experimental thresholds
- no explicit authority tier model per ops unit

Main issue:
Without authority tiers, ops guidance can blur into ops requirements.

ALC-05 Applicability

Score: 1/2

Partial.

What exists:

- Ops concepts naturally apply by environment, runtime profile, service class, and risk level
- budget and alerting models imply scoped use

What is missing:

- each ops unit should explicitly declare:
    - environment applicability
    - service/profile scope
    - stage or release phase
    - risk class
    - traffic/load class
    - exclusions
- no strong universal applicability model appears to be in place

Main issue:
Ops rules are only useful if they apply deterministically to the right situations.

ALC-06 Dependency mapping

Score: 1/2

Partial.

What exists:

- Ops should clearly connect to Telemetry, System quotas/capabilities, Verification evidence, and release/readiness standards
- there are natural ties to performance and cost policy surfaces

What is missing:

- no strong first-class graph such as:
  - ops rule → telemetry signal
  - ops budget → gate or release condition
  - alert policy → monitored component
  - SLO target → evidence/report
  - cost model → quota/policy hook
- downstream consumers are not surfaced deeply enough

Main issue:
Ops has natural dependencies everywhere, but they are not yet governed explicitly.

ALC-07 Resolution rules

Score: 1/2

Partial.

What exists:

- some ops artifacts likely imply scoped selection by environment/profile
- operational policies could in principle be resolved deterministically

What is missing:

- no obvious unified ops resolution doctrine:
  - what applies in dev vs prod
  - what applies by run profile
  - how thresholds are selected
  - how conflicts are resolved
  - what happens when an ops policy is missing

Main issue:
Ops still reads more like a set of definitions than a deterministic resolution surface.

ALC-08 Structure and schema

Score: 2/2

Pass.

This is one of the better parts of Ops.

What exists:

- structured models for alerting, cost, budgets, SLOs, and related ops concerns
- enough contract shape to count as a real library surface

Main weakness:

- structure exists, but the doctrine around that structure is still too thin.

ALC-09 Proof requirements

Score: 1/2

Partial.

What exists:

- Ops naturally implies evidence such as metrics, alerts, traces, latency results, and budget adherence
- the library could support proof-backed readiness

What is missing:

- each ops unit should define:
    - required evidence
    - proof cadence
    - acceptable measurement sources
    - pass/fail thresholds
    - exception handling
- no strong standalone ops proof doctrine is visible

Main issue:
Ops cannot be trusted as a release-quality library until evidence requirements are explicit.

ALC-10 Change control

Score: 1/2

Partial.

What exists:

- versioned ops assets are possible
- structured contracts make controlled revision feasible

What is missing:

- no explicit mutation doctrine for ops:
  - create
  - revise
  - deprecate
  - supersede
  - retire
  - emergency threshold patch
- no blast-radius policy for changing alerts, SLOs, budgets, or cost models

Main issue:
Ops changes can alter production behavior significantly and need stricter mutation handling.

ALC-11 Backcompat

Score: 0/2

Missing.

This is one of the clearest weak areas.

What is missing:

- no visible compatibility labels
- no clear migration path for changed SLOs, alert thresholds, performance budgets, or cost models
- no doctrine for interpreting historical results under old ops definitions

Main issue:
Without backcompat discipline, Ops cannot support stable enterprise operations over time.

ALC-12 Registry quality

Score: 1/2

Partial.

What exists:

- Ops appears to have identifiable files/models
- some inventory of ops concerns exists

What is missing:

- Ops needs a richer first-class registry with:
    - ops unit ID
    - title
    - version
    - owner
    - lifecycle state
    - authority tier
    - applicability keys
    - linked telemetry signals
    - linked proofs
    - compatibility status

Main issue:
Ops has assets, but not yet a mature governed registry.

ALC-13 Drift resistance

Score: 1/2

Partial.

What exists:

- structured ops contracts make drift scanning possible
- Maintenance could eventually audit some of this

What is missing:

- no obvious first-class drift model for:
    - threshold changed without version bump
    - prod alert policy diverged from registry
    - SLO target drift
    - missing telemetry backing an active ops rule
    - deprecated ops unit still in use

Main issue:
Operational drift is one of the most dangerous types of drift and needs direct treatment.

ALC-14 Maintenance readiness

Score: 2/2

Pass.

Ops should be governable through the maintenance modes already established:

- **Coverage Audit**
- **Drift Detection**
- **Backcompat Validation**
- **Rebuild Indexes/Registries**
- **Registry Integrity Cleanup**
  Response: Axion Updates Planning

The structure is thin, but it is still auditable enough to be a maintenance target.

ALC-15 Observability

Score: 0/2

Missing / structurally weak.

This is the most ironic gap in the library.

Ops is supposed to define observability standards, yet the library itself does not appear to have a mature health/observability model for:

- active ops rule coverage
- alert firing quality
- SLO breach trends
- trace coverage
- logging coverage
- budget violations
- stale ops definitions

Main issue:
Ops is supposed to govern observability, but it is not yet observably governing itself.

ALC-16 Operator clarity

Score: 1/2

Partial.

Why only partial:

- the broad purpose of Ops is understandable
- the asset topics are recognizable

What is missing:

- for any ops unit, an operator should be able to answer:
  - what it governs
  - where it applies
  - what telemetry proves it
  - what threshold is active
  - what changed last
  - what replaces it if deprecated
  - whether it is release-blocking or advisory

Main issue:
Ops is readable at the topic level, but not yet governance-legible at the unit level.

What Ops is already good at

Its strongest traits are:

1. The topic surface is correct

The library is aimed at the right operational domains.

2. It has some real contract shape

This is not a total placeholder. There are structured assets here.

3. It is strategically necessary

Ops is one of the clearest paths to making Axion enterprise-grade.

4. It is highly improvable

The foundation is thin, but the fix path is very clear.

What is holding Ops back

1. It lacks doctrine depth

This is the biggest issue.

Ops needs the same pattern stronger libraries already have:

- purpose
- boundaries
- applicability
- dependencies
- proofs
- lifecycle
- registry
- maintenance hooks
- definition of done

2. Backcompat is basically absent

This is a major weakness.

3. Observability is underbuilt

Ops should be one of the most measurable libraries in the estate.

4. Resolution and applicability are too implicit

Ops rules need to resolve deterministically by context.

5. Dependency mapping is too weak

Ops should connect directly to Telemetry, System, Gates, Verification, and release behavior.

Priority fixes for Ops

OPS-01

Turn Ops into a full doctrine library with explicit sections for:

- purpose and boundaries
- monitoring/alert standards
- logging/tracing standards
- SLO/SLA and error budget policy
- performance budgets and profiling
- cost models and quota hooks
- gates and evidence requirements
- minimum viable ops set

OPS-02

Create a full **ops registry** for every active ops unit:

- ops unit ID
- title
- version
- owner
- lifecycle state
- authority tier
- applicability
- linked telemetry signals
- linked proofs
- compatibility label
- release-blocking status

OPS-03

Add an **ops decision/report** contract:

- what rule applied
- environment/profile scope
- thresholds used
- telemetry source checked
- compliance result
- waiver/override status
- release impact

OPS-04

Add explicit compatibility labels:

- threshold-compatible
- measurement-compatible
- breaking
- migration-required
- emergency-only

OPS-05

Add ops drift checks:

- threshold changed without version movement
- telemetry source missing
- deprecated ops rule still active
- logging/tracing coverage mismatch
- SLO target drift
- alert route drift

OPS-06

Add ops health metrics:

- active coverage rate
- alert quality rate
- logging coverage rate
- trace coverage rate
- SLO breach rate
- budget violation rate
- stale ops unit count

Final judgment on Ops

Ops is the weakest major governance library right now.

It has the right topics, but not yet the right governance depth.
If you want Axion to feel enterprise-grade in live operation, Ops is one of the highest-return upgrade targets in the whole estate.

That is why it lands at **56%**.

Planning library scorecard

Planning library scorecard

Overall grade

26 / 32 = 81%
Band: Usable but thin

Planning is one of the stronger execution libraries in Axion. It already looks like a real bridge between upstream intent and downstream production.

That matters because Planning should be the place where Axion answers:

- what must be built
- in what sequence
- with what acceptance targets
- with what coverage expectations
- how obligations become executable work
- what is still unmapped or unresolved

Planning is not weak. Its main limitation is that it still needs tighter downstream and upstream traceability so it becomes a **fully governed obligation-to-work translator**, not just a structured work decomposition layer.

ALC score breakdown — Planning

ALC-01 Identity

Score: 2/2

Pass.

Planning appears to have:

- stable section structure
- named planning artifacts
- work breakdown and acceptance map concepts
- build-plan style contracts
- versionable outputs

This is enough to count as a governed library.

Main weakness:

- per-plan-unit owner/lifecycle metadata should be stronger and more universal

---

ALC-02 Purpose and scope

Score: 2/2

Pass.

The purpose is clear:

- translate resolved inputs into executable work
- define coverage expectations
- sequence work
- bind work to acceptance
- ensure build scope is explicit

That boundary is distinct from:

- **Canonical** defining source truth
- **Standards** defining obligations
- **Templates** defining output forms
- **Orchestration** running stages
- **Verification** proving sufficiency

This is one of Planning's strengths.

ALC-03 Lifecycle

Score: 1/2

Partial.

What exists:

- plans are clearly versionable
- planning outputs can evolve per run
- plan structure suggests controlled regeneration is possible

What is missing:

- no rich lifecycle model for planning units:
  - candidate
  - active
  - superseded
  - retired
  - stale
- no clear handling for old plans after upstream canonical/standards/template changes
- no strong replacement chain for revised planning artifacts

Main issue:
Planning needs stronger stale/superseded behavior because plan validity is highly dependent on upstream change.

ALC-04 Authority and trust tier

Score: 1/2

Partial.

Planning is not as inherently authoritative as Policy or Standards, but it still has real force because it determines what work gets executed.

What exists:

- plans clearly shape build scope
- acceptance maps give planning real downstream effect

What is missing:

- no clear distinction between:
  - required plan items
  - recommended plan items
  - optional enhancements
  - speculative/future work
- no explicit authority tier for planning units

Main issue:
Without work-class tiers, plans can over-expand or under-enforce.

ALC-05 Applicability

Score: 2/2

Pass.

Planning naturally applies by:

- run scope
- artifact class
- project type
- output requirements
- selected templates/obligations

This area looks stronger than in Templates or Knowledge.

Main weakness:

- applicability should be more explicit at the plan-item level for partial runs, reruns, and mode-specific execution

---

ALC-06 Dependency mapping

Score: 1/2

Partial.

What exists:

- Planning clearly depends on canonical truth, standards obligations, and likely template requirements
- downstream consumers likely include orchestration, verification, and template fill/render stages
- acceptance mapping implies dependency awareness

What is missing:

- each plan item should explicitly declare:
    - canonical source entity
    - linked standards obligations
    - linked template targets
    - linked gates
    - linked proofs

        - downstream stage consumers
- no strong first-class dependency graph per work item

Main issue:
Planning understands dependencies conceptually, but not yet deeply enough as governed graph edges.

ALC-07 Resolution rules

Score: 2/2

Pass.

This is a strength.

Why:

- planning is already about decomposition and sequencing
- build-plan and acceptance-map surfaces imply deterministic planning behavior
- coverage rules suggest plan completeness is not left entirely intuitive

Main weakness:

- planning decisions should be more explainable for why work items were included, omitted, or grouped

---

ALC-08 Structure and schema

Score: 2/2

Pass.

Planning appears to have real governance structure:

- work breakdown format
- acceptance map format
- build plan format
- coverage rules

That is a solid contract base.

Main weakness:

- plan-item schema should be richer on provenance, dependency links, and compatibility labels

---

ALC-09 Proof requirements

Score: 2/2

Pass.

Planning already sits close to proof because:

- acceptance mapping creates verifiable targets
- coverage rules imply measurable completeness
- downstream verification can consume plan expectations

This is stronger than several other libraries.

Main weakness:

- plan items should more explicitly declare what proof closes them and what proof is insufficient

---

ALC-10 Change control

Score: 1/2

Partial.

What exists:

- plan artifacts are structured and regenerable
- sequencing and coverage logic support controlled revision in principle

What is missing:

- no explicit mutation doctrine for planning:
  - create
  - revise
  - supersede

    - invalidate
    - split/merge work items
    - partial-rerun recomposition
- no clear blast-radius rule when upstream changes require plan regeneration

Main issue:
Planning changes can quietly alter execution scope, so change control needs to be stricter.

ALC-11 Backcompat

Score: 1/2

Partial.

What exists:

- structured plans and acceptance maps support historical comparison
- runs can likely be associated with the plan that governed them

What is missing:

- no explicit compatibility labels such as:
    - sequencing-compatible
    - acceptance-compatible
    - scope-breaking
    - migration-required
- no clear doctrine for older runs built under older planning assumptions
- no formal stale-plan policy after standards/template/canonical changes

Main issue:
Planning backcompat matters whenever plan changes affect completion expectations.

ALC-12 Registry quality

Score: 2/2

Pass.

Planning appears to have enough structured artifacts and predictable forms to support registry/index behavior well.

This is stronger than Templates, Knowledge, and Ops.

Main weakness:

- registry richness should include lifecycle state, upstream dependencies, downstream obligations, and stale status

---

ALC-13 Drift resistance

Score: 2/2

Pass.

Planning is relatively well positioned for drift resistance because:

- coverage rules can expose missing scope
- acceptance maps can expose closure mismatch
- structured plans can be compared against upstream changes

This makes Planning a good target for maintenance scanning.

Main weakness:

- drift checks should explicitly catch silent plan-scope changes, stale acceptance maps, and orphaned work items

---

ALC-14 Maintenance readiness

Score: 2/2

Pass.

Planning is a strong fit for maintenance governance through:

- **Coverage Audit**
- **Drift Detection**
- **Backcompat Validation**
- **Rebuild Indexes/Registries**
- **Registry Integrity Cleanup**
  Response: Axion Updates Planning

Because it has structured plan and coverage artifacts, it is very auditable.

ALC-15 Observability

Score: 1/2

Partial.

What exists:

- planning coverage and acceptance artifacts create some real visibility
- plan completeness and closure should be measurable

What is missing:

- no obvious first-class planning health metrics such as:
    - unmapped requirement count
    - orphaned work item count
    - stale plan count
    - acceptance gap count
    - never-executed plan item count
    - rerun churn rate
    - coverage deficit rate

Main issue:
Planning has inspection surfaces, but its health telemetry is still thinner than it should be.

ALC-16 Operator clarity

Score: 2/2

Pass.

Planning is comparatively legible.

An operator should already be able to understand:

- what work is in scope
- what the build plan includes
- what acceptance targets exist
- whether coverage is complete or incomplete

Main weakness:

- operators should more easily see exact upstream/downstream traceability for each work item

---

What Planning is already good at

Its strongest traits are:

1. Clear execution role

Planning cleanly translates inputs into actionable work.

2. Strong structure

Work breakdown, build plan, and acceptance map are the right primitives.

3. Good coverage mindset

Coverage rules make this more than just a task list.

4. Maintenance friendliness

Planning is highly auditable because it produces structured scope artifacts.

What is holding Planning back

1. Dependency mapping is not deep enough

This is the biggest issue.

Planning should explicitly connect each work item to:

- canonical sources
- standards obligations

- template outputs
- gate/proof closure

2. Lifecycle/staleness is under-modeled

Plans need stronger invalidation and supersession behavior when upstream libraries change.

3. Backcompat is too light

Planning changes can alter execution and acceptance, so compatibility needs more formal treatment.

4. Observability could be stronger

Planning should expose coverage health, stale scope, and closure risk more directly.

Priority fixes for Planning

PLN-01

Create a richer **plan registry** for every active plan unit:

- plan item ID
- title
- version
- owner
- lifecycle state
- authority tier
- upstream sources
- linked standards
- linked templates
- linked gates
- linked proofs
- stale status
- compatibility label

PLN-02

Add a **planning decision report**:

- why included
- why excluded
- decomposition source
- scope rationale
- sequencing rationale
- downstream obligations created
- stale note if upstream changed

PLN-03

Add explicit work-item mappings:

- plan item → canonical entity
- plan item → standards obligation
- plan item → template target
- plan item → gate/proof closure
- plan item → orchestration stage consumer

PLN-04

Add compatibility labels for planning changes:

- sequencing-compatible
- acceptance-compatible
- scope-breaking
- migration-required
- rerun-required

PLN-05

Add planning drift checks:

- plan changed without version movement
- acceptance map drift
- orphaned work item
- stale plan after upstream library change
- required output no longer covered

PLN-06

Add planning health metrics:

- coverage deficit count
- stale plan count
- orphaned item count
- acceptance gap count
- rerun churn rate
- never-closed item count

Final judgment on Planning

Planning is strong enough to be useful now and strong enough to become governed without a redesign.

Its main next leap is deeper traceability: turning each plan item into a fully governed bridge between **obligation, output, and proof**.

That is why it lands at **81%**.

Canonical library scorecard

Canonical library scorecard

Overall grade

28 / 32 = 88%
Band: Usable but thin, near governed

Canonical is one of the strongest libraries in Axion.

That fits its role. Canonical should be the place where Axion decides:

- what the normalized source truth is
- what entities exist
- what is known vs inferred vs unknown
- what assumptions are active
- what is duplicated or conflicting
- what downstream libraries are allowed to trust

From the scan and the broader Axion structure, Canonical already has a strong contract mindset. Its biggest remaining gap is not basic structure. Its biggest gap is becoming the **explicit provenance and invalidation authority** for the rest of the estate.

ALC score breakdown — Canonical

ALC-01 Identity

Score: 2/2

Pass.

Canonical already appears to have:

- stable section structure
- canonical spec/schema surfaces
- entity and dedupe concepts
- integrity and unknowns/assumptions surfaces
- versionable artifacts

This is a real governed library.

Main weakness:

- per-entity owner/lifecycle metadata should be richer and more universal

---

ALC-02 Purpose and scope

Score: 2/2

Pass.

The purpose is very clear:

- define normalized source truth
- reconcile input ambiguity
- track unknowns and assumptions
- preserve canonical integrity for downstream consumers

That boundary is cleanly distinct from:

- **Intake** collecting raw submissions
- **Planning** decomposing work
- **Templates** shaping outputs
- **Knowledge** supplying reusable support material
- **Verification** proving sufficiency

This is one of Canonical's strongest areas.

ALC-03 Lifecycle

Score: 1/2

Partial.

What exists:

- canonical artifacts are versionable
- entity normalization and integrity imply durable evolution
- assumptions/unknowns imply managed state over time

What is missing:

- no rich visible lifecycle doctrine for canonical units:
  - candidate
  - active
  - superseded
  - retired
  - stale
  - invalidated
- no strong replacement/invalidation model for canonical facts after upstream correction
- no explicit lifecycle handling for resolved unknowns or expired assumptions

Main issue:
Canonical truth changes less often than other libraries, but when it changes, downstream invalidation matters a lot.

ALC-04 Authority and trust tier

Score: 2/2

Pass.

Canonical is clearly authoritative.

Why:

- it defines what downstream consumers may treat as source truth
- it separates normalized truth from raw intake and from advisory knowledge
- it controls the boundary between fact, uncertainty, and assumption

This is one of the most naturally authoritative libraries in Axion.

Main weakness:

- it would still benefit from an explicit distinction between:
  - confirmed canonical fact
  - derived canonical inference
  - unresolved assumption
  - blocked/invalidated claim

ALC-05 Applicability

Score: 2/2

Pass.

Canonical applicability is strong because it is naturally keyed to:

- run context
- normalized entity set
- artifact consumption
- unresolved ambiguity state

Compared with libraries like Templates or Knowledge, applicability here is more disciplined by design.

Main weakness:

- consumers should be able to see more explicit consumption applicability by entity/field class

---

ALC-06 Dependency mapping

Score: 1/2

Partial.

What exists:

- Canonical clearly feeds Planning, Templates, Standards resolution context, and likely Verification expectations
- unknowns/assumptions and integrity surfaces imply downstream awareness
- dedupe/integrity handling naturally connects to intake normalization and planning validity

What is missing:

- each canonical entity or fact should explicitly declare:
    - source provenance
    - downstream consumers
    - dependent plan items
    - dependent template fields
    - dependent gates/proofs
    - invalidation blast radius
- no strong first-class downstream dependency graph yet

Main issue:
Canonical is upstream of almost everything, so dependency graphing should be even stronger than it appears now.

ALC-07 Resolution rules

Score: 2/2

Pass.

This is a strength.

Why:

- canonicalization is inherently about deterministic normalization
- dedupe, integrity, and unknown/assumption handling imply explicit resolution behavior
- the library already seems designed to avoid ad hoc truth formation

Main weakness:

- canonical decision reports should make normalization and assumption resolution more operator-visible

---

ALC-08 Structure and schema

Score: 2/2

Pass.

Canonical appears to have real governance structure:

- canonical spec/schema
- unknowns/assumptions surfaces
- dedupe rules
- integrity rules
- likely entity normalization contracts

This is a strong contract library.

ALC-09 Proof requirements

Score: 2/2

Pass.

Canonical is already well positioned for proof because:

- integrity rules imply validation
- dedupe logic implies evidence handling
- assumptions and unknowns imply explicit uncertainty handling
- downstream systems need to know what is trustworthy

This is one of Canonical's strengths.

Main weakness:

- canonical entities should expose more explicit proof-strength metadata and provenance evidence class

---

ALC-10 Change control

Score: 1/2

Partial.

What exists:

- structured canonical artifacts support controlled revision
- normalization/integrity logic supports disciplined updates in principle

What is missing:

- no explicit mutation doctrine for canonical units:
    - create
    - revise
    - invalidate
    - supersede
    - merge
    - split
    - resolve-unknown
    - retire-assumption
- no clear blast-radius requirement when canonical facts change

Main issue:
Canonical change is high impact and should trigger strict mutation and downstream invalidation behavior.

ALC-11 Backcompat

Score: 1/2

Partial.

What exists:

- versioned canonical artifacts can support historical consistency
- runs can likely be tied to the canonical state used at build time

What is missing:

- no explicit compatibility labels such as:
    - normalization-compatible
    - interpretation-compatible
    - schema-compatible
    - breaking
    - replan-required
- no clear doctrine for what happens to older runs when canonical truth changes
- no formal stale/invalidation policy for downstream consumers

Main issue:
Canonical backcompat is really about downstream validity, and that needs to be more explicit.

ALC-12 Registry quality

Score: 2/2

Pass.

Canonical appears to have enough structure and internal discipline to support strong registry/index behavior.

This is stronger than Templates, Knowledge, Gates, and Ops.

Main weakness:

- registry richness should include lifecycle state, provenance class, invalidation status, and downstream consumer links

---

ALC-13 Drift resistance

Score: 2/2

Pass.

Canonical is well positioned for drift resistance because:

- normalization and dedupe are inherently drift-sensitive
- integrity rules make deviations inspectable
- unknowns/assumptions surfaces give Maintenance something concrete to scan
- canonical state can be compared across runs

This is one of the stronger drift-ready libraries.

Main weakness:

- drift checks should explicitly catch silent interpretation changes, invalidated assumptions still marked active, and provenance mismatches

---

ALC-14 Maintenance readiness

Score: 2/2

Pass.

Canonical is highly suitable for maintenance governance through:

- **Coverage Audit**
- **Drift Detection**
- **Backcompat Validation**
- **Rebuild Indexes/Registries**
- **Registry Integrity Cleanup**
  Response: Axion Updates Planning

Because it is structured and upstream-critical, it is one of the most important maintenance targets.

ALC-15 Observability

Score: 1/2

Partial.

What exists:

- unknowns/assumptions and integrity outputs create some inspectability
- dedupe and normalization behavior should be measurable
- canonical state health is naturally auditable

What is missing:

- no obvious first-class canonical health metrics such as:
    - unresolved unknown count
    - active assumption count
    - invalidated fact count
    - duplicate entity rate
    - stale canonical consumer count
    - provenance confidence distribution

Main issue:
Canonical has inspection surfaces, but not yet a mature health telemetry layer for itself.

ALC-16 Operator clarity

Score: 2/2

Pass.

Canonical is comparatively legible.

An operator should already be able to understand:

- what the canonical set is for a run
- what unknowns and assumptions remain
- where integrity issues exist
- what has been normalized or deduped

Main weakness:

- operator visibility should better explain downstream blast radius when canonical state changes

---

What Canonical is already good at

Its strongest traits are:

1. Clear source-of-truth role

Canonical is easy to place in the architecture and clearly distinct from adjacent libraries.

2. Strong contract shape

Unknowns, assumptions, dedupe, and integrity are exactly the right primitives.

3. High governance maturity

Compared with many other libraries, Canonical already behaves like a serious governed surface.

4. Strong fit for maintenance and drift control

Because it is structured and upstream-critical, it is naturally auditable.

What is holding Canonical back

1. Dependency graphing is not deep enough

This is the biggest issue.

Canonical should explicitly know what downstream artifacts, plans, gates, and proofs depend on each fact or entity.

2. Change control needs more rigor

Canonical changes should trigger formal invalidation and regeneration behavior.

3. Backcompat should be expressed as downstream validity rules

Historical canonical states need clearer compatibility semantics.

4. Observability could be stronger

Canonical health should be more visible in runtime operations.

Priority fixes for Canonical

CAN-01

Create a richer **canonical registry** for every active canonical unit:

- canonical unit ID
- title
- version
- owner
- lifecycle state
- truth class
- provenance class
- dependency links
- invalidation status
- compatibility label

CAN-02

Add a **canonical decision report**:

- source inputs used
- normalization rules applied
- dedupe decisions made
- unknowns created
- assumptions created or resolved
- downstream impact note

CAN-03

Add explicit downstream mappings:

- canonical entity → plan items
- canonical entity → template fields
- canonical entity → standards applicability inputs
- canonical entity → gates/proofs
- canonical entity → rerun/rebuild triggers

CAN-04

Add compatibility labels for canonical changes:

- normalization-compatible
- interpretation-compatible
- schema-compatible
- breaking
- replan-required
- rerender-required

CAN-05

Add canonical drift checks:

- normalized value changed without version movement
- assumption remained active after resolution
- provenance mismatch
- duplicate entity resurfaced
- stale downstream consumer still bound to invalidated fact

CAN-06

Add canonical health metrics:

- unresolved unknown count
- active assumption count
- invalidated fact count
- duplicate rate
- stale consumer count
- provenance class distribution

Final judgment on Canonical

Canonical is one of the strongest libraries in Axion.

It already has the right primitives and good governance discipline.
Its next leap is to become the explicit **provenance and downstream invalidation authority** for the entire estate.

That is why it lands at **88%**.

Orchestration library scorecard

Orchestration library scorecard

Overall grade

27 / 32 = 84%
Band: Usable but thin, near governed

Orchestration is one of the stronger execution-governance libraries in Axion.

That fits its role. Orchestration should be the place where Axion decides:

- what stage runs when
- what each stage consumes and produces
- what gate blocks progression
- what a rerun invalidates
- what a resume is allowed to reuse
- what a run manifest must record
- how stage evidence is bound to execution history

From the scan, Orchestration already has real structure and strong deterministic intent. Its main gap is not basic shape. Its main gap is becoming the **full execution dependency and invalidation authority** across the runtime estate.

ALC score breakdown — Orchestration

ALC-01 Identity

Score: 2/2

Pass.

Orchestration appears to have:

- stable section structure
- stage and run concepts
- stage IO and gate model concepts
- run manifest and stage report contracts
- versionable assets

This is a real governed library.

Main weakness:

- per-stage owner/lifecycle metadata should be richer and more explicit

---

ALC-02 Purpose and scope

Score: 2/2

Pass.

The purpose is very clear:

- define stage order
- define stage IO contracts
- define gate transition points
- define run manifest/report behavior
- govern rerun/resume semantics

That boundary is distinct from:

- **Planning** deciding what work exists
- **System** resolving pins and profiles
- **Gates** evaluating conditions
- **Verification** defining proof sufficiency
- **Kit** packaging final outputs

This is one of Orchestration's strengths.

ALC-03 Lifecycle

Score: 1/2

Partial.

What exists:

- orchestration artifacts are versionable
- stage definitions are durable enough to evolve
- rerun/resume concepts imply controlled state transitions

What is missing:

- no rich lifecycle doctrine for orchestration units:
    - candidate stage
    - active stage
    - deprecated stage
    - superseded stage
    - retired stage
- no strong migration model when stage semantics change
- no universal lifecycle policy for run manifest schema evolution

Main issue:
Execution semantics change more dangerously than they first appear, so lifecycle discipline needs to be stronger.

ALC-04 Authority and trust tier

Score: 2/2

Pass.

Orchestration is clearly authoritative.

Why:

- it defines how execution progresses
- it determines whether a run can continue
- it governs reruns and resume behavior
- it binds execution history to state transitions

This is one of the more naturally binding libraries in Axion.

Main weakness:

- distinction between hard execution rules, recommended sequencing, and advisory execution notes should still be more explicit

---

ALC-05 Applicability

Score: 2/2

Pass.

Orchestration applicability is strong because it is naturally keyed to:

- run mode
- stage
- rerun/resume context
- artifact availability
- gate state

Compared with Templates, Knowledge, and Ops, applicability here is much more disciplined.

Main weakness:

- applicability should be more explicit for partial runs, stage subsets, and alternative execution profiles

---

ALC-06 Dependency mapping

Score: 1/2

Partial.

What exists:

- stage IO contracts clearly imply dependencies
- gates are tied to stage transitions
- run manifests and stage reports imply traceability
- rerun/resume semantics naturally depend on prior artifacts and gate state

What is missing:

- each stage should explicitly declare:
    - upstream artifact dependencies
    - downstream consumers
    - invalidation scope on rerun
    - reusable outputs on resume
    - linked gates
    - linked proof obligations
- no strong first-class blast-radius model for execution graph changes

Main issue:
Orchestration already knows a lot about dependencies, but it still needs a deeper governed dependency graph.

ALC-07 Resolution rules

Score: 2/2

Pass.

This is a major strength.

Why:

- stage order is explicit
- stage IO is explicit
- gate points are explicit
- rerun/resume are already treated as governed behaviors, not ad hoc actions

This is one of the better deterministic surfaces in the estate.

Main weakness:

- stage execution decision reports should explain more clearly why a stage ran, skipped, resumed, or invalidated downstream state

---

ALC-08 Structure and schema

Score: 2/2

Pass.

Orchestration has real governance structure:

- pipeline definition
- stage IO contracts
- gate model
- stage report format
- run manifest format

That is a strong contract base.

ALC-09 Proof requirements

Score: 2/2

Pass.

Orchestration is already close to proof because:

- stage reports are evidence-bearing artifacts
- gate transitions require recorded results
- run manifests support reproducibility and traceability
- rerun/resume semantics imply state evidence requirements

This is stronger than many other libraries.

Main weakness:

- every stage should more explicitly declare what proof is required before progression and what proof becomes stale after rerun

---

ALC-10 Change control

Score: 1/2

Partial.

What exists:

- stage definitions and manifests are structured
- execution behavior can be versioned in principle
- rerun/resume semantics imply controlled mutation paths

What is missing:

- no explicit mutation doctrine for orchestration:
    - add stage
    - revise stage
    - reorder stage
    - change IO contract
    - change gate binding
    - deprecate stage
    - retire stage
- no formal blast-radius standard for stage graph changes

Main issue:
Changing orchestration logic can invalidate large parts of the runtime estate, so change control needs to be stricter.

ALC-11 Backcompat

Score: 1/2

Partial.

What exists:

- run manifests and structured stage reports support historical association
- stage order and IO contracts can likely be pinned per run/version context

What is missing:

- no explicit compatibility labels such as:
  - stage-order-compatible
  - IO-compatible
  - gate-compatible
  - breaking
  - rerun-required
  - full-rebuild-required
- no clear doctrine for legacy runs under previous orchestration semantics

Main issue:
Orchestration backcompat is really about whether old run histories remain interpretable and reproducible after stage logic changes.

ALC-12 Registry quality

Score: 2/2

Pass.

Orchestration appears to have enough structure and determinism to support strong registry/index behavior.

This is stronger than Gates, Templates, Knowledge, and Ops.

Main weakness:

- registry richness should include lifecycle state, invalidation scope, compatibility status, and downstream stage dependency links

---

ALC-13 Drift resistance

Score: 2/2

Pass.

Orchestration is well positioned for drift resistance because:

- stage order can be compared
- IO contracts can be diffed
- gate bindings are inspectable
- rerun/resume semantics give Maintenance concrete state transitions to audit

This is one of the stronger drift-ready libraries.

Main weakness:

- drift checks should explicitly catch silent stage-order changes, IO drift, invalid rerun reuse, and gate-binding drift

---

ALC-14 Maintenance readiness

Score: 2/2

Pass.

Orchestration is highly suitable for maintenance governance through:

- **Coverage Audit**
- **Drift Detection**
- **Backcompat Validation**
- **Rebuild Indexes/Registries**
- **Registry Integrity Cleanup**
  Response: Axion Updates Planning

Because it is structured and execution-critical, it is one of the key maintenance targets.

ALC-15 Observability

Score: 1/2

Partial.

What exists:

- stage reports
- run manifests
- gate transitions
- rerun/resume records

That is already a meaningful observability base.

What is missing:

- no obvious first-class orchestration health metrics such as:
    - stage failure rate
    - rerun frequency
    - resume reuse rate
    - invalidation breadth
    - gate-block frequency by stage
    - skipped-stage anomaly count
    - stale-manifest count

Main issue:
Orchestration is inspectable, but its own runtime health model is still thinner than it should be.

ALC-16 Operator clarity

Score: 2/2

Pass.

Orchestration is comparatively legible.

An operator should already be able to understand:

- stage order
- what each stage consumes and produces

- where gates occur
- what a run manifest contains
- that rerun/resume are governed behaviors

Main weakness:

- operators should more easily see exact downstream invalidation when a stage is rerun or a contract changes

---

What Orchestration is already good at

Its strongest traits are:

1. Strong deterministic shape

Stage order, stage IO, gate points, and run manifests are exactly the right primitives.

2. Clear architectural role

Orchestration is easy to place in the system and clearly distinct from adjacent libraries.

3. Good compatibility with maintenance

Because it is structured and execution-critical, it is highly auditable.

4. Real runtime value

This library is clearly not passive documentation. It governs actual execution behavior.

What is holding Orchestration back

1. Dependency graphing is not deep enough

This is the biggest issue.

It should explicitly know:

- what outputs depend on each stage
- what reruns invalidate
- what resumes can safely reuse
- what proofs are stale after execution changes

2. Change control needs more rigor

Stage graph changes should be treated as high-impact mutations.

3. Backcompat needs to be formalized

Old runs need clearer interpretation and replay guarantees under changed orchestration semantics.

4. Observability could be stronger

Orchestration health should be visible as a first-class operating surface.

Priority fixes for Orchestration

ORC-01

Create a richer **orchestration registry** for every active stage/unit:

- stage ID
- title
- version
- owner
- lifecycle state
- stage order
- IO contract refs
- linked gates
- invalidation scope
- resume reuse policy
- compatibility label

ORC-02

Add a **stage decision report**:

- why this stage ran
- why skipped or resumed
- input artifact set
- gate basis
- reuse basis
- invalidated downstream targets
- replay fingerprint

ORC-03

Add explicit execution dependency mappings:

- stage → upstream artifacts
- stage → downstream artifacts
- stage → linked gates
- stage → proof obligations
- stage → rerun invalidation targets
- stage → reusable resume outputs

ORC-04

Add compatibility labels for orchestration changes:

- stage-order-compatible
- IO-compatible
- gate-compatible
- breaking
- rerun-required
- full-rebuild-required

ORC-05

Add orchestration drift checks:

- stage order changed without version movement
- IO contract drift
- gate binding drift
- invalid resume reuse
- stale manifest against active orchestration definition

ORC-06

Add orchestration health metrics:

- stage fail rate
- rerun rate
- resume reuse rate
- invalidation breadth
- gate-block rate by stage
- stale manifest count

Final judgment on Orchestration

Orchestration is one of the stronger libraries in Axion.

It already has the right structural primitives and good deterministic discipline.
Its next leap is to become the explicit **execution dependency and invalidation authority** for the runtime estate.

That is why it lands at **84%**.

Intake library scorecard

Intake library scorecard

Overall grade

27 / 32 = 84%
Band: Usable but thin, near governed

Intake is one of the stronger libraries in Axion.

That makes sense because Intake is the system's first formal control surface. It should decide:

- what a valid submission is
- how raw inputs are normalized
- what is missing or malformed
- what ambiguity must be surfaced
- what gets rejected early
- what downstream libraries are allowed to consume

From the scan and the broader library pattern, Intake already has strong contract discipline. Its main gap is not basic structure. Its main gap is becoming the fully explicit **submission provenance and ambiguity authority** for the rest of the estate.

ALC score breakdown — Intake

ALC-01 Identity

Score: 2/2

Pass.

Intake appears to have:

- stable section structure
- submission and normalized-input concepts
- validation surfaces
- submission-record contracts
- versionable artifacts

This is a real governed library.

Main weakness:

- per-form/per-surface owner metadata should be richer and more universal

---

ALC-02 Purpose and scope

Score: 2/2

Pass.

The purpose is clear:

- define the valid entry surface
- normalize incoming submissions
- reject malformed or incomplete inputs
- produce a submission record
- hand clean input into downstream canonicalization

That boundary is distinct from:

- **Canonical** defining source truth
- **System** resolving runtime settings
- **Planning** decomposing work
- **Knowledge** supplying reusable support content

This is one of Intake's strengths.

ALC-03 Lifecycle

Score: 1/2

Partial.

What exists:

- intake artifacts are versionable
- submission and normalized-input shapes can evolve
- validation logic is structured enough to manage change

What is missing:

- no rich visible lifecycle doctrine for intake units:
  - candidate
  - active
  - deprecated
  - superseded
  - retired
- no strong migration path for changed submission schemas
- no explicit handling for legacy submission forms still present in old runs

Main issue:
Entry surfaces change less often than templates, but schema changes here have wide downstream effects.

ALC-04 Authority and trust tier

Score: 2/2

Pass.

Intake is clearly authoritative for entry validity.

Why:

- it defines whether a submission is acceptable
- it controls normalization of raw inputs
- it determines what ambiguity is surfaced versus silently accepted
- it creates the initial governed record

This is one of the more naturally authoritative libraries.

Main weakness:

- it would still benefit from explicit distinction between:
  - hard-required fields
  - conditionally required fields
  - advisory fields
  - ignored/unsupported fields

ALC-05 Applicability

Score: 2/2

Pass.

Intake applicability is naturally strong because it is keyed to:

- submission type
- project/profile type
- input form variant
- field presence/requirements
- normalization path

Compared with Knowledge, Templates, and Ops, applicability here is much more disciplined.

Main weakness:

- applicability should be more explicit for alternate intake profiles and specialized submission modes

---

ALC-06 Dependency mapping

Score: 1/2

Partial.

What exists:

- Intake clearly feeds Canonical
- normalized inputs likely affect Standards applicability, Planning scope, and Template selection downstream
- validation and submission records imply provenance awareness

What is missing:

- each intake field/surface should explicitly declare:
    - canonical target mapping
    - normalization rules
    - ambiguity class
    - downstream consumers
    - invalidation impact if corrected later
- no strong first-class downstream dependency graph yet

Main issue:
Intake is upstream enough that dependency mapping should be deeper and more explicit.

ALC-07 Resolution rules

Score: 2/2

Pass.

This is a strength.

Why:

- intake is inherently about deterministic validation and normalization
- malformed versus acceptable input should be explicitly decidable
- normalized input and submission record concepts imply structured resolution behavior

Main weakness:

- intake decision reports should make field-level accept/reject/normalize outcomes more operator-visible

---

ALC-08 Structure and schema

Score: 2/2

Pass.

Intake appears to have real governance structure:

- submission format
- normalized-input format
- submission record
- validation surfaces

That is a strong contract base.

ALC-09 Proof requirements

Score: 2/2

Pass.

Intake is already well positioned for proof because:

- validation outcomes are evidence-bearing
- normalization decisions can be recorded
- rejected/missing/ambiguous inputs can be surfaced explicitly
- downstream systems need to know what was accepted and why

This is one of Intake's strengths.

Main weakness:

- intake records should expose more explicit proof classes for accepted normalization, unresolved ambiguity, and rejection basis

---

ALC-10 Change control

Score: 1/2

Partial.

What exists:

- structured schemas support controlled revision
- normalized-input and submission-record artifacts allow disciplined change in principle

What is missing:

- no explicit mutation doctrine for intake:
    - add field
    - revise field
    - tighten validation
    - relax validation
    - deprecate field
    - retire form variant
    - split/merge submission type
- no clear blast-radius rule when intake schema changes

Main issue:
Changes to intake rules can silently alter downstream canonical and planning behavior.

ALC-11 Backcompat

Score: 1/2

Partial.

What exists:

- versioned intake artifacts can support historical consistency
- old runs can likely be associated with the submission/normalized form used at the time

What is missing:

- no explicit compatibility labels such as:
  - field-compatible
  - normalization-compatible
  - validation-compatible
  - breaking
  - migration-required
- no clear doctrine for older submissions under older validation rules
- no explicit re-normalization policy when intake rules change

Main issue:
Intake backcompat matters because it changes what the system accepts and how raw input becomes governed input.

ALC-12 Registry quality

Score: 2/2

Pass.

Intake appears to have enough structure to support strong registry/index behavior.

This is stronger than Templates, Knowledge, Gates, and Ops.

Main weakness:

- registry richness should include lifecycle state, ambiguity class, downstream target mappings, and compatibility status

---

ALC-13 Drift resistance

Score: 2/2

Pass.

Intake is well positioned for drift resistance because:

- validation rules can be compared
- normalized-input behavior can be diffed
- submission-record outputs give Maintenance something concrete to audit
- schema drift is structurally detectable

This is one of the stronger drift-ready libraries.

Main weakness:

- drift checks should explicitly catch silent validation changes, normalization-rule drift, and deprecated intake surfaces still in use

---

ALC-14 Maintenance readiness

Score: 2/2

Pass.

Intake is highly suitable for maintenance governance through:

- **Coverage Audit**
- **Drift Detection**
- **Backcompat Validation**
- **Rebuild Indexes/Registries**
- **Registry Integrity Cleanup**
  Response: Axion Updates Planning

Because it is structured and upstream-critical, it is a strong maintenance target.

ALC-15 Observability

Score: 1/2

Partial.

What exists:

- validation outcomes
- normalized-input outputs
- submission records
- reject/accept behavior

That already gives Intake meaningful inspectability.

What is missing:

- no obvious first-class intake health metrics such as:
    - rejection rate
    - missing-required-field rate
    - ambiguity rate
    - normalization-failure rate
    - deprecated form usage
    - downstream remap count after corrections

Main issue:
Intake is inspectable, but its health model is still thinner than it should be.

ALC-16 Operator clarity

Score: 2/2

Pass.

Intake is comparatively legible.

An operator should already be able to understand:

- what was submitted
- what normalized successfully
- what failed validation
- what ambiguity remains
- what record was created

Main weakness:

- operators should more easily see exact downstream effects of correcting an intake field after initial acceptance

---

What Intake is already good at

Its strongest traits are:

1. Clear entry-control role

Intake is easy to place in the architecture and clearly distinct from Canonical.

2. Strong contract shape

Submission, normalization, validation, and record creation are exactly the right primitives.

3. Good governance maturity

Compared with several other libraries, Intake already behaves like a serious governed surface.

4. Strong fit for maintenance and drift control

Because it is structured and upstream-critical, it is naturally auditable.

What is holding Intake back

1. Dependency graphing is not deep enough

This is the biggest issue.

Intake should explicitly know what downstream canonical entities, plan items, and template fields depend on each accepted/normalized field.

2. Change control needs more rigor

Intake schema and rule changes should trigger formal downstream impact analysis.

3. Backcompat should be more explicit

Old submissions and re-normalization behavior need clearer compatibility semantics.

4. Observability could be stronger

Intake health should be visible as a first-class operating surface.

Priority fixes for Intake

INT-01

Create a richer **intake registry** for every active intake unit:

- intake unit ID
- title
- version
- owner
- lifecycle state
- requirement class
- normalization rule refs
- canonical target refs
- ambiguity class
- compatibility label

INT-02

Add an **intake decision report**:

- submitted values
- validation rules applied
- normalization rules applied
- rejected fields
- ambiguous fields
- accepted output
- downstream impact note

INT-03

Add explicit downstream mappings:

- intake field → canonical entity/field
- intake field → standards applicability input

- intake field → planning scope input
- intake field → template selection input
- intake correction → rerun/rebuild triggers

INT-04

Add compatibility labels for intake changes:

- field-compatible
- normalization-compatible
- validation-compatible
- breaking
- migration-required
- renormalization-required

INT-05

Add intake drift checks:

- validation changed without version movement
- normalization rule drift
- deprecated intake surface still active
- downstream mapping mismatch
- accepted ambiguity no longer classified correctly

INT-06

Add intake health metrics:

- rejection rate
- ambiguity rate
- normalization failure rate
- missing-required-field rate
- deprecated surface usage
- downstream remap count

Final judgment on Intake

Intake is one of the stronger libraries in Axion.

It already has the right primitives and solid contract discipline.
Its next leap is to become the explicit **submission provenance and ambiguity authority** for the rest of the estate.

That is why it lands at **84%**.

Telemetry library scorecard

Telemetry library scorecard

Overall grade

25 / 32 = 78%
Band: Usable but thin

Telemetry is a meaningful support library, but it is not yet as mature as System, Canonical, Standards, Verification, or Orchestration.

That matters because Telemetry should be the place where Axion answers:

- what events and metrics must exist
- what gets emitted at each stage
- what gets redacted
- what is retained
- what can be used for health analysis
- what signals prove runtime behavior actually happened

Right now, Telemetry has the right foundation, but it still needs stronger governance around lifecycle, dependency binding, compatibility, and operational health semantics.

ALC score breakdown — Telemetry

ALC-01 Identity

Score: 2/2

Pass.

Telemetry appears to have:

- stable section structure
- event and metric model surfaces
- sink/privacy/redaction contracts
- versionable telemetry assets

This is enough to count as a real governed library.

Main weakness:

- per-signal and per-schema owner metadata should be more explicit

---

ALC-02 Purpose and scope

Score: 2/2

Pass.

The purpose is clear:

- define telemetry events and metrics
- govern sinks and routing
- govern privacy/redaction behavior
- support observability and auditability of runs and system behavior

That boundary is distinct from:

- **Ops** defining operational expectations
- **Audit** defining action records
- **System** resolving runtime policy/hooks
- **Maintenance** scanning structural health

This is one of Telemetry's strengths.

ALC-03 Lifecycle

Score: 1/2

Partial.

What exists:

- telemetry schemas and signal definitions are versionable
- sink and redaction rules can evolve in a structured way

What is missing:

- no rich visible lifecycle model for telemetry units:
    - candidate
    - active
    - deprecated
    - superseded
    - retired
- no clear migration path for old event schemas or metric definitions
- no obvious doctrine for retiring signals without losing interpretability

Main issue:
Telemetry changes can silently break monitoring and historical comparability.

ALC-04 Authority and trust tier

Score: 1/2

Partial.

Telemetry is not as inherently binding as Policy or Gates, but it still has real authority because it determines what runtime signals exist and what evidence gets captured.

What exists:

- signal definitions and redaction/privacy rules imply control weight

What is missing:

- no explicit distinction between:
    - required telemetry
    - recommended telemetry
    - diagnostic-only telemetry
    - experimental telemetry
- no authority tier model per signal family

Main issue:
Without signal classes, telemetry can become noisy or under-instrumented.

ALC-05 Applicability

Score: 2/2

Pass.

Telemetry applicability is naturally strong because it is scoped by:

- stage or subsystem
- event type
- sink type

- privacy class
- runtime context

This is stronger than in Knowledge, Templates, and Ops.

Main weakness:

- applicability should be more explicit for profile/environment-specific signal suppression or elevation

---

ALC-06 Dependency mapping

Score: 1/2

Partial.

What exists:

- Telemetry clearly depends on runtime producers
- Ops, Maintenance, System, and possibly Audit depend on telemetry outputs
- privacy/redaction rules imply dependencies on data class and sink behavior

What is missing:

- each signal should explicitly declare:
    - producer
    - consumers
    - linked ops rules
    - linked maintenance scans
    - linked privacy class

    - linked retention policy
- no strong first-class producer/consumer graph yet

Main issue:
Telemetry is cross-cutting enough that dependency graphing should be much richer.

ALC-07 Resolution rules

Score: 2/2

Pass.

This is a solid area.

Why:

- telemetry emission is naturally rule-driven
- sinks and redaction imply deterministic routing/transform behavior
- telemetry definitions appear structured enough to resolve predictably

Main weakness:

- signal emission decisions should be easier to explain at runtime

---

ALC-08 Structure and schema

Score: 2/2

Pass.

Telemetry appears to have real governance structure:

- event model

- metric model
- sink policies
- privacy/redaction rules
- telemetry-related contracts

That is a real library surface.

ALC-09 Proof requirements

Score: 1/2

Partial.

What exists:

- telemetry naturally provides evidence of runtime behavior
- event emissions and metric streams can support proof indirectly

What is missing:

- Telemetry itself should define:
    - required signal coverage
    - signal integrity expectations
    - acceptable loss/failure thresholds
    - proof of emission and routing
    - proof of redaction correctness
- no strong standalone proof doctrine for telemetry integrity

Main issue:
Telemetry is evidentiary, but it does not yet fully prove itself.

ALC-10 Change control

Score: 1/2

Partial.

What exists:

- versioned schemas and structured signal definitions make change control possible

What is missing:

- no explicit mutation doctrine for telemetry:
    - add signal
    - revise signal
    - rename signal
    - deprecate signal
    - change sink routing
    - change redaction class
    - retire schema
- no clear blast-radius policy for telemetry changes

Main issue:
Telemetry changes can silently break dashboards, alerts, and audits.

ALC-11 Backcompat

Score: 1/2

Partial.

What exists:

- schema versioning can support compatibility in principle
- historical runs can likely be tied to signal formats that existed at the time

What is missing:

- no explicit compatibility labels such as:
    - payload-compatible
    - routing-compatible
    - redaction-compatible
    - breaking
    - migration-required
- no clear doctrine for historical comparison when signals change

Main issue:
Telemetry without backcompat discipline weakens trend analysis and operational trust.

ALC-12 Registry quality

Score: 2/2

Pass.

Telemetry appears to have enough structure to support strong registry/index behavior for events, metrics, sinks, and privacy classes.

This is stronger than Gates, Knowledge, Templates, and Ops.

Main weakness:

- registry richness should include lifecycle state, producer/consumer links, compatibility status, and authority tier

---

ALC-13 Drift resistance

Score: 2/2

Pass.

Telemetry is well positioned for drift resistance because:

- signal definitions can be diffed
- sink behavior can be compared
- redaction rules are structurally auditable
- Maintenance can inspect schema and registry drift

This is one of the stronger support libraries for drift readiness.

Main weakness:

- drift checks should explicitly catch silent payload changes, routing drift, and deprecated signals still being emitted

---

ALC-14 Maintenance readiness

Score: 2/2

Pass.

Telemetry is highly suitable for maintenance governance because it is structured and registry-friendly. It should be a strong target for:

- Coverage Audit
- Drift Detection
- Backcompat Validation

- Rebuild Indexes/Registries
- Registry Integrity Cleanup

ALC-15 Observability

Score: 2/2

Pass.

This is naturally one of Telemetry's strongest areas.

What exists:

- the library's entire purpose is to define observable runtime signals
- event and metric structures provide a strong base for visibility

What is missing:

- more explicit self-health metrics for telemetry itself, such as:
    - dropped event rate
    - sink failure rate
    - redaction error rate
    - schema mismatch rate
    - deprecated signal rate

Still, Telemetry earns a pass here.

ALC-16 Operator clarity

Score: 1/2

Partial.

Why only partial:

- operators can likely understand what classes of telemetry exist
- event/metric/sink concepts are straightforward

What is missing:

- for any signal, an operator should be able to answer:
    - why it exists
    - who emits it
    - where it goes
    - what privacy class applies
    - what changed last
    - whether it is required or optional
    - what consumes it downstream

Main issue:
Telemetry is structured, but not yet fully governance-legible at the signal level.

What Telemetry is already good at

Its strongest traits are:

1. Clear role

Telemetry is easy to place in the architecture.

2. Good structure

Events, metrics, sinks, and redaction are the right primitives.

3. Strong maintenance fit

Telemetry is naturally auditable and drift-checkable.

4. Real runtime value

This library clearly governs live runtime evidence, not passive notes.

What is holding Telemetry back

1. Dependency graphing is too light

Telemetry should explicitly connect producers, consumers, ops policies, privacy classes, and maintenance scans.

2. Lifecycle and backcompat need more rigor

Signal evolution needs to preserve interpretability over time.

3. Proof doctrine is too thin

Telemetry should define how signal integrity and routing correctness are proven.

4. Operator clarity should improve

Signal-level explanation needs to be much more direct.

Priority fixes for Telemetry

TEL-01

Create a richer **telemetry registry** for every active signal/unit:

- signal ID
- title
- version
- owner
- lifecycle state
- authority tier
- producer
- consumers
- privacy class
- sink refs
- retention policy
- compatibility label

TEL-02

Add a **telemetry decision report**:

- why emitted
- producer source
- routing decision
- sink targets
- redaction applied
- retention class

- downstream consumers

TEL-03

Add explicit dependency mappings:

- signal → producer
- signal → sink
- signal → privacy/redaction policy
- signal → ops rule consumer
- signal → maintenance health consumer
- signal → audit/report consumer

TEL-04

Add compatibility labels for telemetry changes:

- payload-compatible
- routing-compatible
- redaction-compatible
- breaking
- migration-required

TEL-05

Add telemetry drift checks:

- payload changed without version movement
- routing drift

- redaction rule drift
- deprecated signal still emitted
- sink missing for active required signal

TEL-06

Add telemetry self-health metrics:

- dropped event rate
- sink failure rate
- redaction error rate
- schema mismatch rate
- deprecated signal count
- missing required signal count

Final judgment on Telemetry

Telemetry is a solid support library with real runtime value.

It already has the right structural primitives.
Its next leap is becoming a fully governed **signal authority** with stronger lifecycle, dependency, and compatibility discipline.

That is why it lands at **78%**.

Audit library scorecard

Audit library scorecard

Overall grade

27 / 32 = 84%
Band: Usable but thin, near governed

Audit is one of the stronger support-governance libraries in Axion.

That fits its role. Audit should be the place where Axion answers:

- who changed what
- when it changed
- why it changed
- what the intended outcome was
- what evidence supports the action
- what the blast radius was
- whether the record can be trusted later

From the scan and the broader estate pattern, Audit already has real contract discipline. Its main gap is not basic structure. Its main gap is becoming the explicit **mutation accountability and impact-trace authority** for the whole estate.

ALC score breakdown — Audit

ALC-01 Identity

Score: 2/2

Pass.

Audit appears to have:

- stable section structure
- actor/action/target/outcome concepts
- action ledger schema
- index/query key concepts
- integrity and gate/check surfaces
- versionable artifacts

This is a real governed library.

Main weakness:

- per-record owner and retention responsibility should be more explicit

---

ALC-02 Purpose and scope

Score: 2/2

Pass.

The purpose is clear:

- record operator and system actions
- preserve traceability of changes and events
- support investigation, review, export, and retention
- bind actions to accountable actors and outcomes

That boundary is distinct from:

- **Telemetry** emitting runtime signals

- **System** resolving runtime configuration
- **Maintenance** performing scans/remediations
- **Policy** deciding permissions
- **Verification** proving sufficiency

This is one of Audit's strengths.

ALC-03 Lifecycle

Score: 1/2

Partial.

What exists:

- audit records and schemas are versionable
- retention/redaction/export concepts imply lifecycle awareness
- append-only behavior suggests durability discipline

What is missing:

- no rich visible lifecycle doctrine for audit units:
    - candidate schema
    - active schema
    - deprecated schema
    - superseded schema
    - retired/exported archive state
- no strong treatment for lifecycle of audit indexes, exports, and redaction outcomes

- no explicit replacement path for older record envelopes

Main issue:
Audit data lives long enough that lifecycle and archival semantics should be stronger.

ALC-04 Authority and trust tier

Score: 2/2

Pass.

Audit is clearly authoritative in its domain.

Why:

- it determines what counts as an accountable action record
- it preserves actor/action/outcome history
- it supports later review and investigation
- it provides the official trace of mutation and operation history

This is one of the more naturally authoritative support libraries.

Main weakness:

- it would still benefit from explicit trust classes such as canonical audit record, derived audit summary, export artifact, and redacted view

---

ALC-05 Applicability

Score: 2/2

Pass.

Audit applicability is strong because it is naturally keyed to:

- actor type
- action class
- target class
- time/order
- retention/export/redaction context

Compared with Templates, Knowledge, and Ops, applicability here is much more disciplined.

Main weakness:

- applicability should be more explicit for which actions are mandatory-to-log versus optional-to-log

---

ALC-06 Dependency mapping

Score: 1/2

Partial.

What exists:

- Audit clearly depends on action producers across the estate
- Maintenance, Policy review, operator investigation, and governance reporting likely consume audit records
- retention and export workflows imply downstream consumers and lifecycle dependencies

What is missing:

- each audit record class should explicitly declare:
    - producers

    - required actor fields
    - target classes
    - downstream consumers
    - linked policy/approval classes
    - linked blast-radius or proof references
- no strong first-class graph for mutation → audit → review/export/investigation

Main issue:
Audit sits across everything, so producer/consumer mapping should be stronger.

ALC-07 Resolution rules

Score: 2/2

Pass.

This is a strength.

Why:

- append-only rules imply deterministic write behavior
- index/query key definitions imply deterministic retrieval behavior
- mandatory fields such as actor/action/target/outcome make audit resolution structurally sound

Main weakness:

- audit decision/reporting views should explain more clearly why a given action was logged in a particular class or retention bucket

---

ALC-08 Structure and schema

Score: 2/2

Pass.

Audit appears to have real governance structure:

- action model
- ledger schema
- query/index model
- integrity rules
- ops workflow for retention/redaction/export
- gates/checks

That is a strong contract base.

ALC-09 Proof requirements

Score: 2/2

Pass.

Audit is already close to proof because:

- integrity rules imply trust validation
- append-only rules imply tamper-resistance discipline
- actor/action presence and monotonic timestamps create record validity conditions
- exports/redactions require controlled handling

This is one of Audit's strengths.

Main weakness:

- stronger explicit proof classes for tamper-evidence, redaction legitimacy, and export completeness would improve it further

---

ALC-10 Change control

Score: 1/2

Partial.

What exists:

- append-only and integrity mindset support disciplined mutation handling
- schemas and retention/export processes make controlled change possible

What is missing:

- no explicit mutation doctrine for audit artifacts:
    - schema revise
    - index revise
    - retention policy revise
    - redaction event
    - export event
    - archival transition
- no strong blast-radius policy for changing audit schema or index behavior

Main issue:
Changing audit semantics is high impact and should itself be governed more explicitly.

ALC-11 Backcompat

Score: 1/2

Partial.

What exists:

- versioned schemas and append-only mindset support historical continuity
- exported records can likely be tied to the schema active at the time

What is missing:

- no explicit compatibility labels such as:
    - schema-compatible
    - query-compatible
    - export-compatible
    - breaking
    - migration-required
- no clear doctrine for how old records remain queryable after schema/index changes

Main issue:
Audit loses value if older records become hard to interpret or compare.

ALC-12 Registry quality

Score: 2/2

Pass.

Audit appears to have enough structure to support strong registry/index behavior.

This is stronger than Gates, Knowledge, Templates, and Ops.

Main weakness:

- registry richness should include lifecycle state, retention class, producer classes, and compatibility status

---

ALC-13 Drift resistance

Score: 2/2

Pass.

Audit is well positioned for drift resistance because:

- append-only rules make silent mutation easier to detect
- integrity rules make record degradation inspectable
- index/query key structures give Maintenance concrete targets to compare
- actor/timestamp requirements support consistency checks

This is one of the stronger drift-ready support libraries.

Main weakness:

- drift checks should explicitly catch missing mandatory actors, broken monotonicity, stale indexes, and redaction/export mismatch

---

ALC-14 Maintenance readiness

Score: 2/2

Pass.

Audit is highly suitable for maintenance governance through:

- **Coverage Audit**
- **Drift Detection**

- **Backcompat Validation**
- **Rebuild Indexes/Registries**
- **Registry Integrity Cleanup**

Because it is structured, append-oriented, and index-driven, it is a strong maintenance target.

ALC-15 Observability

Score: 1/2

Partial.

What exists:

- audit records, indexes, and export/redaction workflows provide meaningful inspectability
- integrity failures and missing-field conditions should be measurable
- audit volume and queryability are naturally observable

What is missing:

- no obvious first-class audit health metrics such as:
    - missing actor rate
    - monotonic timestamp violation count
    - stale index count
    - redaction lag
    - export completeness failures
    - orphaned target references
    - append failure count

Main issue:
Audit is inspectable, but its own health model should be more explicit.

ALC-16 Operator clarity

Score: 2/2

Pass.

Audit is comparatively legible.

An operator should already be able to understand:

- what action occurred
- who performed it
- what target was affected
- when it occurred
- what outcome was recorded
- how records can be queried or exported

Main weakness:

- operators should more easily see impact grouping, approval linkage, and blast-radius association for high-risk actions

---

What Audit is already good at

Its strongest traits are:

1. Clear accountability role

Audit is easy to place in the architecture and clearly distinct from telemetry and maintenance.

2. Strong contract shape

Actor/action/target/outcome, append-only rules, integrity checks, and export/redaction workflow are exactly the right primitives.

3. Good governance maturity

Compared with several other libraries, Audit already behaves like a serious governed surface.

4. Strong fit for maintenance and integrity control

Because it is structured and index-based, it is naturally auditable.

What is holding Audit back

1. Dependency graphing is too light

This is the biggest issue.

Audit should explicitly know which producers emit which action classes and which governance consumers depend on which audit record classes.

2. Change control needs more rigor

Audit schema, index, and retention changes should trigger stronger impact analysis.

3. Backcompat should be more explicit

Older audit records need stable interpretation and query guarantees.

4. Observability could be stronger

Audit health should be visible as a first-class operating surface.

Priority fixes for Audit

AUD-01

Create a richer **audit registry** for every active audit unit:

- audit unit ID
- title
- version
- owner
- lifecycle state
- record class
- producer classes
- retention class
- export/redaction policy refs
- compatibility label

AUD-02

Add an **audit decision/report** contract:

- why logged
- producer source
- actor class
- target class
- integrity checks applied

- retention bucket
- export/redaction eligibility
- linked approval/blast-radius refs

AUD-03

Add explicit dependency mappings:

- action class → producing library
- action class → required audit record class
- audit record → policy/approval consumer
- audit record → maintenance/investigation consumer
- audit record → export/redaction workflow

AUD-04

Add compatibility labels for audit changes:

- schema-compatible
- query-compatible
- export-compatible
- breaking
- migration-required

AUD-05

Add audit drift checks:

- missing actor on mandatory action

- monotonic timestamp violation
- index drift
- redaction/export mismatch
- orphaned target reference
- deprecated audit record class still produced

AUD-06

Add audit health metrics:

- missing actor rate
- timestamp violation count
- stale index count
- redaction lag
- export failure count
- orphaned target reference count

Final judgment on Audit

Audit is one of the stronger support libraries in Axion.

It already has the right primitives and solid governance discipline.
Its next leap is to become the explicit **mutation accountability and impact-trace authority** across the estate.

That is why it lands at **84%**.

Kit library scorecard

Kit library scorecard

Overall grade

26 / 32 = 81%
Band: Usable but thin

Kit is a strong downstream library. It already looks like a real packaging and release surface, not a loose export folder.

That matters because Kit should be the place where Axion answers:

- what gets packaged
- what the release tree must contain
- what versions are pinned into the package
- what entrypoints and manifests define the deliverable
- what proof or readiness class the package satisfies
- whether the output is reproducible and export-safe

Kit is not weak. Its main limitation is that it still needs tighter linkage to upstream compatibility, proof, and release-grade governance so it becomes the **authoritative delivery envelope** for the whole estate.

ALC score breakdown — Kit

ALC-01 Identity

Score: 2/2

Pass.

Kit appears to have:

- stable section structure
- manifest and tree concepts
- entrypoint and export rules
- version-aware packaging behavior
- versionable assets

This is enough to count as a real governed library.

Main weakness:

- per-kit-class owner/lifecycle metadata should be more explicit

---

ALC-02 Purpose and scope

Score: 2/2

Pass.

The purpose is clear:

- define packaging structure
- define release/export contents
- define manifest expectations
- define entrypoints
- define kit-level compatibility and readiness expectations

That boundary is distinct from:

- **Templates** producing documents
- **Verification** proving sufficiency

- **System** resolving runtime inputs
- **Maintenance** auditing estate health
- **Orchestration** managing stage execution

This is one of Kit's strengths.

ALC-03 Lifecycle

Score: 1/2

Partial.

What exists:

- kits are clearly versionable
- manifests and release trees imply durable packaging evolution
- export surfaces are structured enough for controlled change

What is missing:

- no rich visible lifecycle doctrine for kit units:
    - candidate
    - active
    - certified
    - deprecated
    - superseded
    - retired
- no strong replacement path for old kit layouts or entrypoint contracts

- no explicit archival/legacy support policy for prior kit generations

Main issue:
Kit is release-facing, so lifecycle should be more explicit than it appears now.

ALC-04 Authority and trust tier

Score: 1/2

Partial.

Kit is not as inherently authoritative as Policy or Canonical, but it still carries operational authority because it defines the official deliverable shape.

What exists:

- kit manifests and export rules imply binding structure
- entrypoint expectations give the package real downstream force

What is missing:

- no explicit distinction between:
    - required production kit
    - release-candidate kit
    - internal/dev kit
    - experimental/export-limited kit
- no clear authority or certification tier model

Main issue:
Without release-class tiers, Kit cannot fully express delivery readiness.

ALC-05 Applicability

Score: 2/2

Pass.

Kit applicability is naturally strong because it is keyed to:

- run/output profile
- release/export mode
- packaging target
- included artifact class
- entrypoint expectations

This is stronger than in Templates, Knowledge, and Ops.

Main weakness:

- applicability should be more explicit for dev/internal/export-limited/certified package variants

---

ALC-06 Dependency mapping

Score: 1/2

Partial.

What exists:

- Kit clearly depends on rendered outputs, manifests, versions, and likely proof/release signals from Verification
- downstream consumers are likely operators, export flows, deployment surfaces, or archive/release systems

What is missing:

- each kit class should explicitly declare:

    - required included artifact classes
    - linked template/render sources
    - linked standards snapshot
    - linked proof bundle requirements
    - linked compatibility constraints
    - downstream consumer class
- no strong first-class upstream/downstream dependency graph yet

Main issue:
Kit is the convergence point of many libraries, so dependency mapping should be richer than it appears now.

ALC-07 Resolution rules

Score: 2/2

Pass.

This is a strength.

Why:

- packaging is inherently deterministic
- manifests and tree rules imply explicit selection behavior
- export behavior appears governed rather than ad hoc

Main weakness:

- kit decision reports should explain more clearly why specific items were included, excluded, or downgraded by release class

---

ALC-08 Structure and schema

Score: 2/2

Pass.

Kit appears to have real governance structure:

- manifest
- tree/layout
- entrypoint rules
- export rules
- version behavior

That is a strong contract base.

ALC-09 Proof requirements

Score: 2/2

Pass.

Kit is already close to proof because:

- package manifests are evidence-bearing
- inclusion rules can be verified
- export/readiness behavior can be tied to upstream proof bundles
- release quality can be reflected in kit completeness

This is stronger than several other libraries.

Main weakness:

- each kit class should explicitly declare the minimum proof bundle or release-grade evidence required

---

ALC-10 Change control

Score: 1/2

Partial.

What exists:

- kit manifests and trees are structured enough to support controlled revision
- export rules and entrypoints can be versioned

What is missing:

- no explicit mutation doctrine for Kit:
    - add artifact class
    - revise manifest schema
    - change tree layout
    - change entrypoint contract
    - deprecate kit class
    - certify/promote kit class
- no formal blast-radius rule for package contract changes

Main issue:
Kit changes can break downstream consumers and should be mutation-governed more tightly.

ALC-11 Backcompat

Score: 1/2

Partial.

What exists:

- manifests and versions support historical consistency in principle
- kits can likely be associated with the package structure active at build time

What is missing:

- no explicit compatibility labels such as:
    - layout-compatible
    - manifest-compatible
    - entrypoint-compatible
    - breaking
    - migration-required
- no clear doctrine for legacy kit consumers when structure changes
- no formal release-grade compatibility model

Main issue:
Kit backcompat matters because it defines what downstream consumers expect to receive.

ALC-12 Registry quality

Score: 2/2

Pass.

Kit appears to have enough structure to support strong registry/index behavior for kit classes, manifests, entrypoints, and export rules.

This is stronger than Templates, Knowledge, Gates, and Ops.

Main weakness:

- registry richness should include lifecycle state, release class, proof requirements, and compatibility status

---

ALC-13 Drift resistance

Score: 2/2

Pass.

Kit is well positioned for drift resistance because:

- manifest contents can be diffed
- tree/layout rules can be compared
- entrypoint behavior can be audited
- maintenance can inspect package schema and registry drift

This is one of the stronger downstream libraries for drift readiness.

Main weakness:

- drift checks should explicitly catch silent layout changes, manifest-field drift, and deprecated kit classes still being produced

---

ALC-14 Maintenance readiness

Score: 2/2

Pass.

Kit is highly suitable for maintenance governance through:

- **Coverage Audit**

- **Drift Detection**
- **Backcompat Validation**
- **Rebuild Indexes/Registries**
- **Registry Integrity Cleanup**

Because it is structured, versioned, and downstream-critical, it is a strong maintenance target.

ALC-15 Observability

Score: 1/2

Partial.

What exists:

- manifests and export outputs create meaningful inspectability
- package completeness and inclusion behavior should be measurable
- release/export outcomes can be audited

What is missing:

- no obvious first-class kit health metrics such as:
    - incomplete package count
    - missing-entrypoint count
    - export failure rate
    - deprecated kit class usage
    - incompatible consumer hit count
    - stale manifest count

Main issue:
Kit is inspectable, but its own operational health model is still thinner than it should be.

ALC-16 Operator clarity

Score: 2/2

Pass.

Kit is comparatively legible.

An operator should already be able to understand:

- what the package contains
- what the manifest records
- what entrypoint is expected
- what export rules apply
- which versions are included

Main weakness:

- operators should more easily see release class, proof status, and compatibility expectations per kit

---

What Kit is already good at

Its strongest traits are:

1. Clear delivery role

Kit is easy to place in the architecture and clearly distinct from upstream content libraries.

2. Strong contract shape

Manifest, tree, entrypoint, and export rules are the right primitives.

3. Good governance maturity

Compared with Templates, Knowledge, Gates, and Ops, Kit already behaves like a serious governed surface.

4. Strong fit for maintenance and release discipline

Because it is structured and downstream-critical, it is naturally auditable.

What is holding Kit back

1. Dependency graphing is too light

This is the biggest issue.

Kit should explicitly know which upstream artifacts, standards snapshots, proof bundles, and compatibility classes are required for each kit class.

2. Release/readiness tiering is under-expressed

Kit needs stronger distinctions between dev, candidate, certified, and enterprise-ready outputs.

3. Backcompat needs to be more explicit

Package consumers need clear compatibility guarantees.

4. Observability could be stronger

Kit health should be visible as a first-class operating surface.

Priority fixes for Kit

KIT-01

Create a richer **kit registry** for every active kit unit:

- kit ID
- title
- version
- owner
- lifecycle state
- release class
- required artifacts
- entrypoint refs
- proof bundle refs
- compatibility label

KIT-02

Add a **kit decision report**:

- why this kit class applied
- included artifact classes
- excluded artifact classes
- entrypoint selected
- manifest version used

- proof status
- release/readiness note

KIT-03

Add explicit dependency mappings:

- kit → required rendered outputs
- kit → required standards snapshot
- kit → required verification bundle
- kit → export/consumer class
- kit → compatibility constraints

KIT-04

Add compatibility labels for kit changes:

- layout-compatible
- manifest-compatible
- entrypoint-compatible
- breaking
- migration-required

KIT-05

Add kit drift checks:

- manifest changed without version movement
- layout drift

- deprecated kit class still produced
- missing required artifact class
- stale proof requirement mapping

KIT-06

Add kit health metrics:

- incomplete package count
- export failure rate
- missing entrypoint count
- deprecated class usage
- stale manifest count
- incompatible consumer count

Final judgment on Kit

Kit is a strong downstream library with real release value.

It already has the right structural primitives and good packaging discipline.
Its next leap is to become the fully explicit **delivery envelope authority** for release class, compatibility, and proof-backed packaging.

That is why it lands at **81%**.

Overall estate summary

Overall estate summary

Current estate grade

Axion's library estate is **structurally strong but unevenly governed**.

It already has a real governance backbone. This is not a loose doc system. The stronger libraries already show:

- clear boundaries
- real schemas/contracts
- deterministic intent
- registry/index behavior
- maintenance compatibility
- evidence/proof awareness

The estate's problem is not lack of architecture.
The problem is **governance consistency**.

Some libraries already behave like governed runtime authorities.
Others still behave more like structured content stores or thin contract shells.

Final score table

Strongest libraries

These are closest to fully governed.

- **System** — 88%
- **Canonical** — 88%

- **Standards** — 84%
- **Verification** — 84%
- **Orchestration** — 84%
- **Intake** — 84%
- **Audit** — 84%

Strong but still thin

These are solid, but need clearer downstream linkage, lifecycle, and observability depth.

- **Maintenance** — 81%
- **Planning** — 81%
- **Kit** — 81%
- **Telemetry** — 78%
- **Policy** — 75%

At-risk libraries

These are the main structural weak points.

- **Gates** — 72%
- **Templates** — 69%
- **Knowledge** — 66%
- **Ops** — 56%

What the estate is best at

1. Contract-first design

Axion is strongest where libraries define:

- schemas
- registries
- stage/report structures
- deterministic selection/resolution behavior

That is why System, Canonical, Standards, Verification, and Orchestration score well.

2. Maintenance compatibility

A lot of the estate is already suitable for:

- coverage audit
- drift detection
- backcompat validation
- registry rebuild
- registry integrity cleanup

That is a major strength.

3. Upstream/downstream architecture

The major layers are conceptually correct:

- Intake
- Canonical
- Standards

- Planning
- Orchestration
- Gates
- Verification
- Kit

The shape is good.

What the estate is weakest at

1. Governance density at the unit level

This is the biggest estate-wide issue.

Many libraries have a strong top-level structure but weaker per-unit governance:

- lifecycle
- owner
- applicability
- dependencies
- proofs
- compatibility labels
- supersession

This is most visible in:

- Templates

- Knowledge
- Ops
- Gates

2. Cross-library dependency graphing

Almost every library needs stronger first-class mappings for:

- what it consumes
- what it constrains
- what it produces
- what depends on it
- what breaks if it changes

This is the single biggest cross-estate weakness.

3. Backcompat discipline

Versioning exists in many places, but explicit compatibility semantics are often thin or missing.

The estate needs more universal labels like:

- compatible
- breaking
- migration-required
- rerun-required
- rebuild-required

4. Self-observability

Many libraries are inspectable, but fewer are truly observable as governed assets.

You need more first-class health metrics per library.

Final priority ranking

This is the order I would upgrade the libraries in if the goal is to improve the whole estate as fast as possible.

Tier 1 — highest return, highest risk

These should be upgraded first.

1. Ops

Why first:

- lowest score
- directly impacts enterprise readiness
- currently too thin for its importance
- governs performance, alerting, SLOs, tracing, cost, and reliability

2. Knowledge

Why second:

- huge influence surface
- trust/freshness/applicability are under-governed
- easy source of drift and retrieval quality problems

3. Templates

Why third:

- major output surface
- content scale is ahead of governance density
- changes here impact many downstream artifacts quickly

4. Gates

Why fourth:

- enforcement authority is under-governed
- gate drift is dangerous
- operator explainability and compatibility need strengthening

---

Tier 2 — governance spine reinforcement

These should be next because they govern how the rest of the estate behaves.

5. Policy

Needs stronger decision reporting, drift control, and compatibility discipline.

6. Maintenance

Needs to become the explicit meta-governor of doctrine compliance across all libraries.

7. Telemetry

Needs stronger producer/consumer graphing, lifecycle, and signal compatibility semantics.

8. Kit

Needs stronger release-class, compatibility, and proof-bundle linkage.

Tier 3 — high-quality libraries that need deepening

These are already strong, but should be upgraded after the weaker surfaces.

9. Planning

Main gap is deeper traceability from obligation → work → proof.

10. Audit

Main gap is mutation impact linkage and richer producer/consumer mapping.

11. Standards

Main gap is stronger downstream bindings into templates, gates, proofs, and policy.

12. Verification

Main gap is becoming the universal proof authority across the whole estate.

13. Orchestration

Main gap is explicit execution dependency and invalidation graphing.

14. Intake

Main gap is submission provenance and downstream impact visibility.

15. Canonical

Main gap is explicit downstream invalidation and provenance graphing.

16. System

Main gap is becoming the formal resolver authority for the full estate.

Best upgrade sequence by phase

Phase 1 — stop the biggest governance leaks

Do these first:

- Ops
- Knowledge
- Templates
- Gates

This phase fixes the weakest and riskiest surfaces.

Phase 2 — strengthen estate control

Then:

- Policy
- Maintenance
- Telemetry
- Kit

This phase strengthens estate-wide governance, visibility, and delivery discipline.

Phase 3 — deepen the already-strong core

Then:

- Planning
- Audit
- Standards
- Verification
- Orchestration
- Intake
- Canonical

- System

This phase turns the strong core into a truly governed core.

The main strategic conclusion

Axion does **not** need a new architecture.

It needs:

- richer unit-level governance
- stronger dependency graphing
- explicit compatibility semantics
- better per-library health metrics
- stronger trust/freshness controls for knowledge surfaces
- stronger enforcement rigor in gates
- a real ops doctrine
- maintenance acting as the full estate governor

That is the real path from “well-structured system” to “enterprise-grade governed estate.”